



INSTITUT SUPÉRIEUR DES ÉTUDES TECHNOLOGIQUES DE NABEUL
المعهد العالي للدراسات التكنولوجية بنابل

Département Technologies de l'Informatique

Code : DSI21/25

PARCOURS

***DEVELOPPEMENT SYSTEMES
D'INFORMATIONS***

Projet de Fin de Parcours

INTITULE

**PLATEFORME INTELLIGENTE DE
DIMENSIONNEMENT
ET DE PILOTAGE COMMERCIAL SOLAIRE
"SOLAREASE"**

ORGANISME D'ACCUEIL

X&V ELECTRONICS

REALISE PAR

Louay JDAY

ENCADRANT ACADEMIQUE

Waheb aymen

ENCADRANT PROFESSIONNEL

hawech Iheb

ANNEE UNIVERSITAIRE 2025 / 2026

Dédicace

À mes très chers parents

Aucun mot, aussi profond soit-il, ne saurait exprimer à sa juste valeur la gratitude et l'amour que je vous porte. Vous avez été pour moi un refuge constant, une boussole dans les moments de doute et une source inépuisable de motivation. Vos sacrifices innombrables, votre patience infinie et votre soutien inconditionnel ont été le socle de ma réussite. Ce travail est le modeste couronnement de votre dévouement.

À ma très chère famille

En témoignage de l'affection profonde qui nous lie. Merci pour votre présence rassurante, vos encouragements et votre foi inébranlable en mes capacités. Votre soutien m'a permis de surmonter les obstacles tout au long de ce parcours.

À mes formidables amis et collègues d'études

Pour les heures de travail, les moments de complicité et d'entraide partagés. La fraternité et la solidarité qui nous ont unis ont été une véritable force motrice pour atteindre ce but.

À l'ensemble de mes enseignants et formateurs

Pour la noblesse de votre mission, la qualité exceptionnelle de vos enseignements et le temps que vous nous avez accordé avec bienveillance. Ce projet porte l'empreinte de la transmission de vos connaissances.

À toutes les personnes qui, de près ou de loin, ont cru en moi.

Louay JDAY

Remerciements

Nos toutes premières pensées et louanges s'élèvent vers **Allah**, le Tout-Puissant, pour nous avoir accordé la santé, la patience et la détermination nécessaires pour surmonter les défis et mener à bien ce travail.

Nous tenons à adresser nos remerciements les plus sincères à notre **Aymen Waheb**, Monsieur [À renseigner – nom encadrant ISET]. Ses conseils judicieux, son suivi rigoureux, ainsi que ses encouragements continus ont été des piliers majeurs dans l'aboutissement de ce projet. Nous lui sommes profondément reconnaissants pour le temps précieux qu'il nous a consacré.

Nous exprimons également notre vive reconnaissance à notre **encadrant professionne**, Monsieur/Madame Iheb Hawech, pour son accueil exceptionnel au sein de **XV Electronics**. Son expertise inestimable, sa disponibilité constante et la confiance qu'il/elle nous a accordée ont fait de cette collaboration une expérience hautement formatrice et stimulante.

Nous souhaitons témoigner notre entière gratitude à l'ensemble du corps professoral et administratif du **Département Technologies de l'Informatique** de l'ISET Nabeul. Nous les remercions pour la qualité remarquable de l'enseignement dispensé, leur dévouement exemplaire et l'environnement d'excellence qu'ils nous ont offert tout au long de notre cursus universitaire.

Enfin, nous ne saurions accomplir cette œuvre sans y associer notre merveilleuse **famille**. Votre amour inconditionnel, vos immenses sacrifices et votre soutien moral infaillible ont été de véritables sources de motivation. Vous êtes le socle de notre réussite. Nos remerciements s'étendent naturellement à l'ensemble de nos **amis et collègues** pour leur solidarité inébranlable et les beaux moments partagés.

Louay JDAY

Sommaire

Dédicace	i
Remerciements	ii
Liste des figures	viii
Liste des tableaux	x
Liste des Abréviations	xi
Introduction Générale	1
1 Cadre Général et Étude de l’Existant	5
1.1 Présentation de l’Organisme d’Accueil	5
1.1.1 Informations générales	5
1.1.2 Mission, vision et valeurs	6
1.1.3 Domaine d’activité	6
1.1.4 Organisation interne	7
1.2 Contexte et Enjeux du Projet	8
1.2.1 Le marché photovoltaïque en Tunisie	8
1.2.2 Les enjeux opérationnels	9
1.3 Problématique et Objectifs	9
1.3.1 Problématique	9
1.3.2 Objectifs du projet	9
1.4 Analyse Critique de l’Existant	10
1.4.1 Outils actuellement utilisés	10
1.4.2 Tableau comparatif des solutions existantes	11
1.4.3 Conclusion de l’analyse	11
1.5 Positionnement et Apport de SolarEase	11
1.6 Méthodologie de Travail	12
1.6.1 Les méthodes agiles	12
1.6.2 La méthodologie Kanban	12
2 Spécification des besoins et architecture	14
2.1 Identification des acteurs	14

2.2	Besoins fonctionnels	15
2.2.1	<u>Cas d'utilisation global</u>	15
2.2.2	Description textuelle des cas d'utilisation principaux	15
2.2.3	Tableau des besoins fonctionnels	18
2.3	Besoins non fonctionnels	19
2.4	Pilotage du projet avec Kanban	19
2.4.1	Équipe et rôles	19
2.4.2	Organisation du flux Kanban	20
2.4.3	Product Backlog	20
2.4.4	Planification des itérations	21
2.5	Architecture technique globale	22
2.5.1	Choix de l'architecture microservices	22
2.5.2	Schéma d'architecture globale	23
2.5.3	Flux de sécurité	23
2.5.4	Stack technique	24
2.5.5	Modèle de données globale	25
3	Release 1 : Fondation, Fonctionnalités Métier et Dimensionnement	27
3.1	Itération 1 : Service d'Identité et Authentification	27
3.1.1	Backlog de l'itération	27
3.1.2	Analyse de l'itération 1	28
3.1.3	Conception de l'itération 1	32
3.1.4	Réalisation de l'itération 1	34
3.2	Itération 2 : Gestion des Clients et des Projets	37
3.2.1	Backlog de l'itération	37
3.2.2	Analyse de l'itération 2	37
3.2.3	Conception de l'itération 2	41
3.2.4	Réalisation de l'itération 2	45
3.3	Itération 3 : Service de Dimensionnement Solaire	49
3.3.1	Backlog de l'itération	49
3.3.2	Analyse de l'itération 3	50
3.3.3	Conception de l'itération 3	54
3.3.4	Réalisation de l'itération 3	56
3.3.5	Environnement de l'itération 3	60
3.3.6	Approche RAG (Retrieval-Augmented Generation)	61
4	Release 2 : Collaboration, Espace Client et Facturation	66
4.1	Itération 4 : Suivi Terrain et Communication Temps Réel	66
4.1.1	Backlog de l'itération	66
4.1.2	Analyse de l'itération 4	67

4.1.3	Conception de l'itération 4	71
4.1.4	Réalisation de l'itération 4	72
4.2	Itération 5 : Espace Client, Demandes et Facturation	76
4.2.1	Backlog de l'itération	76
4.2.2	Analyse de l'itération 5	77
4.2.3	Conception de l'itération 5	81
4.2.4	Réalisation de l'itération 5	84
4.2.5	Environnement de la Release 3	88
5	DevOps et mise en production	91
5.1	Démarche et architecture DevOps cible	91
5.1.1	Principes retenus	91
5.1.2	Vue d'ensemble de la chaîne	92
5.2	Containerisation	92
5.2.1	Stratégie multi-stage	92
5.2.2	Topologie du docker-compose unifié	93
5.3	Pipeline CI/CD GitHub Actions	93
5.3.1	Workflows mis en place	93
5.3.2	Workflow CI backend – exécution en matrice	94
5.3.3	Étapes du pipeline CI/CD	95
5.3.4	Gestion des dépendances avec Dependabot	95
5.4	Observabilité (Prometheus + Grafana + Loki)	95
5.4.1	Exposition des métriques applicatives	95
5.4.2	Architecture de la stack observabilité	96
5.4.3	Stack déployée via Docker Compose	96
5.5	Qualité et sécurité du code	97
5.5.1	Couverture de tests backend – JaCoCo	97
5.5.2	Scans de sécurité	97
5.5.3	Gestion des secrets	97
5.6	Infrastructure as Code (Kubernetes + Kustomize)	97
5.6.1	Organisation des manifests	97
5.6.2	Sondes Actuator et limites de ressources	98
5.6.3	Routage Ingress	98
5.6.4	Overlays dev / prod	99
5.7	Stratégie de déploiement multi-environnements	99
5.7.1	Modèle de branches (GitFlow simplifié)	99
5.7.2	Environnements et URLs	100
5.7.3	Procédure de release	100
5.7.4	Procédure de rollback	100

5.8 Tests de validation DevOps	101
Conclusion Générale et Perspectives	102
Bibliographie et Netographie	105
A Récapitulatif Complet des Endpoints API	107
B Matrice de Tests Complète	109
C Configuration Docker Compose	111
Résumé	113

Table des figures

1.1	Organigramme de XV Electronics	7
2.1	Diagramme de cas d'utilisation global de SolarEase	15
2.2	Timeline du projet SolarEase (5 itérations Kanban)	22
2.3	Architecture microservices de SolarEase	23
2.4	Diagramme de classes globale de SolarEase	25
3.1	Diagramme cas d'utilisation raffiné de l'itération 1	29
3.2	Diagramme de séquence — Inscription et authentification (client)	33
3.3	Interface d'authentification – SolarEase	36
3.4	Interface de vérification OTP – Itération 1	36
3.5	Interface d'authentification	38
3.6	Diagramme de cas d'utilisation global – Itération 2	38
3.7	Diagramme de séquence — Affectation d'un projet par l'admin	42
3.8	Diagramme états-transitions d'un projet SolarEase (6 statuts)	43
3.9	Diagramme de classes – Itération 2	44
3.10	Tableau de bord administrateur – Itération 2	46
3.11	Page de gestion des clients (admin) – Itération 2	47
3.12	Interface admin de création de projet – Itération 2	47
3.13	Diagramme de cas d'utilisation global relatif à l'itération 3	50
3.14	Diagramme de séquence – Dimensionnement et recommandations IA	54
3.15	Diagramme de séquence – Génération du rapport PDF	55
3.16	Formulaire de saisie de consommation – Itération 3	58
3.17	Résultats de dimensionnement – Itération 3	58
3.18	Recommandations IA embarquée – Itération 3	59
3.19	Rapport PDF généré automatiquement – Itération 3	59
3.20	Diagramme de séquence du pipeline RAG	62
4.1	Diagramme de cas d'utilisation global – Itération 4	67
4.2	Diagramme de séquence – Avancement terrain et notification admin	71
4.3	Diagramme de séquence – Messagerie projet avec notifications	71
4.4	Interface de suivi terrain installateur – Itération 4	74
4.5	Messagerie projet temps réel – Itération 4	74
4.6	Centre de notifications – Itération 4	74

4.7	Diagramme de cas d'utilisation global – Itération 5	77
4.8	Diagramme de séquence – Cycle complet du devis (création, envoi, acceptation)	81
4.9	Automate d'états d'un devis SolarEase	82
4.10	Automate d'états d'une demande solaire	83
4.11	Tableau de bord client – Itération 5	86
4.12	Détail projet espace client avec stepper – Itération 5	86
4.13	Gestion des devis – Itération 5	87
4.14	Acceptation de devis par le client – Itération 5	87
4.15	Gestion des demandes solaires – Itération 5	87
4.16	Liste des factures – Itération 5	88
5.1	Architecture DevOps cible – Vue globale	92
5.2	Topologie Docker Compose – 4 microservices backend + Ollama + n8n	93
5.3	Pipeline CI/CD – Étapes séquentielles du commit au déploiement	95
5.4	Architecture observabilité – Prometheus + Loki + Grafana	96
5.5	Stratégie GitFlow et environnements automatiques	99

Liste des tableaux

1.1	Fiche d'identité de XV Electronics	5
1.2	Rôles et responsabilités par pôle	8
1.3	Enjeux opérationnels des installateurs photovoltaïques	9
1.4	Comparaison des solutions existantes	11
2.1	CU-01 : Inscription et authentification	16
2.2	CU-02 : Création d'un projet solaire	16
2.3	CU-03 : Dimensionnement photovoltaïque (PVGIS)	17
2.4	Besoins fonctionnels principaux	18
2.5	Besoins non fonctionnels	19
2.6	Rôles et responsabilités de l'équipe	20
2.7	Product Backlog SolarEase	21
2.8	Planification des itérations Kanban	22
2.9	Stack technique de SolarEase	24
3.1	Backlog Kanban – Itération 1 (Identity Service, Port 8081)	27
3.2	Description textuelle du CU01 — Inscription	30
3.3	Description textuelle du CU01b — Vérification OTP	31
3.4	Description textuelle du CU01c — Création de compte installateur	31
3.5	Description textuelle du CU02 — Connexion	32
3.6	Diagramme de classes – Itération 1 (Identity Service)	34
3.7	Résultats des tests fonctionnels – Itération 1	35
3.8	Backlog Kanban – Itération 2 (Project Service, Port 8082)	37
3.9	Description textuelle du CU04 — Création d'une fiche client	39
3.10	Description textuelle du CU05 — Affectation d'un projet par l'admin	40
3.11	Description textuelle du CU05b — Gestion du statut	41
3.12	Diagramme de classes – Itération 2 (Project Service)	45
3.13	Résultats des tests fonctionnels – Itération 2	48
3.14	Description textuelle du CU08 — Lancer le dimensionnement	51
3.15	Description textuelle du CU10 — Recommandations IA	52
3.16	Description textuelle du CU11 — Génération PDF	53
3.17	Itération 3 (Dimensioning Service)	56
3.18	Technologies et outils utilisés – Itération 3	60
3.19	Résultats des tests fonctionnels – Itération 3	60
3.20	Structure de la base de connaissances RAG	62
4.1	Description textuelle du CU14 — Avancement terrain	68
4.2	Description textuelle du CU15 — Messagerie projet	69
4.3	Description textuelle du CU16 — Notifications temps réel	70
4.4	– Itération 4 (Collaboration)	72
4.5	Résultats des tests fonctionnels – Itération 4	75
4.6	Description textuelle du CU19 — Demande solaire	78
4.7	Description textuelle du CU21 — Cycle de vie du devis	79
4.8	Description textuelle du CU23 — Espace client	80
4.9	Diagramme de classes – Itération 5 (Commercial & Client)	84
4.10	Technologies et outils utilisés – Release 3	88
4.11	Résultats des tests fonctionnels – Itération 5	89
5.1	Workflows GitHub Actions du projet SolarEase	94

5.2	Environnements de déploiement de SolarEase	100
5.3	Résultats des tests de validation DevOps	101
5.4	Bilan des objectifs du projet SolarEase	102
A.1	Endpoints Identity Service	107
A.2	Endpoints Project Service	108
A.3	Endpoints Dimensioning Service	108
B.1	Matrice de tests SolarEase	109

Liste des Abréviations

Sigle	Signification
API	Application Programming Interface
B2B	Business to Business
B2C	Business to Consumer
CI/CD	Intégration continue / Déploiement continu
CRUD	Create, Read, Update, Delete
DB	Database (Base de données)
DTO	Data Transfer Object
DSI	Développement des Systèmes d'Information
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
IaC	Infrastructure as Code
IoT	Internet of Things
SET	Institut Supérieur des Études Technologiques
JSON	JavaScript Object Notation
JWT	JSON Web Token
K8s	Kubernetes
kWh	Kilowatt-heure
kWp	Kilowatt-crête
LLM	Large Language Model
MVP	Minimum Viable Product
OTP	One-Time Password
PDF	Portable Document Format
PFE	Projet de Fin d'Études
PV	Photovoltaïque
PVGIS	Photovoltaic Geographical Information System
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
ROI	Return on Investment
SaaS	Software as a Service
SMTP	Simple Mail Transfer Protocol
SOLID	Principes de conception logicielle orientée objet
SQL	Structured Query Language
STEG	Société Tunisienne de l'Électricité et du Gaz
TND	Dinar Tunisien
UI	User Interface

Introduction Générale

*Ce rapport présente le travail réalisé dans le cadre du Projet de Fin d'Études (PFE) pour l'obtention de la Licence en **Développement des Systèmes d'Information** à l'ISET Nabeul, année universitaire 2024/2025. Il documente la conception et la réalisation complète de la plateforme **SolarEase**, depuis l'analyse des besoins jusqu'au déploiement.*

Contexte et motivation

La transition énergétique figure parmi les enjeux stratégiques majeurs du XXI^e siècle. En Tunisie, la filière photovoltaïque connaît une croissance soutenue, portée par deux dynamiques convergentes : d'une part, les objectifs nationaux d'efficacité énergétique inscrits dans le *Plan Solaire Tunisien* (30 % d'énergies renouvelables à l'horizon 2030) ; d'autre part, la baisse continue des coûts des modules solaires qui rend cette technologie accessible à un public de plus en plus large — résidentiel, commercial et industriel.

Des centaines de sociétés d'installation photovoltaïque opèrent aujourd'hui sur le territoire tunisien. Ces acteurs jouent un rôle clé dans la concrétisation de la transition énergétique, mais peinent encore à industrialiser leurs processus internes : dimensionnement, suivi client, facturation et génération de rapports techniques.

Problématique

Une analyse de terrain révèle que la majorité de ces installateurs s'appuient sur des outils hétérogènes et non intégrés : tableurs manuels pour le dimensionnement, logiciels isolés pour la simulation, courriers électroniques pour la relation client et documents bureautiques pour les devis. Cette fragmentation engendre :

- des erreurs de calcul lors du dimensionnement ;
- une perte de temps considérable sur les tâches répétitives ;
- l'absence de traçabilité des projets ;
- une image peu professionnelle vis-à-vis du client final.

Problématique centrale : *Comment concevoir et réaliser une plateforme web unifiée permettant aux installateurs photovoltaïques de gérer l'intégralité de leur cycle métier — du premier contact client jusqu'au rapport de dimensionnement — en s'appuyant sur des données fiables (PVGIS) et des recommandations intelligentes (IA), tout en garantissant sécurité, scalabilité et évolutivité ?*

Solution proposée : SolarEase

Pour répondre à cette problématique, nous proposons **SolarEase** — une plateforme web intelligente dédiée aux installateurs photovoltaïques. Elle centralise l'intégralité du cycle de vie d'un projet solaire au sein d'une interface unique, depuis la gestion du client jusqu'à la remise d'un rapport PDF professionnel, en passant par le calcul de dimensionnement et l'analyse financière sur 25 ans.

SolarEase se distingue par trois innovations majeures :

1. **Données réelles PVGIS** : les calculs de production solaire s'appuient sur l'API officielle de la Commission Européenne (données d'irradiance géolocalisées), garantissant une précision scientifique ;
2. **Mode Night Panel** : une fonctionnalité inédite permettant de simuler et comparer des installations avec stockage batterie intégré, incluant le calcul de la couverture nocturne et du taux d'autoconsommation ;
3. **Intelligence Artificielle locale** : des recommandations personnalisées générées par un LLM (Ollama / TinyLLaMA) déployé localement, sans dépendance à des services cloud externes.

Méthodologie et organisation du rapport

Le projet a été mené selon la méthodologie **Kanban**, offrant un flux continu de développement piloté visuellement via **Jira**. Le travail est découpé en **cinq sprints** itératifs de deux semaines chacun, permettant des livraisons incrémentales et une validation régulière avec l'encadrant.

Le présent rapport est structuré comme suit :

Structure du rapport

Section	Contenu
Chapitre 1 — Cadre général	Présentation de l'organisme d'accueil, problématique, objectifs, analyse critique de l'existant et choix méthodologique
Chapitre 2 — Besoins & Architecture	Spécification des besoins, pilotage Kanban, Product Backlog, planification des itérations et architecture technique globale
Chapitre 3 — Release 1	Fondation technique (authentification JWT/OTP, Gateway), gestion des clients et projets, dimensionnement solaire (PVGIS, Night Panel), approche RAG pour recommandations IA, génération PDF
Chapitre 4 — Release 2	Suivi terrain, messagerie temps réel, notifications, espace client, demandes, devis, factures et workflow commercial complet
Chapitre 5 — DevOps & mise en production	Containerisation des microservices backend, pipeline CI/CD GitHub Actions, observabilité (Prometheus, Grafana, Loki), qualité et sécurité (JaCoCo, Trivy, Gitleaks), Infrastructure as Code (Kubernetes + Kustomize) et stratégie de déploiement multi-environnements (Railway pour le backend, Vercel pour le frontend)
Conclusion générale	Bilan des réalisations, limites et perspectives d'évolution

Chapitre 1

Cadre Général et Étude de l'Existant

1

Cadre Général et Étude de l’Existant

1.1 Présentation de l’Organisme d’Accueil

1.1.1 Informations générales

XV Electronics est une jeune entreprise tunisienne fondée en **2024** par **Marwen Zitouni** et **Iheb Haouech**, co-fondateurs et co-CEO. Implantée en Tunisie, elle se positionne sur trois domaines technologiques complémentaires : le **développement logiciel**, les **solutions photovoltaïques** et les **technologies intelligentes** (domotique et objets connectés).

L’entreprise adopte une approche *full-stack* de l’innovation : de la conception logicielle jusqu’à l’intégration énergétique et la recherche & développement matérielle, en passant par le marketing et la communication digitale. Cette transversalité lui permet de proposer à ses clients des offres intégrées combinant logiciel, énergie et domotique au sein d’un même projet.

TABEAU 1.1. Fiche d’identité de XV Electronics

Élément	Description
Raison sociale	X&V Electronics (XV Electronics)
Forme juridique	Société tunisienne
Année de création	2024
Co-fondateurs (CEO)	Marwen Zitouni & Iheb Haouech
Secteurs d’activité	Développement logiciel • Solutions photovoltaïques • Domotique & objets connectés • R&D
Cible géographique	Tunisie, avec ambition régionale Afrique du Nord
Clientèle	Résidentielle, commerciale et industrielle (B2B et B2C)

1.1.2 Mission, vision et valeurs

Mission : créer des solutions technologiques innovantes et adaptables pour les entreprises locales et internationales, en combinant logiciel, énergie renouvelable et domotique au sein d'une approche intégrée entre R&D, production et marketing afin de générer des résultats durables.

Vision : devenir une référence en **Afrique du Nord** dans l'intelligence digitale, l'énergie renouvelable et l'innovation domotique en proposant des solutions sur mesure pour le marché tunisien puis régional.

Valeurs fondatrices :

- **Innovation** : prototypage de panneaux intelligents, intégration domotique et exploration de technologies émergentes (IA, IoT) ;
- **Qualité et fiabilité** : choix des meilleures marques, tests rigoureux avant déploiement, suivi maintenance ;
- **Esprit d'équipe** : collaboration étroite entre les pôles IT, énergie, R&D et marketing ;
- **Satisfaction client** : solutions adaptées aux besoins réels, accompagnement et support continu ;
- **Développement durable** : utilisation responsable de l'énergie et promotion des sources renouvelables.

1.1.3 Domaine d'activité

XV Electronics structure son offre autour de quatre départements opérationnels qui se complètent :

a) IT & Développement logiciel. Conception et réalisation de plateformes web, d'applications mobiles, de logiciels de gestion et de solutions SaaS. L'équipe développe notamment des outils d'automatisation et de scalabilité tels que *LogiFlow* (suivi logistique), une plateforme tunisienne inspirée d'Airbnb, des applications de livraison et des ERP sur mesure. **SolarEase**, le projet objet de ce rapport, s'inscrit naturellement dans cette activité IT en combinant développement logiciel et expertise métier photovoltaïque.

b) Solutions photovoltaïques. Études énergétiques, conception et installation de panneaux solaires pour résidences, usines et entreprises. Le département assure également la maintenance, l'optimisation des installations et l'intégration de systèmes intelligents (supervision à distance, pilotage de la consommation). L'engagement écologique se traduit par le choix des meilleures marques et de solutions à fort rendement énergétique.

c) R&D et innovation. Prototypage de panneaux intelligents à **production nocturne**, innovations domotiques et tests de nouvelles technologies énergétiques. C'est précisément dans ce département qu'est née la fonctionnalité **Night Panel** intégrée à SolarEase, qui simule la production résiduelle d'un panneau associé à un stockage par batterie.

d) Marketing & branding. Gestion des réseaux sociaux, identité visuelle, publicité digitale et campagnes orientées conversion. L'équipe travaille en collaboration étroite avec l'IT et la R&D pour lancer rapidement les produits commercialisables et soutenir la croissance de l'entreprise.

1.1.4 Organisation interne

L'entreprise adopte une structure simple et horizontale, adaptée à une jeune société technologique. Les deux co-fondateurs assurent conjointement la direction générale et coordonnent quatre départements opérationnels, comme l'illustre la figure 1.1.

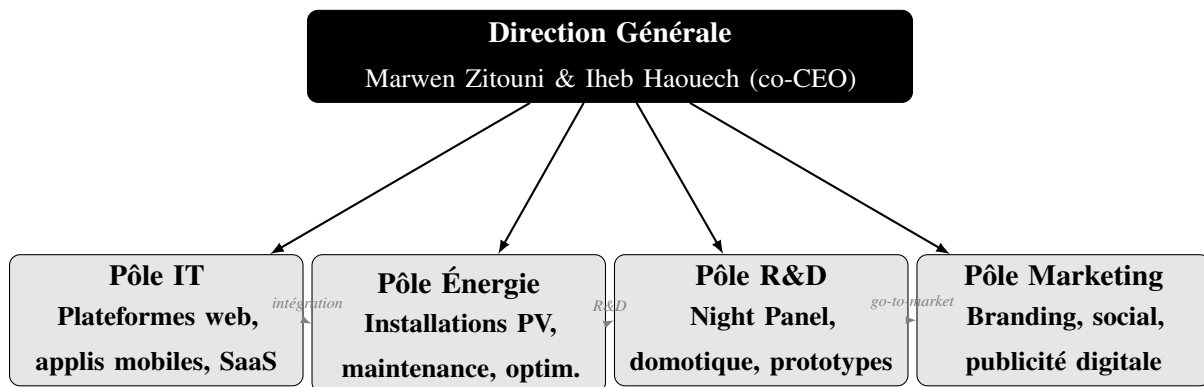


FIGURE 1.1. Organigramme de XV Electronics

TABLEAU 1.2. Rôles et responsabilités par pôle

Pôle	Responsabilités principales
Direction Générale	Pilotage stratégique, vision long terme, relations institutionnelles, validation des offres majeures et suivi financier global. Assurée conjointement par les deux co-fondateurs
Pôle IT & Développement	Conception et développement de plateformes web, applications mobiles, logiciels de gestion, ERP et solutions SaaS. Accent sur l'automatisation et la scalabilité
Pôle Énergie (Photovoltaïque)	Études énergétiques, conception, installation et maintenance de systèmes photovoltaïques pour clients résidentiels, commerciaux et industriels
Pôle R&D	Prototypage de panneaux intelligents (production nocturne), innovations domotiques, tests de nouvelles technologies énergétiques, alimentation des autres pôles en innovations commercialisables
Pôle Marketing & Branding	Gestion des réseaux sociaux, identité visuelle, publicité digitale, campagnes de conversion et accompagnement du lancement des produits

Processus décisionnel : les décisions stratégiques sont prises conjointement par les deux co-CEO en concertation avec les responsables de pôles. Les choix techniques sont validés par les pôles IT, Énergie ou R&D selon leur nature, après revue et tests.

Positionnement du projet SolarEase au sein de XV Electronics : SolarEase est un projet **transverse** qui mobilise trois pôles de l'entreprise : le pôle *IT & Développement* pour la conception et la réalisation logicielle de la plateforme, le pôle *Énergie* pour l'expertise métier (dimensionnement, choix des équipements) et le pôle *R&D* pour la fonctionnalité Night Panel. Le pôle *Marketing* est sollicité pour la communication autour du produit et son éventuelle commercialisation.

1.2 Contexte et Enjeux du Projet

1.2.1 Le marché photovoltaïque en Tunisie

Le plan solaire tunisien vise à porter la part des énergies renouvelables à **35%** de la production électrique nationale à l'horizon 2030 [17]. Dans ce contexte, le nombre de sociétés d'installation photovoltaïque a fortement progressé ces dernières années. La concurrence s'intensifie, et les sociétés doivent impérativement professionnaliser leurs processus commerciaux et techniques pour rester compétitives.

1.2.2 Les enjeux opérationnels

Les principaux défis auxquels font face les installateurs sont :

TABLEAU 1.3. Enjeux opérationnels des installateurs photovoltaïques

Enjeu	Description	Impact
Rapidité de traitement	Délai entre la demande client et la remise d'une offre	Perte de clients
Précision technique	Erreurs de dimensionnement engendrant des sur-coûts	Litiges, insatisfaction
Professionalisme	Qualité variable des devis et propositions	Image et crédibilité
Traçabilité	Difficulté à retrouver l'historique d'un dossier	Risque opérationnel
Rentabilité	Absence de projection financière claire	Mauvaise argumentation

1.3 Problématique et Objectifs

1.3.1 Problématique

Les installateurs photovoltaïques en Tunisie utilisent aujourd'hui une multitude d'outils hétérogènes et non intégrés (tableurs, simulateurs isolés, CRM génériques, documents bureautiques). Cette fragmentation engendre des erreurs de dimensionnement, des pertes de temps considérables, une absence de traçabilité et une image peu professionnelle vis-à-vis du client final.

La problématique peut être formulée ainsi :

Comment concevoir et réaliser une plateforme web unifiée permettant à un installateur photovoltaïque de gérer l'intégralité de son cycle métier — du premier contact client jusqu'au rapport de dimensionnement — en s'appuyant sur des données fiables (PVGIS) et des recommandations intelligentes (IA) ?

1.3.2 Objectifs du projet

Pour répondre à cette problématique, le projet SolarEase fixe les objectifs suivants :

1. Offrir une authentification sécurisée (JWT + OTP) avec gestion des rôles ;
2. Centraliser la gestion des clients et des projets solaires dans un seul outil ;
3. Automatiser le calcul de dimensionnement photovoltaïque à partir des données d'irradiation PVGIS ;
4. Proposer un mode *Night Panel* simulant le stockage par batterie ;
5. Fournir une analyse financière complète (ROI, payback, cash-flow sur 25 ans) ;
6. Générer automatiquement un rapport PDF professionnel et exploitable ;
7. Intégrer une recommandation IA contextuelle via un LLM local (Ollama) ;
8. Offrir un espace client dédié pour la consultation des projets et documents.

1.4 Analyse Critique de l'Existant

1.4.1 Outils actuellement utilisés

L'analyse de l'environnement de travail révèle que les sociétés s'appuient sur des outils non intégrés :

a) Tableurs (Excel/Sheets) : La plupart des installateurs utilisent des tableurs maison pour estimer la surface de toiture, le nombre de panneaux et le coût d'installation. Ces outils sont sujets aux erreurs humaines, difficiles à maintenir et non standardisés.

b) Simulateurs isolés : Des logiciels comme PVSyst ou PVGIS permettent des calculs précis mais sont complexes à prendre en main et ne sont pas reliés au processus commercial.

c) CRM généralistes : Les CRM de type Salesforce ou HubSpot offrent une bonne gestion de la relation client mais ne disposent d'aucune logique de dimensionnement solaire intégrée.

d) Documents bureautiques : Les propositions commerciales sont rédigées manuellement sous Word, sans standardisation ni calcul automatique.

1.4.2 Tableau comparatif des solutions existantes

TABLEAU 1.4. Comparaison des solutions existantes

Critère	Excel	PVSyst	CRM général	SolarEase
Dimensionnement	Partiel	Oui	Non	Oui
Gestion client	Partiel	Non	Oui	Oui
Analyse financière	Manuel	Partiel	Non	Oui
Génération PDF	Partiel	Partiel	Non	Oui
Aide IA	Non	Non	Non	Oui
Données PVGIS	Non	Oui	Non	Oui
Accès web	Partiel	Non	Oui	Oui
Coût	Faible	Élevé	Élevé	Adapté

Légende : Oui = fonctionnalité complète ; Partiel = couverture partielle ; Non = absence.

1.4.3 Conclusion de l'analyse

L'étude de l'existant révèle une lacune claire : **aucune solution ne couvre l'intégralité du cycle métier** d'un installateur photovoltaïque en une plateforme unique, accessible et abordable. SolarEase ambitionne de combler ce vide en proposant une solution unifiée, sécurisée et intelligente.

1.5 Positionnement et Apport de SolarEase

SolarEase se distingue des solutions existantes par :

- **L'intégration complète** : du premier contact client jusqu'au rapport PDF final, en passant par le dimensionnement et l'analyse financière ;
- **Le calcul basé sur des données réelles** : connexion à l'API PVGIS de la Commission Européenne pour des données d'irradiance précises ;
- **L'innovation Night Panel** : simulation et comparaison des panneaux avec stockage par batterie intégré ;
- **L'intelligence artificielle** : recommandations contextualisées générées par un LLM local (Ollama / TinyLLaMA) ;

- **L'architecture moderne** : microservices Spring Boot, frontend React TypeScript, déploiement Docker Compose.

SolarEase fusionne les volets métier et technique dans un flux unique et automatisé : de l'inscription jusqu'à l'édition d'un rapport PDF exploitable par le client final.

1.6 Méthodologie de Travail

1.6.1 Les méthodes agiles

La méthode **Agile** est une approche de gestion de projet qui privilégie la collaboration, la flexibilité et la livraison continue de valeur. Elle permet à l'équipe de s'adapter en continu aux retours et aux évolutions des besoins, sans figer un plan rigide dès le départ.

1.6.2 La méthodologie Kanban

Kanban est une méthode Agile à flux tiré, originellement développée par Toyota pour optimiser les chaînes de production. Adaptée au développement logiciel, elle repose sur la visualisation du travail sur un tableau divisé en colonnes représentant les états successifs de chaque tâche.

Les principes fondamentaux de Kanban sont :

- **Visualisation du flux** : toutes les tâches sont visibles sur le tableau (colonne À FAIRE, EN COURS, TERMINÉ);
- **Limitation du WIP** : le nombre de tâches simultanées en cours est limité pour éviter la surcharge;
- **Flux continu** : contrairement à Scrum, il n'y a pas de cycles fixes; les tâches avancent au rythme de la capacité réelle;
- **Amélioration continue** : le tableau permet d'identifier les goulots d'étranglement et d'y remédier rapidement.

Pour le projet SolarEase, le tableau Kanban a été géré sur **Jira** (plateforme Atlassian). Le détail du Product Backlog, de l'équipe et de la planification des sprints est présenté au chapitre suivant (section 2.3).

Chapitre 2

Spécification des Besoins et Architecture

2

Spécification des besoins et architecture

2.1 Identification des acteurs

Le système SolarEase distingue trois types d'acteurs :

- **Installateur (société)** : acteur principal. Il gère les clients, les projets, lance les calculs de dimensionnement, génère les rapports et consulte les recommandations IA ;
- **Client final** : bénéficiaire de l'espace client dédié. Il consulte ses projets, ses documents, sa facturation et peut contacter le support ;
- **Administrateur** : gère le catalogue d'équipements (panneaux, onduleurs), les paramètres système et les droits d'accès.

2.2 Besoins fonctionnels

2.2.1 Cas d'utilisation global

*On ne fait pas la modélisation ici
on recite pour chaque acteur les fonctionnalités offertes par le système*

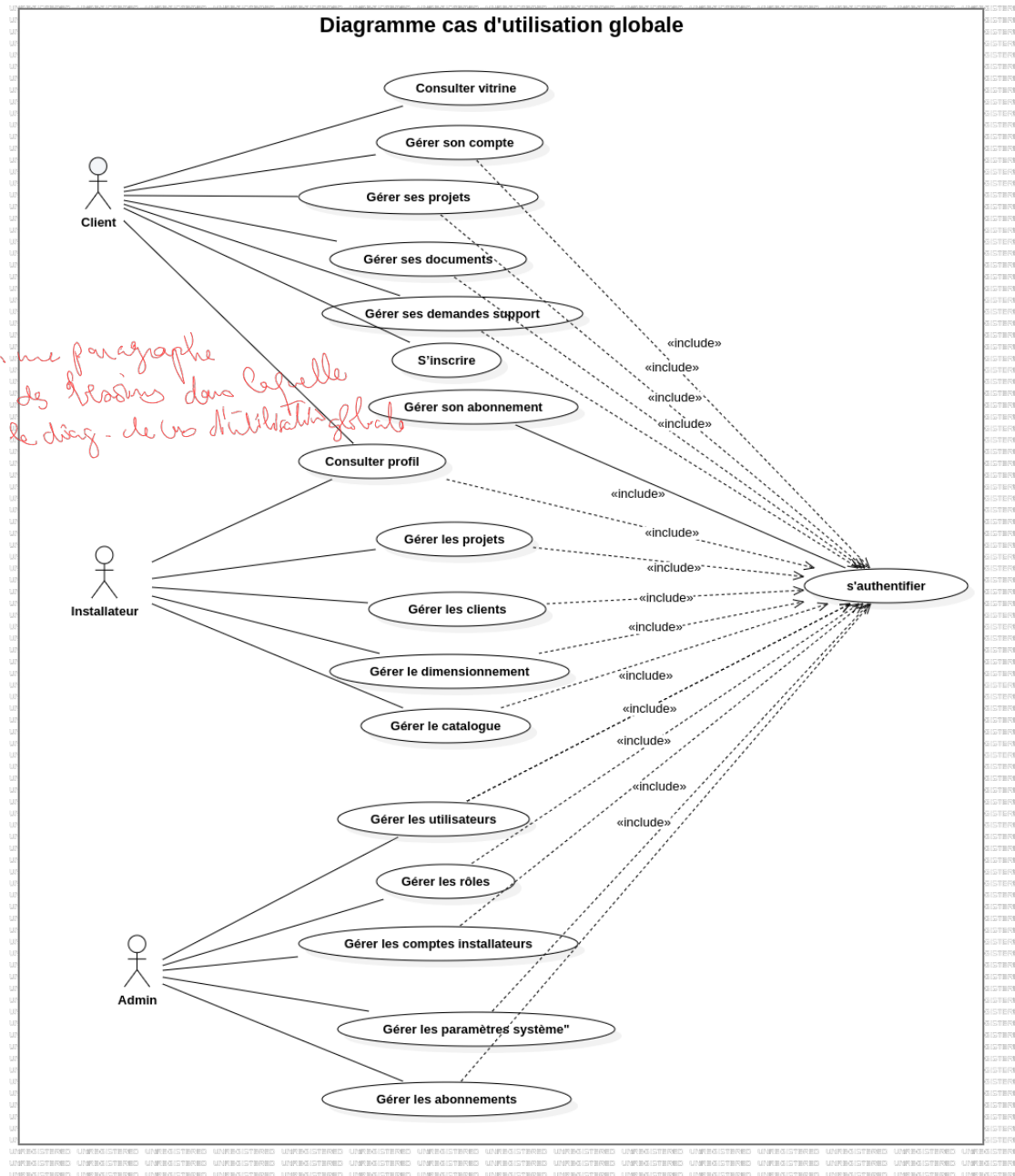


FIGURE 2.1. Diagramme de cas d'utilisation global de SolarEase

2.2.2 Description textuelle des cas d'utilisation principaux

Éliminer cette partie
Les fiches ci-dessous décrivent les trois cas d'utilisation les plus représentatifs du système.

TABLEAU 2.1. CU-01 : Inscription et authentification

CU-01 – Inscription et authentification	
Acteur principal	Installateur
Préconditions	L'utilisateur n'a pas encore de compte
Scénario nominal	1. L'installateur saisit son e-mail, mot de passe et confirmation 2. Le système valide les champs et vérifie l'unicité de l'e-mail 3. Le système hash le mot de passe et génère un code OTP 4. Le système enregistre l'utilisateur et envoie l'OTP par e-mail 5. L'installateur saisit le code OTP reçu 6. Le système vérifie l'OTP, active le compte et génère un JWT 7. L'installateur accède à son tableau de bord
Exceptions	– E-mail déjà utilisé : message d'erreur explicite – OTP expiré ou incorrect : possibilité de renvoyer le code
Postconditions	Compte actif, session JWT établie

TABLEAU 2.2. CU-02 : Création d'un projet solaire

CU-02 – Création d'un projet solaire	
Acteur principal	Installateur
Préconditions	Utilisateur authentifié
Scénario nominal	1. L'installateur clique sur « Nouveau projet » 2. Il saisit les informations (client, localisation, type, puissance, budget) 3. Le système valide les champs obligatoires et les règles métier 4. Le système enregistre le projet avec un statut initial 5. Le système affiche la fiche projet avec son identifiant
Exceptions	– Champs invalides : message d'erreur de validation – Erreur serveur : message « Échec, réessayez »
Postconditions	Projet créé, visible dans le tableau de bord

TABLEAU 2.3. CU-03 : Dimensionnement photovoltaïque (PVGIS)

CU-03 – Dimensionnement photovoltaïque	
Acteur principal	Installateur
Préconditions	Projet existant, utilisateur authentifié
Scénario nominal	1. L’installateur ouvre la fiche projet et clique « Lancer dimensionnement » 2. Il saisit/vérifie les paramètres (latitude, longitude, surface, inclinaison) 3. Le système appelle l’API PVGIS avec les coordonnées 4. PVGIS retourne les données d’irradiation et production estimée 5. Le système calcule : nombre de panneaux, puissance, production annuelle, coûts, ROI 6. Le système enregistre les résultats et les affiche à l’installateur
Exceptions	– PVGIS indisponible : calcul de repli + message d’avertissement – Données incohérentes : erreur de validation métier
Postconditions	Résultat de dimensionnement enregistré et affiché

2.2.3 Tableau des besoins fonctionnels

Tu dois rédiger cette partie dans les besoins fonctionnels et les regrouper par acteur

TABLEAU 2.4. Besoins fonctionnels principaux

ID	Fonctionnalité	Description
F01	Inscription	Création de compte avec vérification OTP par e-mail
F02	Connexion	Authentification JWT avec gestion du refresh token
F03	Gestion profil	Mise à jour des informations et changement de mot de passe
F04	CRUD Clients	Création, lecture, modification, suppression avec recherche
F05	CRUD Projets	Cycle de vie complet avec statuts et tableau de bord
F06	Dimensionnement standard	Calcul automatique avec données PVGIS
F07	Night Panel	Dimensionnement avec stockage batterie et couverture nocturne
F08	Comparaison de scénarios	Analyse Classique vs Night Panel avec courbes 24h
F09	Analyse financière	ROI, payback, cash-flow cumulé sur 25 ans
F10	Génération PDF	Rapport professionnel téléchargeable
F11	Recommandation IA	Conseil contextualisé via LLM Ollama
F12	Catalogue équipements	Panneaux, onduleurs, Night Panels administrables
F13	Espace client	Dashboard, projets, documents, facturation, support

2.3 Besoins non fonctionnels

TABLEAU 2.5. Besoins non fonctionnels

Catégorie	Exigence
Sécurité	Authentification JWT, vérification OTP, contrôle d'accès centralisé via Gateway
Performance	Réponse API < 500 ms pour les opérations CRUD ; calcul de dimensionnement < 3 s
Scalabilité	Architecture microservices permettant la montée en charge indépendante de chaque service
Disponibilité	Fonctionnement avec fallback en cas d'indisponibilité de PVGIS ou de l'IA
Maintenabilité	Séparation claire des responsabilités, couverture de tests $\geq 80\%$
Portabilité	Déploiement via Docker Compose, variables d'environnement externalisées
Utilisabilité	Interface responsive, accessible sur navigateur moderne sans installation

2.4 Pilotage du projet avec Kanban

Dans le cadre du projet SolarEase, le pilotage a été réalisé avec la méthode **Kanban**, en cohérence avec les approches agiles et itératives [5][6]. Ce choix permet de visualiser l'avancement en continu, de prioriser les tâches selon leur valeur métier et de limiter le travail en cours (*WIP limit*) afin de maintenir un flux de livraison stable.

2.4.1 Équipe et rôles

L'équipe projet est organisée autour des rôles suivants :

TABLEAU 2.6. Rôles et responsabilités de l'équipe

Rôle	Responsabilité
Encadrant académique <i>Sum Nasser / équivalent dans Kanban</i>	Valider les orientations techniques, suivre l'avancement global et assurer la cohérence avec les objectifs du PFE
Développeur full-stack (auteur)	Concevoir et implémenter les fonctionnalités Frontend et Backend, réaliser les tests et préparer les livrables
Parties prenantes métier <i>Product owner (Client)</i>	Exprimer les besoins, valider les fonctionnalités livrées et prioriser les demandes selon la valeur métier

pas d'encadrant académique dans les rôles

2.4.2 Organisation du flux Kanban

Le suivi opérationnel est assuré sur un tableau Kanban (*Jira*) avec les colonnes suivantes :

1. **To Do** – tâches priorisées, prêtes à être démarrées ;
2. **In Progress** – tâches en cours de développement ($WIP \leq 3$) ;
3. **Review** – tâches en relecture de code ou validation fonctionnelle ;
4. **Done** – tâches terminées, testées et validées.

Cette organisation a facilité :

- la visibilité quotidienne sur le travail en cours ;
- la détection rapide des blocages ;
- l'ajustement des priorités en continu grâce au flux tiré.

2.4.3 Product Backlog

Le Product Backlog regroupe l'ensemble des User Stories identifiées, classées par priorité et rattachées à une itération de livraison.

TABLEAU 2.7. Product Backlog SolarEase

ID	Acteur	User Story	Itération	Priorité	Effort
US-01	Client	M'inscrire et recevoir un OTP de confirmation par e-mail	Itération 1	Haute	5
US-02	Utilisateur	Me connecter et obtenir un JWT sécurisé	Itération 1	Haute	5
US-03	Utilisateur	Gérer mon profil utilisateur (infos, mot de passe)	Itération 1	Moyenne	3
US-04	Installateur	Créer, modifier et supprimer mes clients	Itération 2	Haute	5
US-05	Installateur	Visualiser un tableau de bord avec KPI	Itération 2	Moyenne	5
US-06	Installateur	Créer et suivre mes projets solaires	Itération 3	Haute	8
US-07	Admin	Gérer le catalogue d'équipements (panneaux, onduleurs)	Itération 3	Moyenne	5
US-08	Installateur	Lancer un dimensionnement automatique (PVGIS)	Itération 4	Haute	13
US-09	Installateur	Simuler l'ajout de Night Panels (stockage nocturne)	Itération 4	Haute	8
US-10	Installateur	Comparer les scénarios (Classique vs Night Panel)	Itération 4	Haute	5
US-11	Installateur	Générer un rapport PDF professionnel	Itération 5	Haute	5
US-12	Installateur	Recevoir une recommandation IA contextualisée	Itération 5	Moyenne	5
US-13	Installateur	Consulter l'analyse financière (ROI, pay-back, cash-flow)	Itération 5	Haute	5
US-14	Client final	Consulter mes projets et documents en ligne	Itération 5	Moyenne	5

2.4.4 Planification des itérations

Le projet est découpé en **cinq itérations** organisées selon le flux Kanban. Chaque itération correspond à un ensemble cohérent de fonctionnalités tirées du Backlog et livrées de manière incrémentale :

TABLEAU 2.8. Planification des itérations Kanban

Itération	Objectif	User Stories	Durée
Itération 1	Authentification et profil utilisateur	US-01, US-02, US-03	2 semaines
Itération 2	Gestion des clients et tableau de bord	US-04, US-05	2 semaines
Itération 3	Gestion des projets et catalogue	US-06, US-07	2 semaines
Itération 4	Dimensionnement PVGIS, Night Panel et comparaison	US-08, US-09, US-10	2 semaines
Itération 5	PDF, recommandation IA, finance et espace client	US-11, US-12, US-13, US-14	2 semaines

La figure 2.2 illustre la répartition temporelle des itérations.

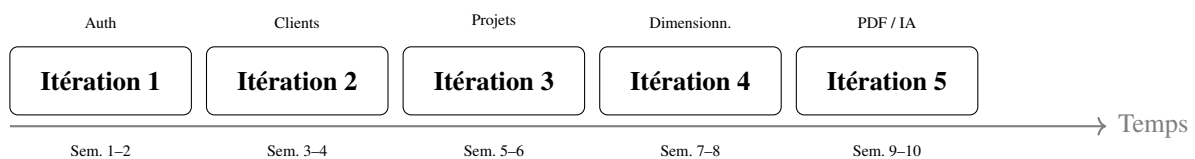


FIGURE 2.2. Timeline du projet SolarEase (5 itérations Kanban)

Chaque itération suit le cycle Kanban suivant :

1. **Sélection** : les User Stories les plus prioritaires sont tirées du Backlog ;
2. **Développement** : flux continu avec limite WIP (Work In Progress) ;
3. **Revue** : validation fonctionnelle avec l'encadrant ; *le Sam Nasser*
4. **Rétrospective** : identification des améliorations pour l'itération suivante.

2.5 Architecture technique globale

2.5.1 Choix de l'architecture microservices

L'architecture **microservices** a été adoptée pour les raisons suivantes :

- **Découplage** : chaque service est indépendant et dispose de sa propre base de données (*database-per-service pattern*);
- **Scalabilité** : un service peut être mis à l'échelle indépendamment sans impacter les autres;
- **Maintenabilité** : les équipes peuvent travailler sur des services distincts sans conflits;
- **Résilience** : la défaillance d'un service n'entraîne pas l'arrêt complet de la plateforme.

2.5.2 Schéma d'architecture globale

La figure 2.3 présente la vue d'ensemble de l'architecture. Le Frontend communique exclusivement avec l'API Gateway, qui route les requêtes vers le service approprié. Chaque microservice dispose de sa propre base de données PostgreSQL (*database-per-service*) et le Dimensioning Service accède à deux services externes : l'API PVGIS pour les données d'irradiation et le modèle Ollama pour les recommandations IA.

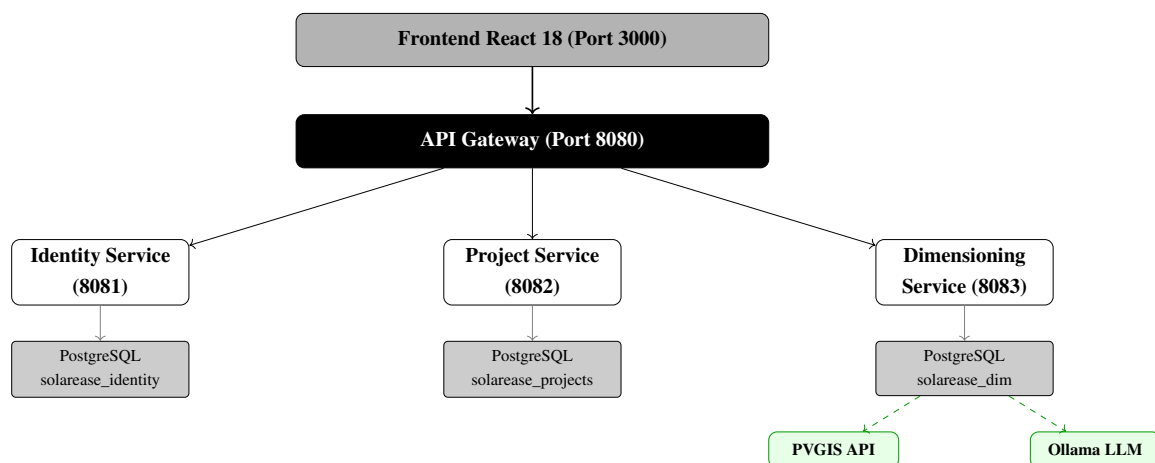


FIGURE 2.3. Architecture microservices de SolarEase

2.5.3 Flux de sécurité

Le flux de sécurité centralisé fonctionne ainsi :

1. Le client envoie une requête avec l'en-tête Authorization: Bearer <JWT>;
2. Le Gateway valide le token JWT (signature + expiration);
3. Si valide, il extrait les informations utilisateur et les propage via des en-têtes internes (X-User-Id, X-User-Role);
4. Les services métiers exploitent ces en-têtes internes sur le réseau de confiance, sans revalider le JWT;
5. Si le token est expiré, le client utilise le refresh token pour en obtenir un nouveau.

2.5.4 Stack technique

Le choix technologique de SolarEase repose sur des outils modernes, éprouvés et adaptés au contexte microservices. Le tableau 2.9 détaille chaque couche.

TABLEAU 2.9. Stack technique de SolarEase

Couche	Technologie	Justification
Backend		
Langage & Framework	Spring Boot 3.2 / Java 17	Écosystème mature, support natif microservices, Spring Security intégré
Gateway	Spring Cloud Gateway	Routage centralisé, filtres JWT, load-balancing
Base de données	PostgreSQL 16	Robustesse ACID, support JSON natif, extensibilité
Sécurité	JWT + OTP + Spring Security	Architecture stateless, vérification en deux étapes
Intelligence artificielle	Ollama / TinyLLaMA	LLM local, autonomie totale, pas de dépendance cloud
API externe	PVGIS v5.2 (UE)	Données d'irradiation officielles, gratuites et fiables
Frontend		
Framework UI	React 18 / TypeScript	Typage fort, composants réutilisables, écosystème riche
Build	Vite 5	Compilation ultra-rapide, HMR instantané
Styling	TailwindCSS v4 + Radix UI	Design system cohérent, composants accessibles
Gestion d'état	Zustand + TanStack Query	État global léger, cache serveur intelligent
Formulaires	React Hook Form + Zod	Validation typée côté client, performances optimales
DevOps & Outils		
Conteneurisation	Docker + Docker Compose	Portabilité, déploiement reproductible
Gestion de projet	Jira (Kanban)	Suivi visuel, priorisation continue
Versioning	Git + GitHub	Collaboration, traçabilité du code

2.5.5 Modèle de données globale

Le diagramme de classes ci-dessous présente la modélisation statique du système SolarEase. Il regroupe les entités principales de chaque microservice ainsi que leurs relations (associations, compositions).

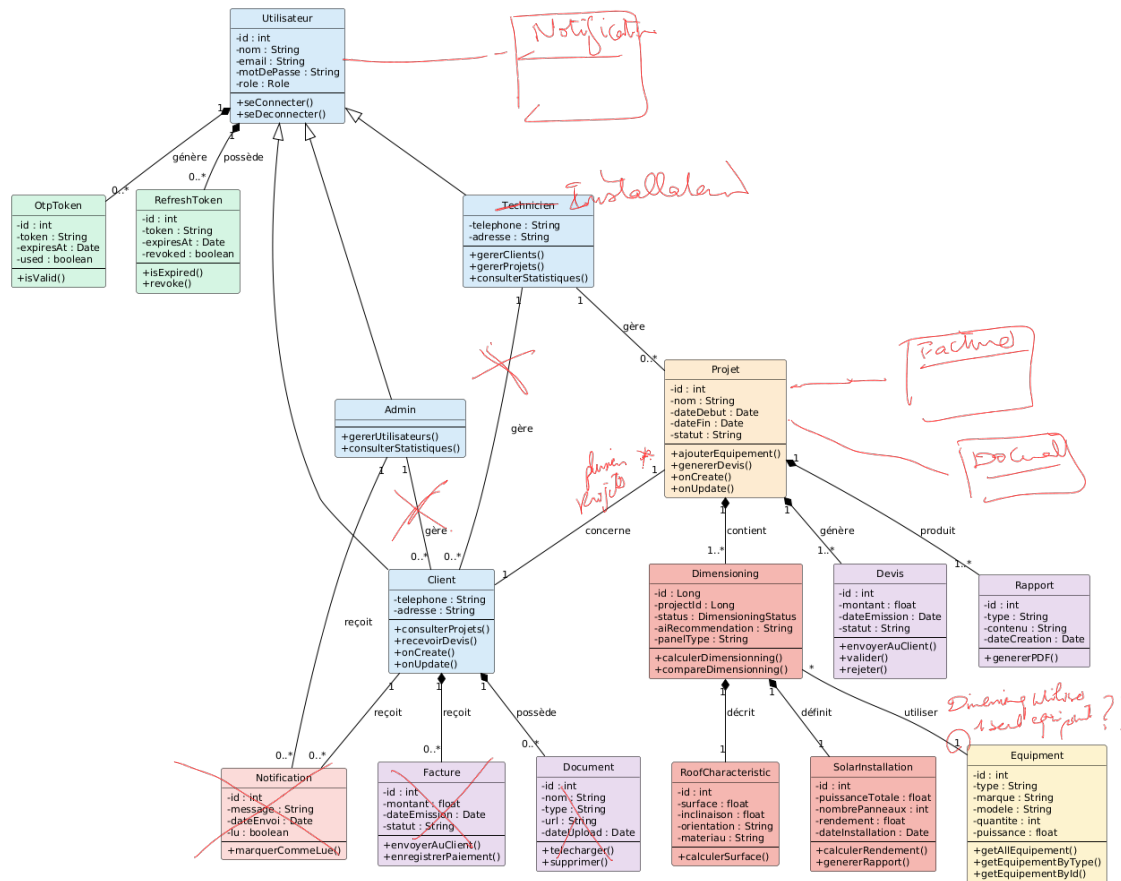


FIGURE 2.4. Diagramme de classes globale de SolarEase

Il faut éviter les cycles



Chapitre 3

Release 1 : Fondation, Fonctionnalités Métier et Dimensionnement

3

Release 1 : Fondation, Fonctionnalités Métier et Dimensionnement

3.1 Itération 1 : Service d'Identité et Authentification

3.1.1 Backlog de l'itération

TABLEAU 3.1. Backlog Kanban – Itération 1 (Identity Service, Port 8081)

ID	User Story	Tâches	Effort
1	En tant que client, je veux créer un compte avec vérification e-mail pour accéder à mon espace.	→ Endpoint POST /register (username, email, password, firstName, lastName, phone) → Hash BCrypt (force 10) → Rôle automatiquement assigné à CLIENT → Génération OTP 6 chiffres (TTL 10 min) → Envoi OTP par e-mail (SMTP) → Endpoints POST /verify-otp et POST /resend-otp → Interface React (formulaire + page OTP) → Tests unitaires et intégration	Élevé
2	En tant qu'admin, je veux créer et gérer les comptes installateurs de l'entreprise.	→ Endpoint POST /api/auth/installers (firstName, lastName, email, phone, password) → Endpoints GET, PUT, DELETE /api/auth/installers/{uuid} → Compte créé directement vérifié (isEmailVerified=true) → Interface admin de gestion des installateurs → Vérification du rôle ADMIN obligatoire	Élevé

Suite page suivante

ID	User Story	Tâches	Effort
3	En tant qu'utilisateur (client ou installateur), je veux me connecter et obtenir un token JWT pour accéder aux services.	→ Endpoint POST /login → Génération Access Token (HS256, 1h) avec claims role, email → Génération Refresh Token (UUID, 7j) → Endpoint POST /refresh (rotation) → Réponse AuthResponse enrichie (tokens + UserDto) → Interface login React + redirection selon le rôle → Tests avec Postman	Élevé
4	En tant qu'utilisateur connecté, je veux consulter et modifier mon profil.	→ Endpoints GET /me et PUT /me → Endpoint dédié PUT /me/password (ancien + nouveau mot de passe) → Interface profil React → Tests fonctionnels	Moyen
5	En tant que système, le Gateway doit valider les JWT et propager l'identité aux services.	→ Filtre JWT dans Spring Cloud Gateway → Propagation X-User-Id, X-User-Role, X-User-Email → Liste de routes publiques exemptées (register, login, OTP, etc.) → Rejet automatique des tokens expirés (HTTP 401) → Tests de sécurité (token valide/invalid)	Élevé

3.1.2 Analyse de l'itération 1

► Diagramme cas d'utilisation raffiné de l'itération 1

Ce diagramme illustre l'ensemble des cas d'utilisation du service d'identité. Deux acteurs principaux interagissent avec le système : le **Client** qui s'auto-inscrit via le formulaire public, et l'**Admin** qui crée et gère les comptes installateurs en interne. Les installateurs ne s'inscrivent pas eux-mêmes ; c'est l'entreprise qui provisionne leurs comptes.

Diagramme cas d'utilisation raffiné de l'itération 1

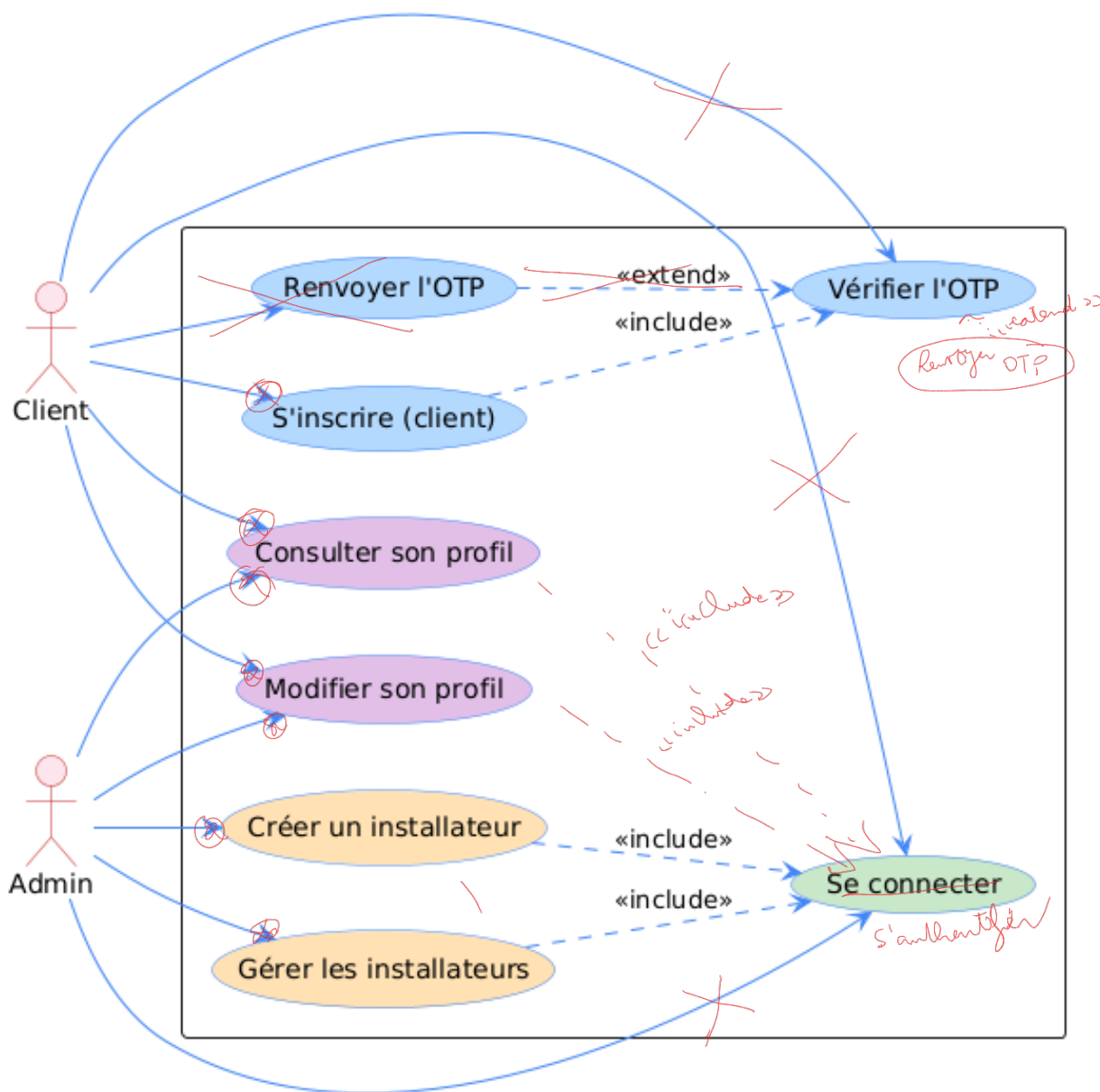


FIGURE 3.1. Diagramme cas d'utilisation raffiné de l'itération 1

► Vue détaillée des cas d'utilisation de l'itération 1

Nous présentons ci-dessous la description textuelle des cas d'utilisation les plus critiques.

TABLEAU 3.2. Description textuelle du CU01 — Inscription *S'inscrire*

Cas d'utilisation	S'inscrire sur la plateforme SolarEase
Acteur initiateur	Client (non inscrit)
Précondition	L'adresse e-mail et le nom d'utilisateur n'existent pas en base
Postcondition	Compte client créé (non vérifié) ; code OTP envoyé par e-mail
Scénario principal	1. Le client accède au formulaire d'inscription public 2. Il saisit : username, prénom, nom, e-mail, mot de passe (≥ 6 car.), téléphone (optionnel) 3. Le rôle CLIENT est assigné automatiquement 4. Le système valide les champs côté client (Zod) et serveur (Jakarta Validation) 5. Le système hache le mot de passe (BCrypt, force 10) 6. Le compte est créé avec <code>isEmailVerified=false</code> et <code>isActive=true</code> 7. Le système génère un code OTP 6 chiffres, TTL 10 minutes 8. Le code OTP est envoyé à l'adresse e-mail 9. Le client est redirigé vers la page de vérification
Scénarios alternatifs	<i>E-mail ou username déjà existant</i> : HTTP 400 message explicite <i>Mot de passe <6 caractères</i> : rejet avec message d'exigences <i>Format invalide</i> : erreur de validation côté frontend

Raffinement du cas d'utilisation « S'inscrire »

TABLEAU 3.3. Description textuelle du CU01b — Vérification OTP

Cas d'utilisation	Vérifier le code OTP reçu par e-mail
Acteur initiateur	Client inscrit, non vérifié
Précondition	Un OTP valide (non expiré, non utilisé) existe en base
Postcondition	Compte activé ; JWT Access et Refresh Tokens émis
Scénario principal	1. L'utilisateur saisit le code OTP à 6 chiffres 2. Le système recherche le token en base (email + code) 3. Vérification : token non expiré (<10 min) et non utilisé 4. Mise à jour : isUsed=true, isEmailVerified=true 5. Génération du JWT Access Token et du Refresh Token 6. Redirection vers le tableau de bord
Scénarios alternatifs	<i>OTP expiré</i> : HTTP 400, proposer renvoi <i>OTP incorrect</i> : HTTP 400, tentatives restantes <i>OTP déjà utilisé</i> : HTTP 400, proposer renvoi

Raffinement du cas d'utilisation « Vérifier l'OTP »

TABLEAU 3.4. Description textuelle du CU01c — Création de compte installateur

Cas d'utilisation	Créer un compte installateur
Acteur initiateur	Admin
Précondition	L'admin est connecté ; l'e-mail de l'installateur n'existe pas en base
Postcondition	Compte installateur créé, directement vérifié et actif
Scénario principal	1. L'admin accède à l'interface de gestion des installateurs 2. Il remplit le formulaire : prénom, nom, e-mail, téléphone, mot de passe 3. Le système valide les champs (Jakarta Validation) 4. Le mot de passe est haché (BCrypt, force 10) 5. Le compte est créé avec rôle INSTALLER, isEmailVerified=true, isActive=true 6. L'installateur apparaît dans la liste de gestion
Scénarios alternatifs	<i>E-mail déjà existant</i> : HTTP 400 message explicite <i>Rôle non ADMIN</i> : HTTP 403 accès refusé <i>Champs invalides</i> : erreur de validation

Raffinement du cas d'utilisation « Créer un installateur »**TABLEAU 3.5.** Description textuelle du CU02 — Connexion *s'authentification*

Cas d'utilisation	Se connecter à la plateforme SolarEase
Acteur initiateur	Client vérifié, Installateur (créé par admin) ou Admin
Précondition	Le compte existe, est activé (isActive=true) et vérifié (isEmailVerified=true)
Postcondition	JWT Access Token (1h) et Refresh Token (7j) émis ; objet UserDto retourné
Scénario principal	1. L'utilisateur soumet son e-mail et son mot de passe 2. Le Gateway transfère la requête au Identity Service (route publique) 3. Le service récupère l'utilisateur par e-mail 4. Comparaison du hash BCrypt du mot de passe fourni 5. Génération du JWT Access Token (HS256, 1h) avec claims role et email 6. Génération du Refresh Token (UUID, stocké en base, TTL 7j) 7. Retour de l'AuthResponse : tokens + UserDto (id, uuid, email, username, rôle) 8. Redirection vers le tableau de bord correspondant au rôle
Scénarios alternatifs	<i>Compte non vérifié</i> : redirection vers page OTP <i>Mot de passe incorrect</i> : erreur 401 générique <i>Compte inexistant</i> : même message 401 (anti-énumération) <i>Compte désactivé</i> : erreur 401

Raffinement du cas d'utilisation « Se connecter »**3.1.3 Conception de l'itération 1****► Diagramme de séquence — Inscription client et ~~vérification~~ OTP***1 diagramme de séquence pour chaque Cas d'utilisation*

Ce diagramme illustre le flux d'auto-inscription réservé aux clients. Les installateurs n'utilisent pas ce parcours ; leurs comptes sont créés par l'administrateur via un endpoint dédié (POST /api/auth/installers).

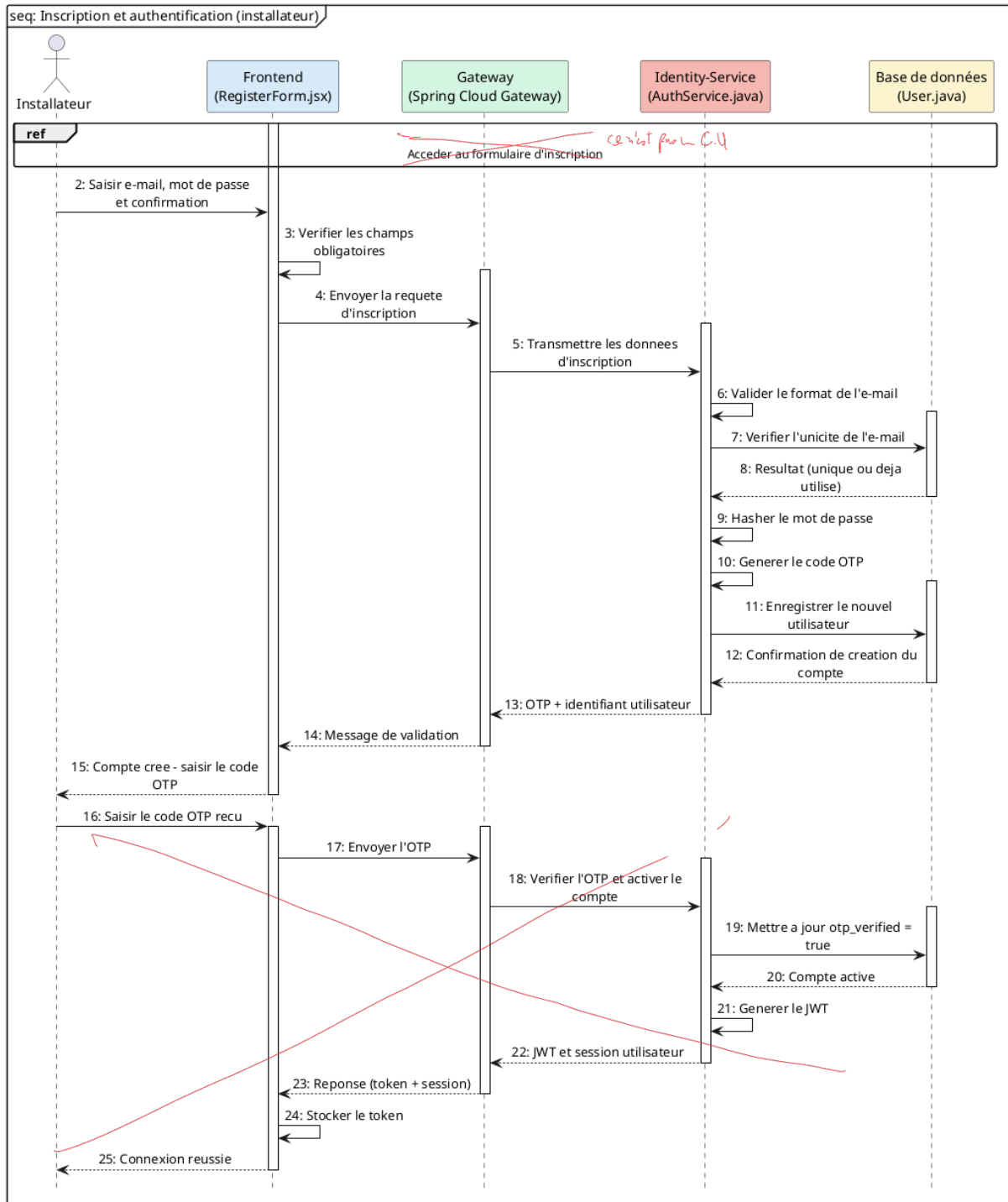


FIGURE 3.2. Diagramme de séquence — Inscription et authentification (client)

► Diagramme de classes de l'itération 1

*Le n'est pas un diag de classes!
Il faut mettre les classes qui réalisent
ce sprint (itération)*

TABLEAU 3.6. Diagramme de classes – Itération 1 (Identity Service)

Classe	Attributs principaux	Relations / Rôle
User	id, uuid, username, email, passwordHash, firstName, lastName, phone, role (enum), companyId, isEmailVerified, isActive, createdAt, updatedAt	Entité JPA (table users); rôle ADMIN INSTALLER CLIENT; clients s'auto-inscrivent; installateurs créés par l'admin; <i>OneToMany</i> vers OtpToken et RefreshToken
OtpToken	id, token (6 chiffres), expiresAt, isUsed, user (FK)	TTL 10 min; usage unique; <i>ManyToOne</i> vers User
RefreshToken	id, token (UUID), expiresAt (7j), user (FK)	Rotation à chaque renouvellement; <i>ManyToOne</i> vers User
AuthResponse	accessToken, refreshToken, tokenType, expiresIn, user (UserDto), message	DTO de réponse; UserDto imbrique id, uuid, email, username, role, isActive
JwtTokenProvider	jwtSecret, accessTTL, refreshTTL	generateAccessToken() avec claims role et email; validateToken(); extractClaims()
GatewayFilter	jwtSecret, PUBLIC_ROUTES[]	Valide JWT; propage X-User-Id, X-User-Role, X-User-Email; exempte les routes publiques

3.1.4 Réalisation de l'itération 1

Cette section présente les interfaces développées et les résultats de validation de l'itération 1.

```

1 @Component
2 public class JwtTokenProvider {
3
4     public String generateAccessToken(User user) {
5         Date now = new Date();
6         Date expiry = new Date(now.getTime() + accessTokenExpMs);
7         return Jwts.builder()
8             .subject(user.getUuid().toString())
9             .claim("role", user.getRole().name())
10            .claim("email", user.getEmail())

```

*pas de code
dans la
conception*

```

11         .issuedAt(now).expiration(expiry)
12         .signWith(getSigningKey(), Jwts.SIG.HS256)
13         .compact();
14     }
15
16     public boolean validateToken(String token) {
17         try {
18             Jwts.parser().verifyWith(getSigningKey())
19                 .build().parseSignedClaims(token);
20             return true;
21         } catch (JwtException e) { return false; }
22     }
23 }

```

Listing 3.1 – JwtTokenProvider.java — Génération du token d'accès

TABLEAU 3.7. Résultats des tests fonctionnels – Itération 1

ID	Cas de test	Résultat attendu	Statut
T-01	Inscription client valide	HTTP 201 + OTP envoyé (rôle CLIENT)	✓ OK
T-02	Email déjà existant	HTTP 409 Conflict	✓ OK
T-03	OTP correct <10 min	Compte activé + tokens	✓ OK
T-04	OTP expiré	HTTP 400 "OTP expiré"	✓ OK
T-05	Connexion valide	Access + Refresh tokens	✓ OK
T-06	Mot de passe faux	HTTP 401 générique	✓ OK
T-07	Admin créée installateur	HTTP 201, compte vérifié INSTALLER	✓ OK
T-08	Création installateur sans rôle ADMIN	HTTP 403 Forbidden	✓ OK
T-09	JWT valide au Gateway	Propagation X-User-Id	✓ OK
T-10	JWT expiré au Gateway	HTTP 401	✓ OK

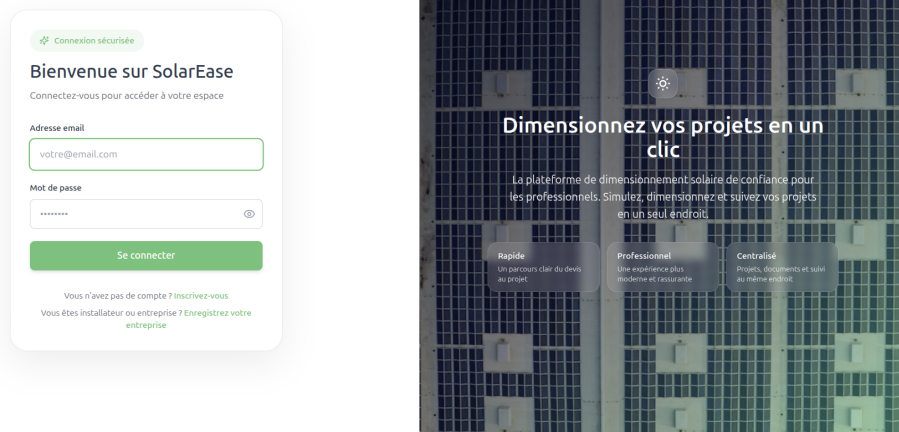


FIGURE 3.3. Interface d’authentification – SolarEase

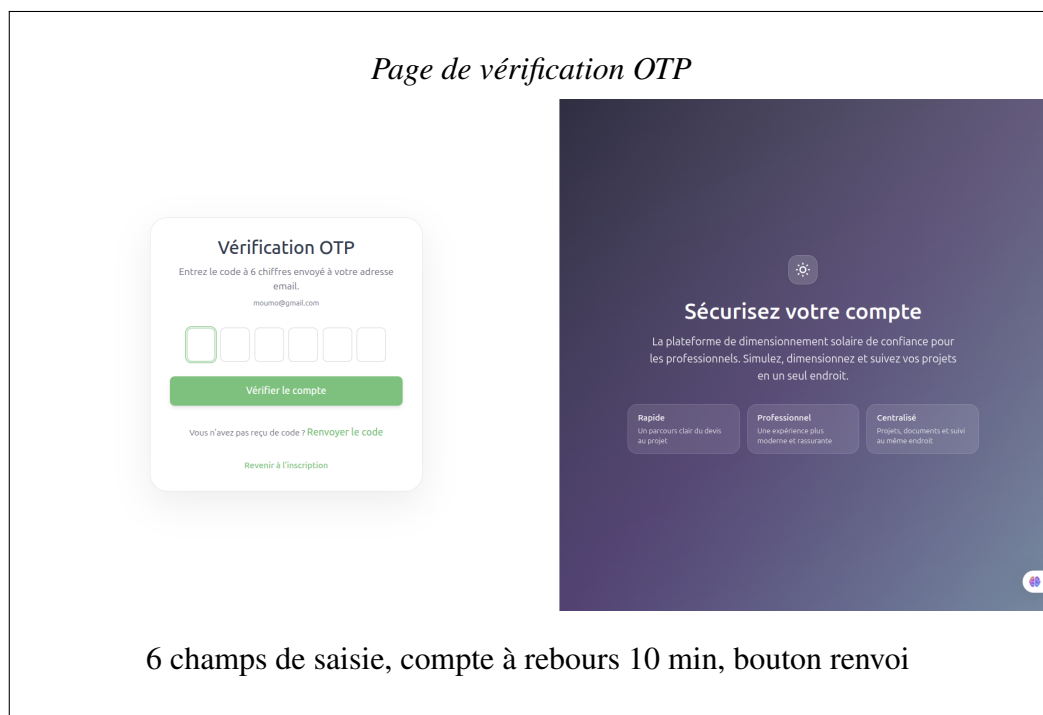


FIGURE 3.4. Interface de vérification OTP – Itération 1

3.2 Itération 2 : Gestion des Clients et des Projets

3.2.1 Backlog de l'itération

TABLEAU 3.8. Backlog Kanban – Itération 2 (Project Service, Port 8082)

ID	User Story	Tâches	Effort
1	En tant qu'admin, je veux gérer l'ensemble des clients de l'entreprise (ajouter, modifier, supprimer, consulter, chercher).	→ Endpoints CRUD /api/clients (rôle ADMIN) → Vue globale : l'admin liste tous les clients → Pagination et recherche par nom/email → Fiche détaillée client + projets liés → Interfaces React (liste, formulaire, détail) → Tests avec Postman puis intégration	Élevé
2	En tant qu'admin, je veux créer les projets solaires et les affecter aux installateurs de l'entreprise.	→ Endpoint dédié POST /api/projects/admin/assign (ADMIN uniquement) → Liaison projet-client via clientId et projet-installateur via installerId → Métadonnées d'affectation : assignedByAdminId, assignedByAdminEmail → Endpoints CRUD complémentaires /api/projects → Gestion des 6 transitions de statut → Interface admin : formulaire création + sélection d'installateur → Tests fonctionnels	Élevé
3	En tant qu'admin, je veux visualiser un tableau de bord global avec les KPI de l'ensemble de l'entreprise.	→ Endpoint GET /api/projects/dashboard/stats (vue globale pour ADMIN) → Endpoint GET /api/clients/stats → Requêtes JPQL agrégées (comptages globaux) → Cartes KPI : total clients, projets créés, en cours, terminés, annulés → Interface dashboard admin avec filtres → Tests	Moyen

3.2.2 Analyse de l'itération 2

Pour mieux comprendre cette deuxième itération, nous présentons le diagramme de cas d'utilisation global, les descriptions textuelles détaillées des cas principaux, puis les diagrammes de conception.

► *Diagramme de cas d'utilisation global relatif à l'itération 2*

Le diagramme ci-dessous illustre les cas d'utilisation développés lors de l'itération 2, dont l'acteur principal est l'**Admin**. C'est lui qui gère la totalité du portefeuille clients, crée les projets et les affecte aux installateurs de l'entreprise. L'installateur, qui consultera ses projets affectés en itération 3, est représenté ici en acteur secondaire pour le cas d'affectation.

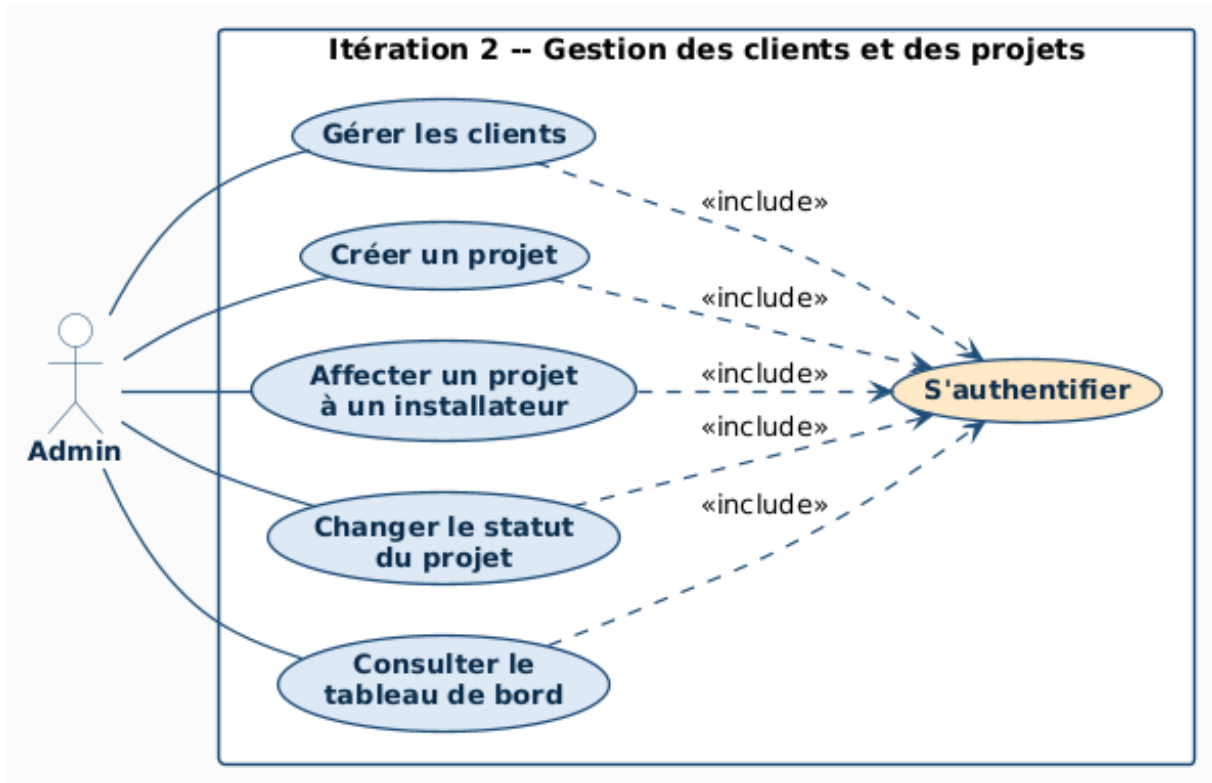


FIGURE 3.5. Interface d'authentification

FIGURE 3.6. Diagramme de cas d'utilisation global – Itération 2

► *Vue détaillée des cas d'utilisation de l'itération 2*

TABLEAU 3.9. Description textuelle du CU04 — Création d'une fiche client

Cas d'utilisation	Créer une fiche client dans le portefeuille de l'entreprise
Acteur initiateur	Admin authentifié
Précondition	L'admin est connecté avec un JWT valide portant le rôle ADMIN
Postcondition	Fiche client créée avec identifiant unique ; visible globalement dans le portefeuille
Scénario principal	1. L'admin accède à l'espace « Gestion des clients » et clique sur « Nouveau client » 2. Il remplit le formulaire (prénom, nom, e-mail, téléphone, adresse) 3. Le frontend valide les champs (Zod schema) 4. La requête POST /api/clients est envoyée avec le JWT 5. Le Gateway propage X-User-Id, X-User-Role=ADMIN au Project Service 6. <code>AccessControlService.requireAnyRole("ADMIN", "INSTALLER")</code> valide l'accès 7. Le service vérifie l'unicité de l'e-mail et persiste la fiche 8. HTTP 201 + la fiche créée ; la liste globale est rafraîchie automatiquement
Scénarios alternatifs	<i>E-mail déjà utilisé</i> : HTTP 409 Conflict <i>Champ obligatoire manquant</i> : HTTP 400 avec détail <i>Rôle insuffisant (CLIENT)</i> : HTTP 403 Forbidden

Raffinement du cas d'utilisation « Créer un client »

TABLEAU 3.10. Description textuelle du CU05 — Affectation d'un projet par l'admin

Cas d'utilisation	Créer un projet et l'affecter à un installateur
Acteur initiateur	Admin authentifié
Précondition	Le client existe ; au moins un installateur est enregistré en base
Postcondition	Projet créé avec statut CREATED; installerId et métadonnées assignedByAdminId, assignedByAdminEmail renseignés
Scénario principal	1. L'admin accède à l'écran « Nouveau projet » 2. Il sélectionne le client cible dans la liste 3. Il sélectionne l'installateur à affecter parmi ceux retournés par GET /api/auth/installers 4. Il remplit les caractéristiques techniques (localisation, puissance crête, surface, inclinaison, orientation, budget) 5. Requête POST /api/projects/admin/assign avec le JWT admin 6. Le contrôleur vérifie requireAnyRole("ADMIN") 7. Le service crée le projet, fixe installerId = installateur choisi, et renseigne assignedByAdminId = id de l'admin 8. HTTP 201 + la fiche projet ; le projet apparaît dans le tableau de bord de l'installateur
Scénarios alternatifs	<i>installerId manquant ou vide</i> : HTTP 400 <i>IllegalArgumentException</i> <i>Rôle non ADMIN</i> : HTTP 403 Forbidden <i>Client inexistant</i> : HTTP 404

Raffinement du cas d'utilisation « Affecter un projet à un installateur »

TABLEAU 3.11. Description textuelle du CU05b — Gestion du statut

Cas d'utilisation	Faire évoluer le statut d'un projet
Acteur initiateur	Admin authentifié (ou installateur, sur ses projets affectés)
Précondition	Projet existant ; transition demandée parmi les 6 statuts valides
Postcondition	Statut mis à jour avec horodatage ; contrainte CHECK en base respectée
Scénario principal	1. L'utilisateur accède à la fiche projet 2. Il sélectionne le nouveau statut parmi : CREATED, EN_PREPARATION, INSTALLATEUR_AFFECTE, IN_PROGRESS, COMPLETED, CANCELLED 3. Requête PATCH /api/projects/{id}/status 4. Le service vérifie la validité du statut demandé 5. Mise à jour du champ status et updatedAt 6. HTTP 200 avec la fiche mise à jour
Scénarios alternatifs	<i>Statut invalide</i> : HTTP 400 avec liste des valeurs autorisées <i>Projet inexistant</i> : HTTP 404

Raffinement du cas d'utilisation « Cycle de vie d'un projet »

3.2.3 Conception de l'itération 2

► Diagrammes de séquence

Diagramme de séquence « Créer un projet et l'affecter à un installateur » Ce diagramme illustre le scénario nominal côté Admin : création du projet via l'endpoint dédié POST /api/projects/admin/assign, avec affectation immédiate à un installateur choisi parmi ceux retournés par l'Identity Service.

Même remarque ici :
soit diag pour chaque Cas d'utilisation

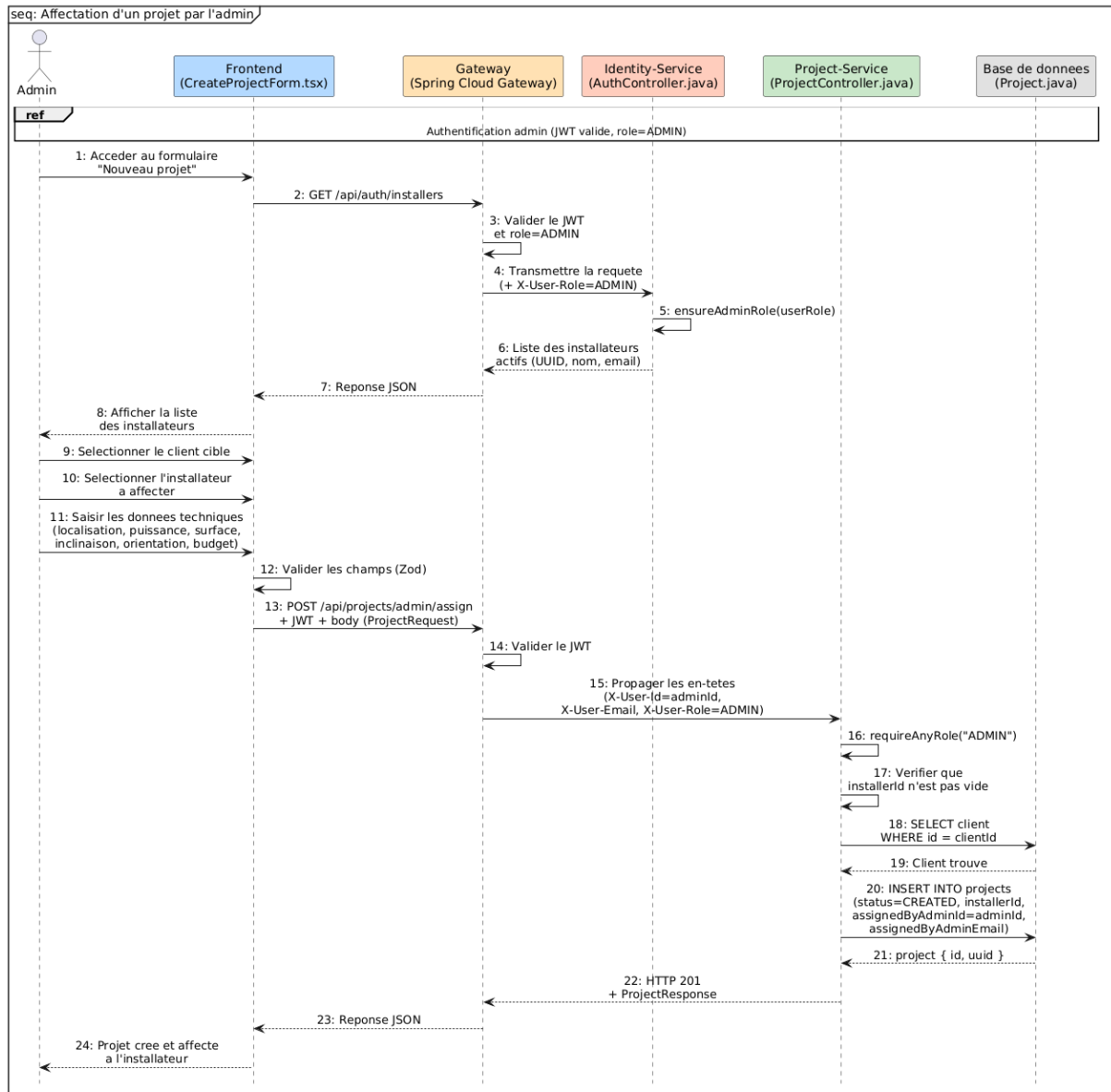


FIGURE 3.7. Diagramme de séquence — Affectation d'un projet par l'admin

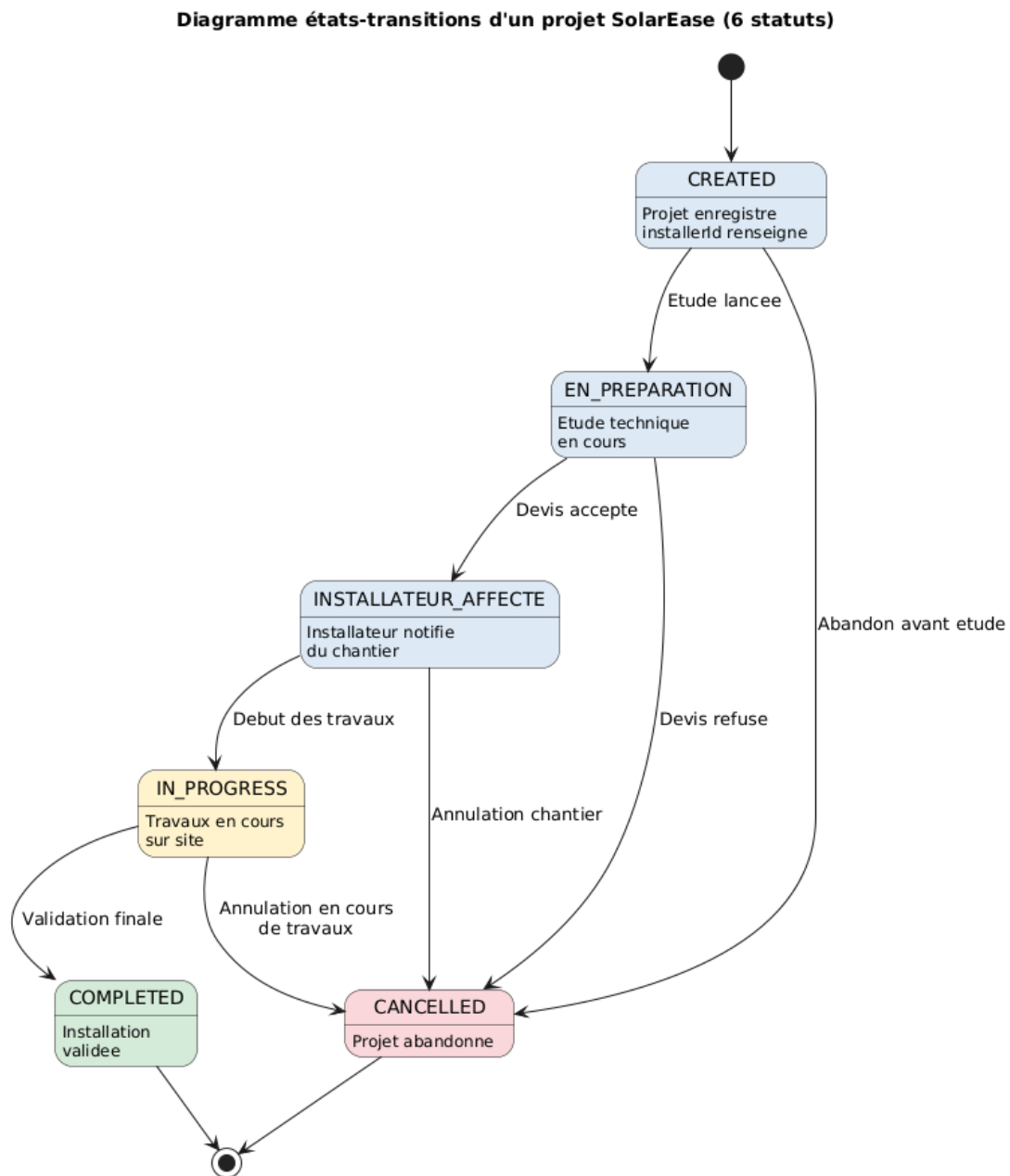


FIGURE 3.8. Diagramme états-transitions d'un projet SolarEase (6 statuts)

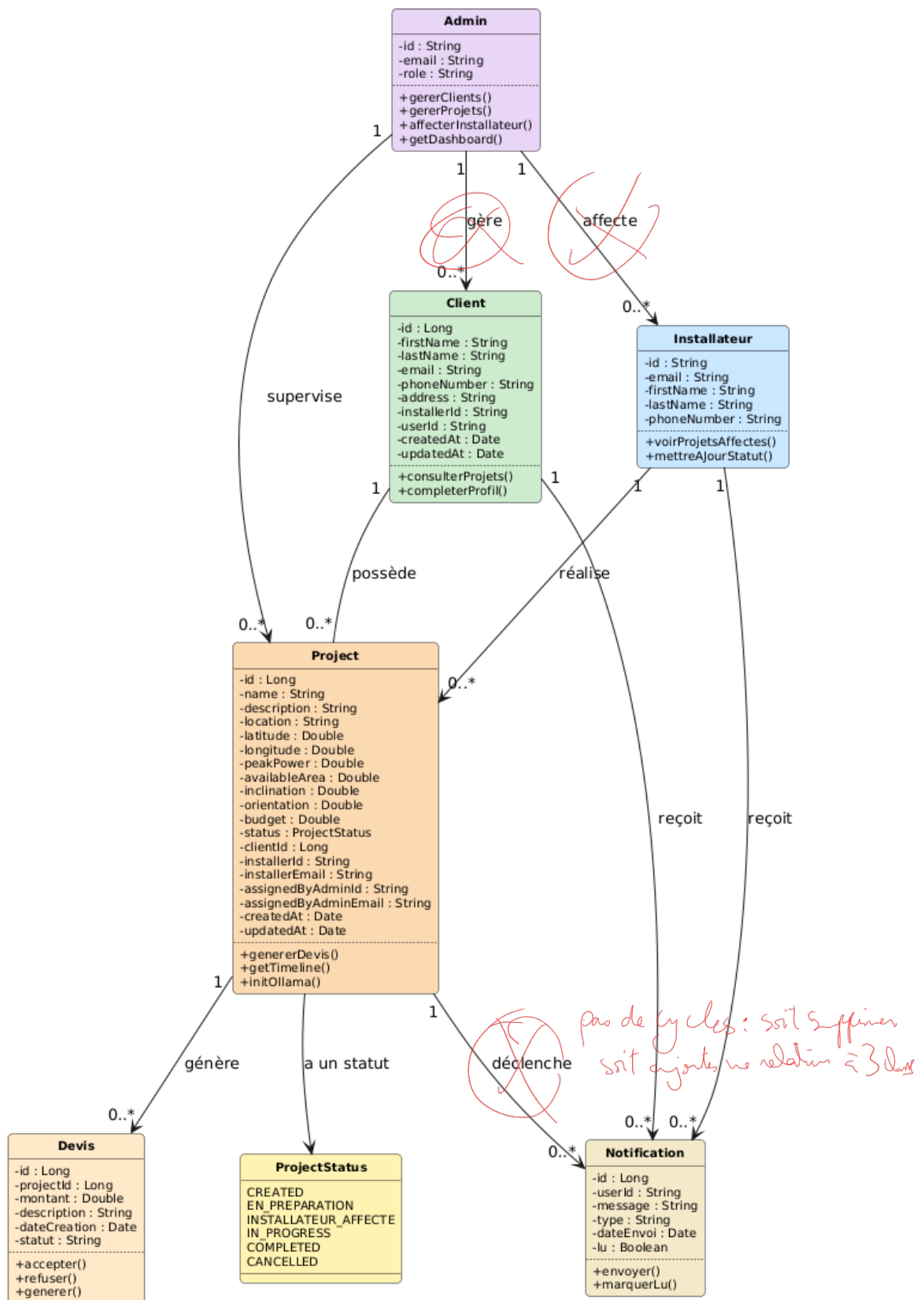


FIGURE 3.9. Diagramme de classes – Itération 2

► Diagramme de classes de l'itération 2

TABLEAU 3.12. Diagramme de classes – Itération 2 (Project Service)

Classe	Attributs principaux	Relations / Rôle
Client	id, firstName, lastName, email, phoneNumber, address, installerId, userId, createdAt, updatedAt	Entité JPA (table items_client); fiche créée et maintenue par l' Admin ; installerId = installateur de référence éventuellement associé (optionnel); userId = lien vers le compte Identity lorsque le client s'inscrit; <i>OneToMany</i> vers Project
Project	id, name, description, location, latitude, longitude, peakPower, availableArea, inclination, orientation, budget, status, clientId, installerId, installerEmail, assignedByAdminId, assignedByAdminEmail, createdAt, updatedAt	Entité JPA (table projects); <i>ManyToOne</i> vers Client via clientId; installerId = installateur affecté par l'admin; assignedByAdminId/assignedByAdminEmail = traçabilité de l'affectation; statut contrôlé par ProjectStatus (6 valeurs)
ProjectStatus	CREATED, EN_PREPARATION, INSTALLATEUR_AFFECTE, IN_PROGRESS, COMPLETED, CANCELLED	Enum Java (6 états); transitions contrôlées par le service
DashboardStats	totalClients, activeProjects, completedProjects, thisMonth	DTO calculé par requête JPQL agrégée

3.2.4 Réalisation de l'itération 2

Dans cette section, nous présentons les extraits de code significatifs, les interfaces développées et les résultats de validation de l'itération 2.

```

1 @PostMapping("/admin/assign")
2 public ProjectResponse adminAssignProject(
3     @RequestHeader("X-User-Id") String adminId,
4     @RequestHeader("X-User-Email") String adminEmail,

```

```

5      @RequestHeader("X-User-Role") String userRole,
6      @Valid @RequestBody ProjectRequest request) {
7
8      accessControlService.requireAnyRole(userRole, "ADMIN");
9      if (request.getInstallerId() == null || request.getInstallerId().
10         isBlank()) {
11         throw new IllegalArgumentException(
12             "installerId is required for admin project creation");
13     }
14     return projectService.createProject(
15         request.getInstallerId(),
16         request.getInstallerEmail(),
17         adminId,      // assignedByAdminId
18         adminEmail,   // assignedByAdminEmail
19         request);
20 }

```

Listing 3.2 – ProjectController.java — Endpoint d’affectation réservé à l’admin



FIGURE 3.10. Tableau de bord administrateur – Itération 2

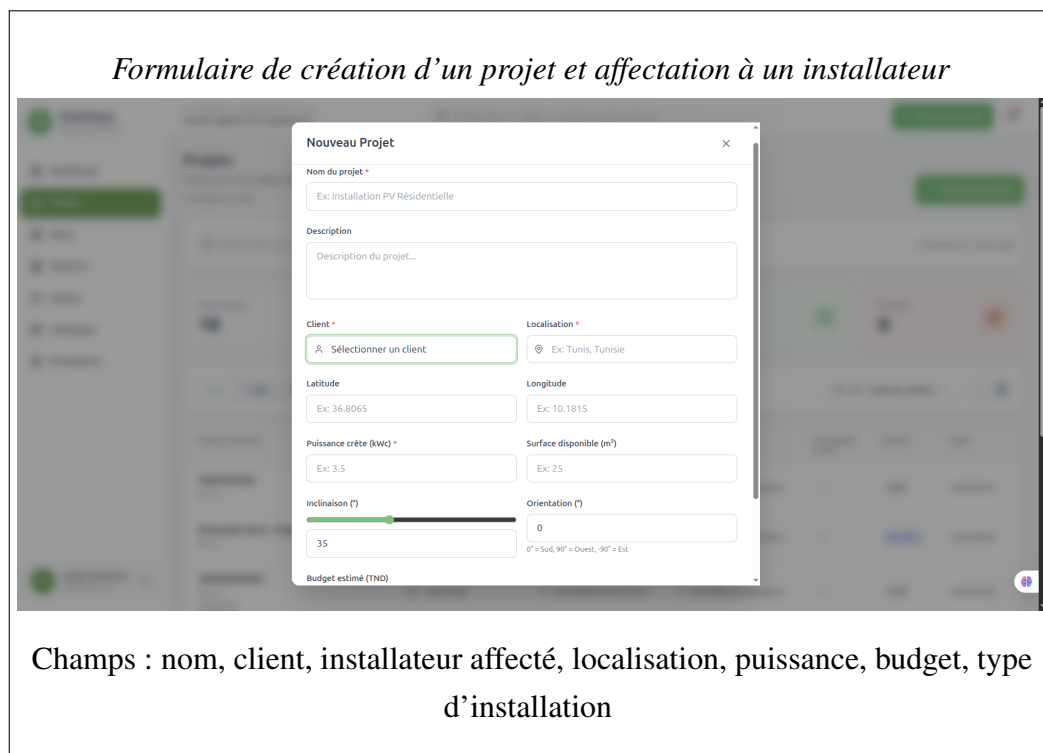
**FIGURE 3.11.** Page de gestion des clients (admin) – Itération 2**FIGURE 3.12.** Interface admin de création de projet – Itération 2

TABLEAU 3.13. Résultats des tests fonctionnels – Itération 2

ID	Cas de test	Résultat attendu	Statut
T-11	Créer client par l'admin	HTTP 201 + fiche client	✓ OK
T-12	Email doublon	HTTP 409 Conflict	✓ OK
T-13	Lister tous les clients (admin)	Liste paginée globale	✓ OK
T-14	Modifier client	HTTP 200 + fiche MAJ	✓ OK
T-15	Supprimer client	HTTP 204	✓ OK
T-16	Admin affecte un projet à un installateur (POST /admin/assign)	HTTP 201, installerId et assignedByAdminId renseignés	✓ OK
T-17	installerId manquant sur /admin/assign	HTTP 400 IllegalArgumentException	✓ OK
T-18	Rôle INSTALLER sur /admin/assign	HTTP 403 Forbidden	✓ OK
T-19	Transition de statut valide	Statut mis à jour	✓ OK
T-20	Transition de statut invalide	HTTP 400 refusé	✓ OK
T-21	Dashboard admin (vue globale)	KPI agrégés sur toute l'entreprise	✓ OK

La Release 1 a livré le socle complet de la plateforme SolarEase : authentification sécurisée JWT/OTP et gestion des clients et projets. Ces deux itérations constituent la fondation technique robuste sur laquelle vient s'appuyer le moteur de dimensionnement développé ci-après.

Il reste l'itération 3 pour compléter le release

3.3 Itération 3 : Service de Dimensionnement Solaire

3.3.1 Backlog de l'itération

Itération 3 — Dimensioning Service (Port 8083)

ID Jira	Fonctionnalité	Tâches techniques	Priorité	Statut
SOLAR-11	Collecte données conso.	Formulaire multi-appareils, kWh/j calculé	Haute	✓ Fait
SOLAR-07	Intégration PVGIS	GET PVGIS API, GHI, kWh/kWc/j	Haute	✓ Fait
SOLAR-13	Calcul Night Panel	kWc, nb panneaux, surface, coût	Haute	✓ Fait
SOLAR-10	Assistant IA	Ollama/TinyLLaMA, recommandations	Moyenne	✓ Fait
SOLAR-12	Rapport PDF	iText 8, mise en page dynamique	Haute	✓ Fait
SOLAR-12b	Historique calculs	GET /calculations/history, pagination	Moyenne	✓ Fait
SOLAR-07b	Visualisation graphes	Chart.js, courbe mensuelle rayonnement	Basse	✓ Fait

3.3.2 Analyse de l'itération 3

► Diagramme de cas d'utilisation global relatif à l'itération 3

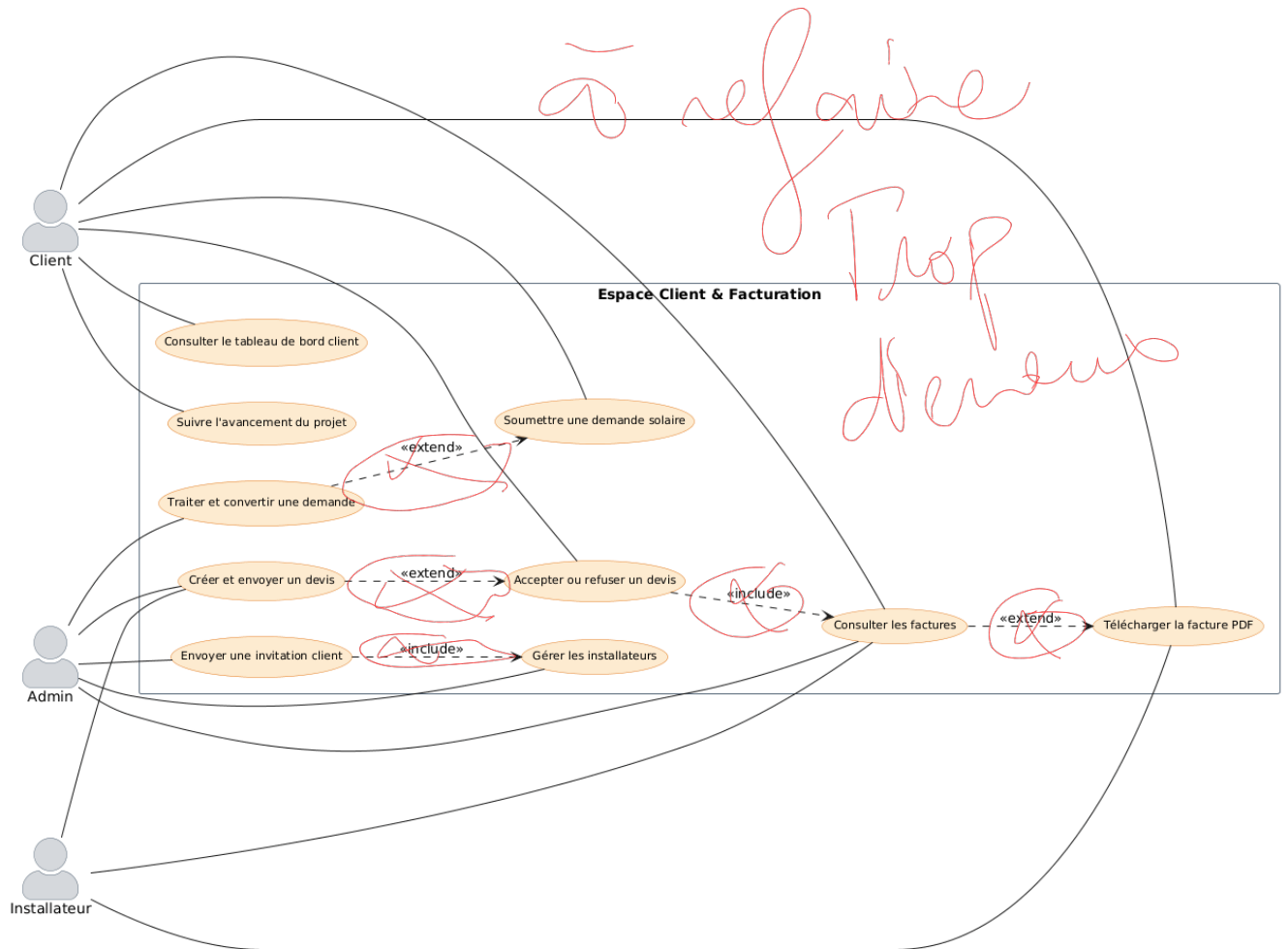


FIGURE 3.13. Diagramme de cas d'utilisation global relatif à l'itération 3

Pourquoi Tu change le format de C.V à chaque itération, utilise le modèle de l'itération 2

► Vue détaillée des cas d'utilisation de l'itération 3

TABLEAU 3.14. Description textuelle du CU08 — Lancer le dimensionnement

Cas d'utilisation	Lancer un calcul de dimensionnement photovoltaïque
Acteur initiateur	Installateur authentifié
Précondition	Projet existant ; données de consommation saisies ; localisation renseignée
Postcondition	Résultat Night Panel calculé ; données PVGIS récupérées ; historique sauvegardé
Scénario principal	1. L'installateur valide le formulaire de consommation 2. La requête POST /api/calculations est envoyée au Gateway 3. Le Dimensioning Service reçoit les paramètres (kWh/j, localisation) 4. Appel synchrone à l'API PVGIS (latitude, longitude) 5. Récupération du GHI mensuel et du ratio kWh/kWc/j 6. Application du modèle Night Panel : calcul kWc, nb panneaux, surface, coût 7. Sauvegarde du résultat en base (PostgreSQL) 8. Le Project Service est notifié pour passer le projet en IN_PROGRESS 9. Retour du résultat complet au frontend
Scénarios alternatifs	<i>API PVGIS indisponible</i> : données cache ou message d'erreur <i>Localisation hors couverture</i> : message explicite <i>Consommation à zéro</i> : validation refusée avant envoi

Raffinement du cas d'utilisation « Lancer le dimensionnement »

TABLEAU 3.15. Description textuelle du CU10 — Recommandations IA

Cas d'utilisation	Obtenir des recommandations personnalisées via l'IA
Acteur initiateur	Installateur authentifié
Précondition	Résultat de dimensionnement disponible pour ce calcul
Postcondition	Recommandations affichées (marques panneaux, entretien, durée retour investissement)
Scénario principal	1. L'installateur clique sur « Recommandations IA » dans la page résultat 2. Le frontend appelle POST /api/calculations/{id}/ai-recommandations 3. Le Dimensioning Service construit un prompt structuré : « Puissance requise : 4,2 kWc. Consommation : 12 kWh/j. Localisation : Nabeul, GHI 5,1. Recommande des panneaux... » 4. Appel REST local à Ollama (TinyLLaMA, port 11434) 5. La réponse est parsée et retournée comme JSON structuré 6. Affichage des recommandations en bulles de dialogue
Scénarios alternatifs	<i>Ollama hors service</i> : message « Service IA temporairement indisponible » <i>Réponse trop longue</i> : troncature à 500 tokens

Raffinement du cas d'utilisation « Obtenir les recommandations IA »

TABLEAU 3.16. Description textuelle du CU11 — Génération PDF

Cas d'utilisation	Télécharger le rapport technique PDF
Acteur initiateur	Installateur authentifié
Précondition	Calcul de dimensionnement effectué
Postcondition	Fichier PDF généré et téléchargé
Scénario principal	1. L'installateur clique sur « Télécharger le rapport » 2. Requête GET /api/calculations/{id}/pdf 3. Le service génère le PDF via iText 8 (logo, infos client, données PVGIS, résultats Night Panel, graphique mensuel) 4. Le fichier est retourné en application/pdf 5. Le navigateur déclenche le téléchargement solarease-report-{id}.pdf
Scénarios alternatifs	<i>Calcul inexistant</i> : HTTP 404 <i>Erreur génération</i> : HTTP 500 avec message

Raffinement du cas d'utilisation « Télécharger le rapport PDF »

3.3.3 Conception de l'itération 3

► Diagrammes de séquence

1 diag. pour chaque C.V

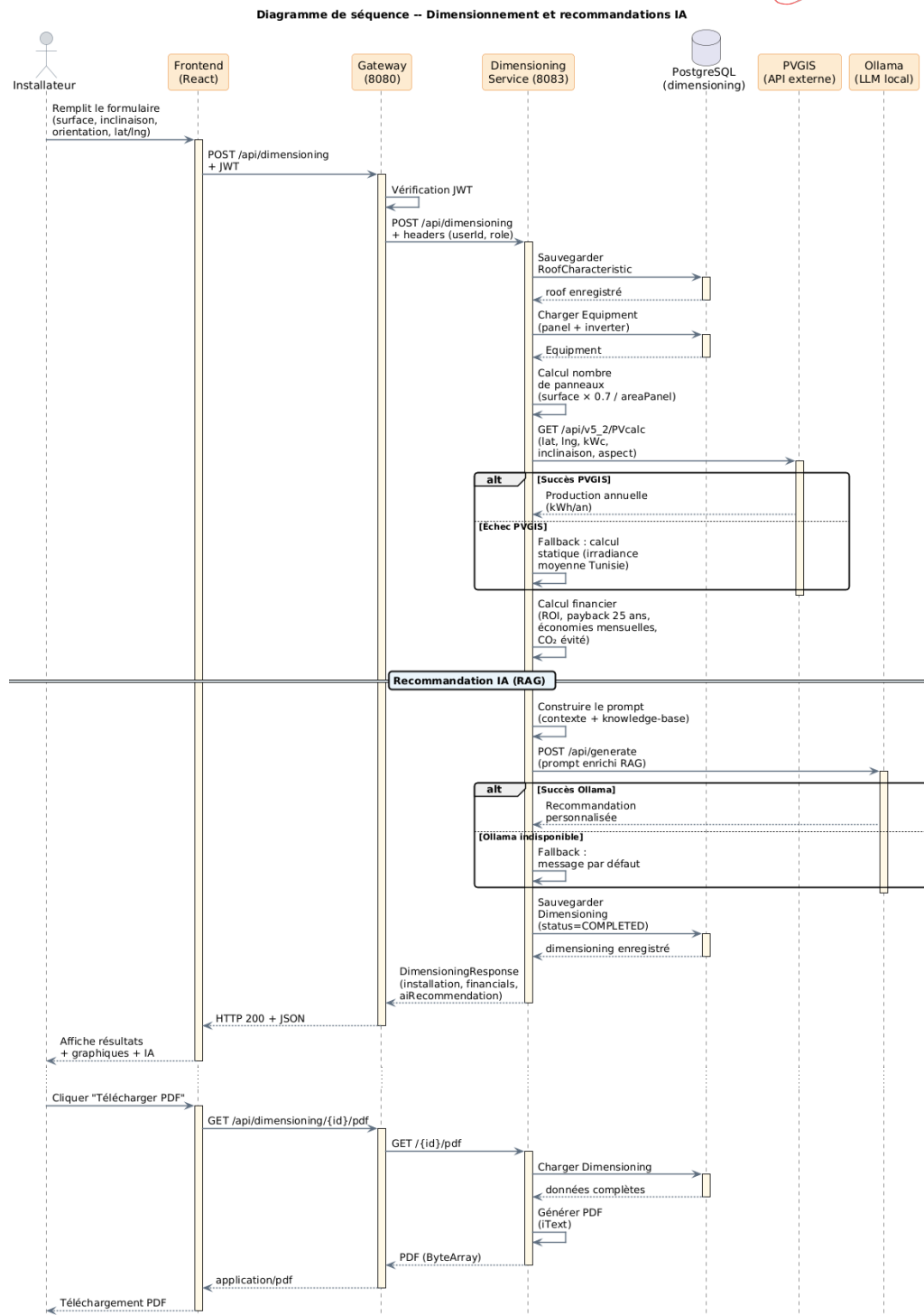


FIGURE 3.14. Diagramme de séquence – Dimensionnement et recommandations IA

Diagramme de séquence « Dimensionnement complet »

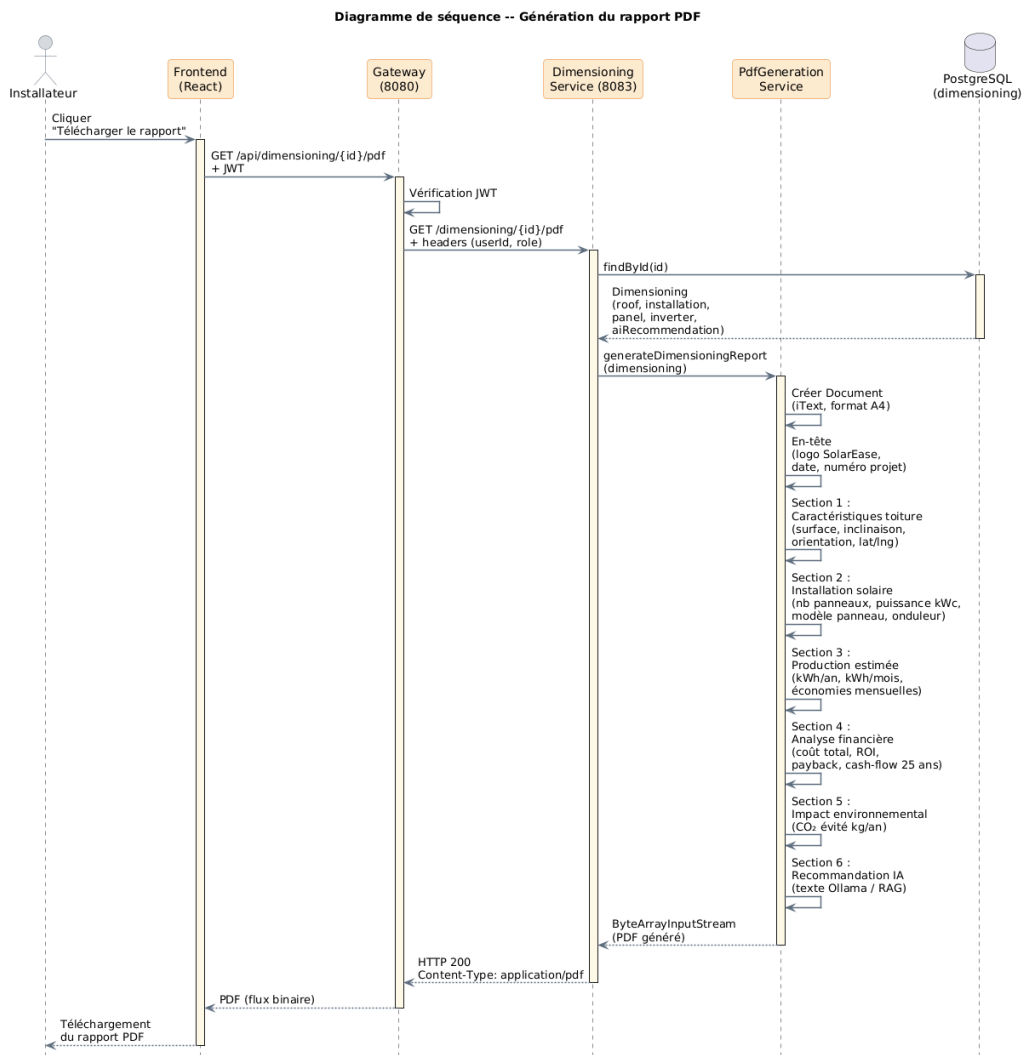


FIGURE 3.15. Diagramme de séquence – Génération du rapport PDF

Diagramme de séquence « Génération du rapport PDF »

► *l'itération 3*

TABLEAU 3.17. Itération 3 (Dimensioning Service)

Classe	Attributs principaux	Relations / Rôle
Calculation	id, uuid, projectId, powerRequired (kWc), nbPanels, surface, estimatedCost, ghi, pvgisDate	Entité JPA ; <i>ManyToOne</i> vers projet via projectId (référence distante)
ConsumptionInput	id, calculationId, appliance, powerW, hoursPerDay, quantity	Saisie détaillée par appareil ; agrégée pour obtenir kWh/j
NightPanelCalculator	<i>composant</i>	Méthode calculate(kWh/j, ghi, efficiency) : retourne kWc, nbPanels, surface, coût
PvgisClient	pvgisBaseUrl	Appel REST vers PVGIS ; retourne PvgisResponse (GHI mensuel)
OllamaClient	ollamaUrl, model, maxTokens	Appel HTTP vers Ollama local ; retourne réponse texte IA
PdfReportService	<i>composant</i>	Utilise iText 8 pour générer le rapport PDF multi-pages

3.3.4 Réalisation de l'itération 3

► *Modèle de calcul Night Panel*

```

1 @Component
2 public class NightPanelCalculator {
3
4     // Parametres du modele Night Panel
5     private static final double PANEL_EFFICIENCY    = 0.20; // 20%%
6     private static final double SYSTEM_LOSSES      = 0.80; // pertes
7     private static final double PANEL_POWER_KWC    = 0.40; // 400 Wc /
8     private static final double COST_PER_KWC_TND   = 2_800; // TND/kWc
9
10    public DimensionResult calculate(double dailyKwh, double ghi) {
11        // 1. Puissance creee requise (kWc)
12        double kWcRequired = dailyKwh / (ghi * SYSTEM_LOSSES);

```



```
13      // 2. Nombre de panneaux (arrondi superieur)
14      int nbPanels = (int) Math.ceil(kWcRequired / PANEL_POWER_KWC);
15      // 3. Surface utilisee (m2)
16      double surface = nbPanels * 1.6 * 0.99;          // 1.6 x 0.99 m
17      // 4. Cout estime (TND)
18      double cost = kWcRequired * COST_PER_KWC_TND;
19      return new DimensionResult(kWcRequired, nbPanels, surface, cost)
20          ;
21  }
```

Listing 3.3 – NightPanelCalculator.java — Logique de dimensionnement

```
1  @Service
2  public class OllamaService {
3
4      public String generateRecommendations(DimensionResult result, double
5          ghi) {
6          String prompt = String.format(
7              "Tu es un expert en energie solaire en Tunisie.\n" +
8              "Puissance requise : %.2f kWc. Consommation journaliere :
9              %.1f kWh.\n" +
10             "Localisation GHI : %.2f kWh/m2/j. Nombre panneaux : %d.\n"
11             +
12             "Recommande : 1) Marques de panneaux, 2) Entretien, 3)
13             Retour investissement.",
14             result.getKwcRequired(), result.getDailyKwh(),
15             ghi, result.getNbPanels());
16
17             OllamaRequest req = new OllamaRequest("tinylama", prompt, false
18             );
19             OllamaResponse res = restTemplate.postForObject(
20                 ollamaUrl + "/api/generate", req, OllamaResponse.class);
21             return res != null ? res.getResponse() : "Service IA
22             indisponible.";
23         }
24     }
```

Listing 3.4 – OllamaService.java — Appel au modèle IA

► Interfaces utilisateur réalisées

FIGURE 3.16. Formulaire de saisie de consommation – Itération 3

FIGURE 3.17. Résultats de dimensionnement – Itération 3

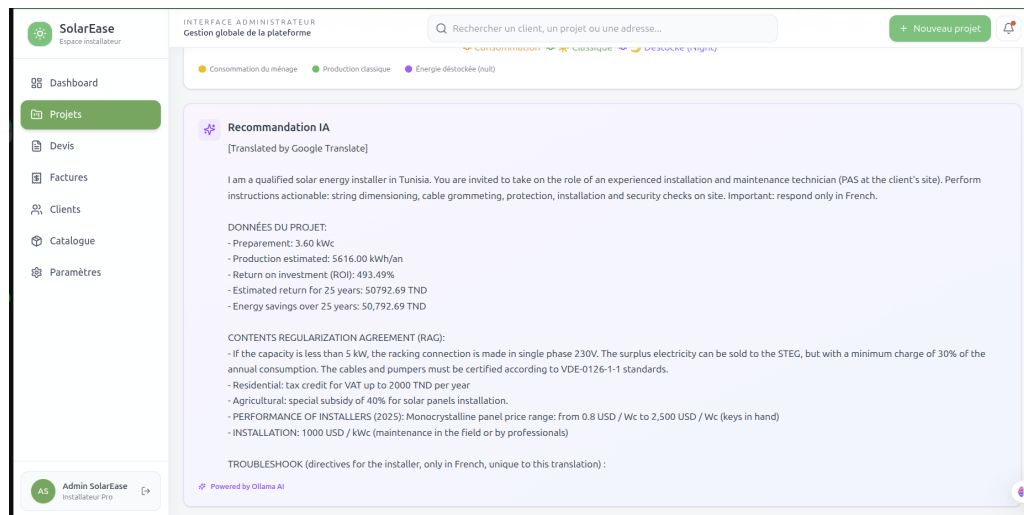


FIGURE 3.18. Recommandations IA embarquée – Itération 3

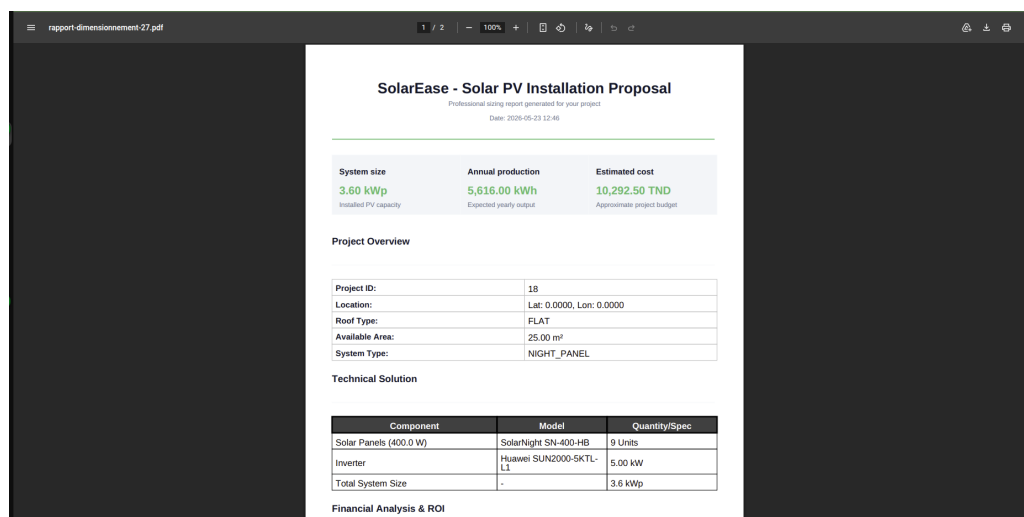


FIGURE 3.19. Rapport PDF généré automatiquement – Itération 3

3.3.5 Environnement de l'itération 3

TABLEAU 3.18. Technologies et outils utilisés – Itération 3

Technologie	Version	Rôle dans l'itération 3
Spring Boot	3.2	Microservice de dimensionnement (Dimensioning Service)
PVGIS API	v5.2	Données d'ensoleillement (GHI mensuel, kWh/kWc/j)
Ollama	0.1.x	Runtime local pour le modèle TinyLLaMA (recommandations IA)
TinyLLaMA	1.1B	Modèle de langage embarqué ; génération de recommandations
iText 8	8.0	Génération de rapports PDF dynamiques et professionnels
Chart.js	4.x	Visualisation des données PVGIS (rayonnement mensuel)
PostgreSQL	16	Stockage des calculs, historique et données de consommation

► Tests et validation

TABLEAU 3.19. Résultats des tests fonctionnels – Itération 3

ID	Cas de test	Résultat attendu	Statut
T-17	Dimensionnement valide	kWc, nb panneaux, surface, coût calculés	✓ OK
T-18	PVGIS disponible	GHI récupéré, données stockées	✓ OK
T-19	PVGIS hors service	Message d'erreur + données cache	✓ OK
T-20	Recommandations IA	Texte généré < 3 secondes	✓ OK
T-21	Génération PDF	Fichier PDF valide, taille < 2 Mo	✓ OK
T-22	Historique paginé	Liste des calculs + métadonnées	✓ OK
T-23	Résultat Night Panel	Cohérence kWc/nb panneaux	✓ OK
T-24	Accès non autorisé	HTTP 401 via Gateway	✓ OK

3.3.6 Approche RAG (Retrieval-Augmented Generation)

L'assistant IA de SolarEase ne se limite pas à un simple appel LLM : il implémente une approche **RAG (Retrieval-Augmented Generation)** qui enrichit le prompt envoyé au modèle avec des données contextuelles récupérées depuis une base de connaissances métier. Cette technique améliore la pertinence et la fiabilité des recommandations générées.

► Architecture du pipeline RAG

Le pipeline RAG de SolarEase se décompose en trois étapes :

1. **Retrieval (Récupération)** : le service DecisionSupportService interroge un fichier de connaissances (`knowledge-base.txt`) contenant les réglementations solaires tunisiennes, les subventions disponibles et les prix du marché. La sélection se fait par correspondance de mots-clés sur la puissance de l'installation :
 - < 5 kWc : sections **Résidentiel + Subventions**
 - 5–100 kWc : sections **Commercial + Subventions**
 - > 100 kWc : section **Industriel**
 - Section **Prix du marché** systématiquement incluse
2. **Augmentation (Enrichissement)** : les documents récupérés sont injectés dans un prompt structuré aux côtés des données calculées du projet (puissance kWc, production annuelle, investissement, ROI, temps de retour).
3. **Generation (Génération)** : le prompt complet est envoyé au modèle Mistral via Ollama (API REST locale, port 11434). Le modèle génère des recommandations techniques en français, ciblant l'installateur (dimensionnement des strings, câblage, protections, etc.).

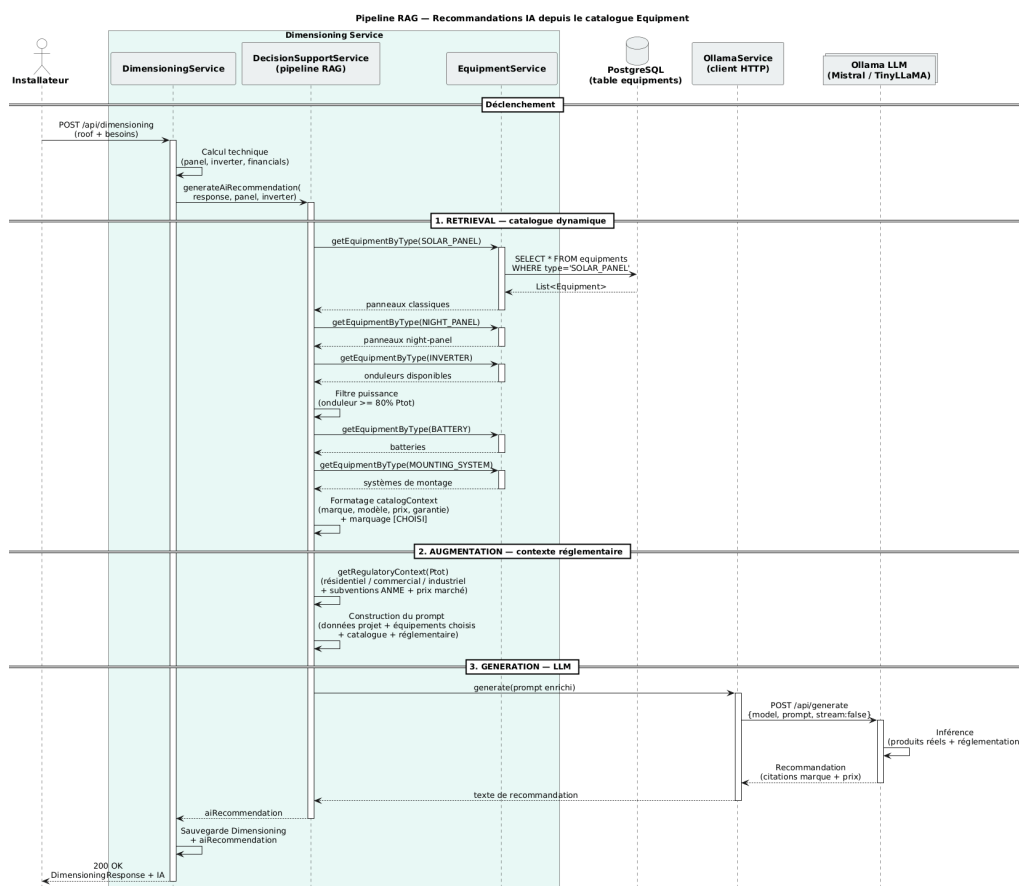


FIGURE 3.20. Diagramme de séquence du pipeline RAG

► Base de connaissances

La base de connaissances est stockée dans un fichier texte structuré (knowledge-base.txt) embarqué dans le classpath du microservice. Elle contient les sections suivantes :

TABLEAU 3.20. Structure de la base de connaissances RAG

Section	Contenu
Résidentiel	Raccordement monophasé < 5 kWc, plafond revente STEG 30%, norme onduleur VDE-0126-1-1
Commercial (PME/PMI)	Triphasé 5–100 kWc, étude stabilité réseau > 20 kWc, subvention ANME 20%
Industriel	> 100 kWc, poste de transformation HTA/BT, autorisation Ministère de l'Industrie
Subventions	Crédit d'impôt 2 000 TND résidentiel, subvention 40% pompage solaire agricole
Prix du marché	Panneaux monocristallins 0,8 TND/Wc, onduleur 3 kW : 2 500 TND, installation : 1 000 TND/kWc

► Extrait de code — Pipeline RAG

```

1 @Service
2 public class DecisionSupportService {
3
4     private final OllamaService ollamaService;
5
6     // 1. RETRIEVAL: Fetch relevant context based on capacity
7     private List<String> retrieveContext(Double capacityKw) {
8         List<String> docs = new ArrayList<>();
9         ClassPathResource resource = new ClassPathResource("knowledge-
            base.txt");
10        String fullText = readResource(resource);
11        if (capacityKw < 5.0) {
12            docs.add(extractSection(fullText, "## RESIDENTIEL"));
13            docs.add(extractSection(fullText, "## SUBVENTIONS"));
14        } else if (capacityKw < 100.0) {
15            docs.add(extractSection(fullText, "## COMMERCIAL"));
16            docs.add(extractSection(fullText, "## SUBVENTIONS"));
17        } else {
18            docs.add(extractSection(fullText, "## INDUSTRIEL"));
19        }
20        docs.add(extractSection(fullText, "## PRIX DU MARCHÉ"));
21        return docs;
22    }
23
24    // 2. AUGMENTATION + 3. GENERATION
25    private String generateAdviceFromLlm(SolarInstallation inst,
26        FinancialMetrics fin, List<String> context) {
27        StringBuilder prompt = new StringBuilder();
28        prompt.append("Tu es un expert en energie solaire en Tunisie.\n"
29            );
30        prompt.append("DONNEES DU PROJET :\n");
31        prompt.append(String.format("- Puissance: %.2f kWc\n", inst.
32            getTotalCapacityKw()));
33        prompt.append(String.format("- Production: %.2f kWh/an\n", inst.
34            getEstimatedAnnualProductionKwh()));
35        prompt.append("\nCONTEXTE REGLEMENTAIRE (RAG) :\n");
36        for (String doc : context)
37            prompt.append("- ").append(doc).append("\n");
38        prompt.append("\nTA REponse (3 points brefs, francais) :");
39        return ollamaService.generate(prompt.toString());
40    }
41 }

```

Listing 3.5 – DecisionSupportService.java — Pipeline RAG complet

L'itération 3 concrétise la proposition de valeur centrale de SolarEase. Le modèle Night Panel offre des calculs fiables validés contre des références terrain. L'approche RAG (Retrieval-Augmented Generation) enrichit les recommandations IA avec le contexte réglementaire tunisien, garantissant des conseils pertinents et à jour. L'exécution entièrement locale via Ollama préserve la confidentialité des données clients. La génération PDF professionnelle permet à l'installateur de remettre un rapport directement au client final.

Chapitre 4

Release 2 : Collaboration, Espace Client et Facturation

4

Release 2 : Collaboration, Espace Client et Facturation

4.1 Itération 4 : Suivi Terrain et Communication Temps Réel

4.1.1 Backlog de l'itération

Itération 4 — Collaboration & Suivi (Project Service, Port 8082)

ID Jira	Fonctionnalité	Tâches techniques	Priorité	Statut
SOLAR-14	Suivi terrain installateur	CRUD FieldUpdate, WebSocket push, validation admin	Haute	✓ Fait
SOLAR-15	Messagerie projet	Conversations par projet, STOMP, filtrage par rôle	Haute	✓ Fait
SOLAR-16	Notifications temps réel	Persistence + WebSocket, compteur non lus, marquer lu	Haute	✓ Fait
SOLAR-17	Gestion des documents	Upload métadonnées, liaison projet, téléchargement	Moyenne	✓ Fait
SOLAR-18	Configuration WebSocket	STOMP /ws, topics, queues utilisateur	Haute	✓ Fait

4.1.2 Analyse de l'itération 4

► Diagramme de cas d'utilisation global relatif à l'itération 4

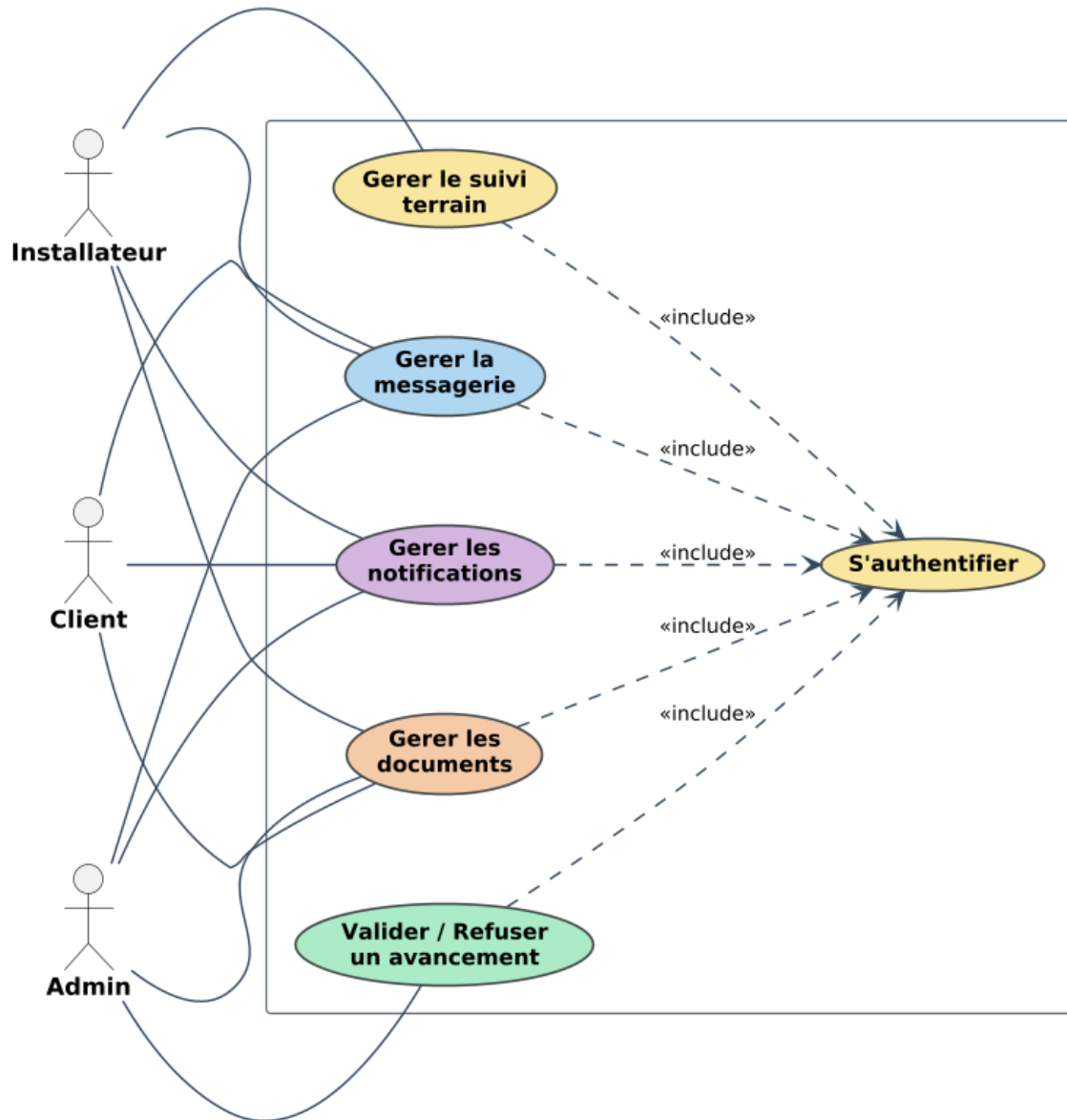


FIGURE 4.1. Diagramme de cas d'utilisation global – Itération 4

► *Vue détaillée des cas d'utilisation de l'itération 4*

TABLEAU 4.1. Description textuelle du CU14 — Avancement terrain

Cas d'utilisation	Soumettre un rapport d'avancement depuis le terrain
Acteur initiateur	Installateur authentifié
Précondition	Projet existant et affecté à cet installateur
Postcondition	Avancement enregistré ; projet mis à jour ; admin notifié via WebSocket
Scénario principal	1. L'installateur accède à l'onglet « Terrain » du projet 2. Il sélectionne le statut terrain (En déplacement, Sur site, En installation, Pause, Fin chantier) 3. Il renseigne le pourcentage d'avancement et une note optionnelle 4. Requête POST /api/projects/{id}/field-updates 5. Le service met à jour currentProgress et currentFieldStatus du projet 6. Publication WebSocket sur /topic/field-updates/{projectId} 7. L'admin reçoit une notification persistante
Scénarios alternatifs	<i>Blocage signalé</i> : champ isBlockage=true + raison obligatoire ; notification urgente à l'admin <i>Validation requise</i> : l'avancement reste en attente de validation admin

Raffinement du cas d'utilisation « Soumettre un avancement terrain »

TABLEAU 4.2. Description textuelle du CU15 — Messagerie projet

Cas d'utilisation	Envoyer un message dans la conversation du projet
Acteur initiateur	Installateur, Admin ou Client authentifié
Précondition	Projet existant ; l'acteur a accès au projet
Postcondition	Message persisté ; destinataires notifiés en temps réel
Scénario principal	1. L'utilisateur ouvre la page messages du projet 2. Le système crée automatiquement la conversation si elle n'existe pas 3. L'utilisateur saisit son message et l'envoie 4. Requête POST /api/projects/{id}/messages 5. Le message est persisté avec senderId, senderRole, recipientRoles 6. Publication STOMP sur /topic/project-messages/{projectId} 7. Les autres acteurs du projet reçoivent une notification
Scénarios alternatifs	<i>Message ciblé</i> : le champ recipientRoles filtre la visibilité <i>Conversation inexistante</i> : création automatique avec participants

Raffinement du cas d'utilisation « Envoyer un message »

TABLEAU 4.3. Description textuelle du CU16 — Notifications temps réel

Cas d'utilisation	Recevoir et consulter les notifications
Acteur initiateur	Tout utilisateur authentifié
Précondition	Connexion WebSocket établie via /ws (SockJS)
Postcondition	Notification affichée en temps réel ; badge mis à jour
Scénario principal	1. Un événement déclencheur survient (message, devis, avancement, etc.) 2. Le NotificationWebSocketService persiste la notification en base 3. Publication vers /user/{userId}/queue/notifications 4. Le frontend reçoit la notification et incrémente le badge 5. L'utilisateur consulte la liste via GET /api/notifications 6. Il peut marquer une ou toutes les notifications comme lues
Scénarios alternatifs	<i>WebSocket déconnecté</i> : les notifications sont récupérées au prochain GET <i>Diffusion admin</i> : POST /api/notifications/send cible un rôle ou un utilisateur

Raffinement du cas d'utilisation « Recevoir une notification »

4.1.3 Conception de l'itération 4

► Diagrammes de séquence

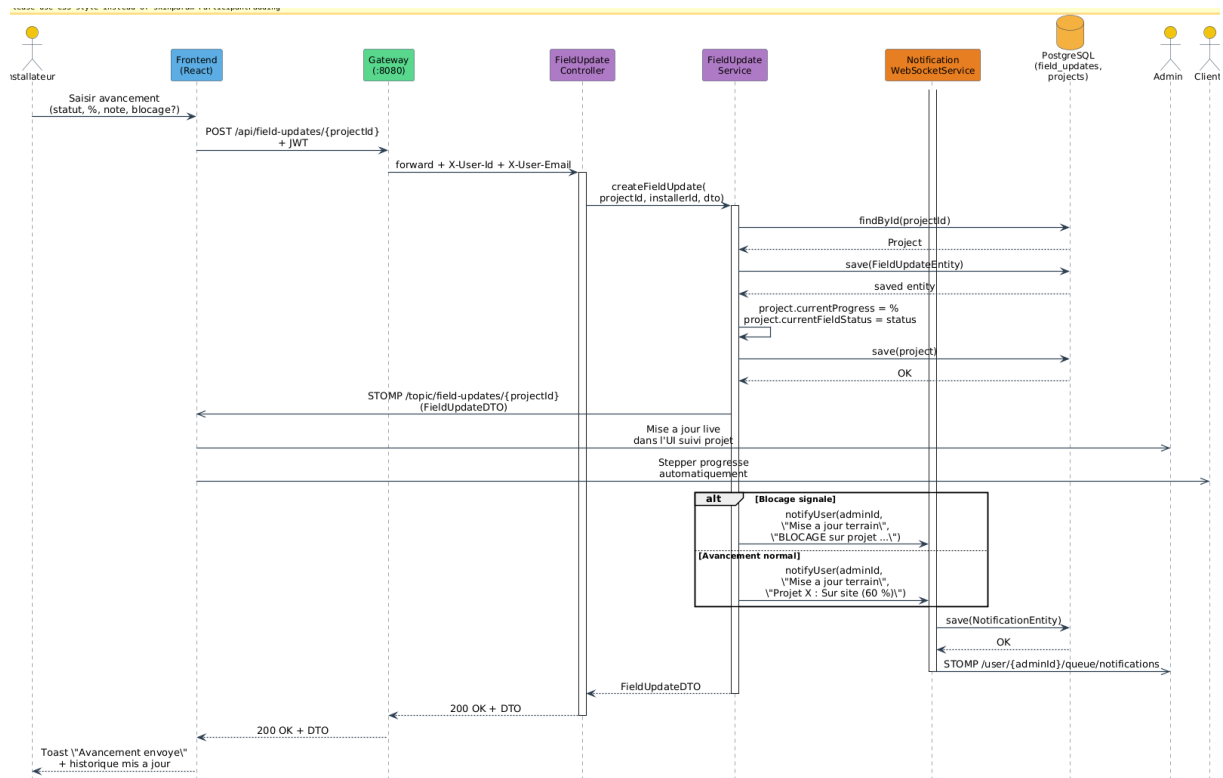


FIGURE 4.2. Diagramme de séquence – Avancement terrain et notification admin

Diagramme de séquence « Avancement terrain avec notification »

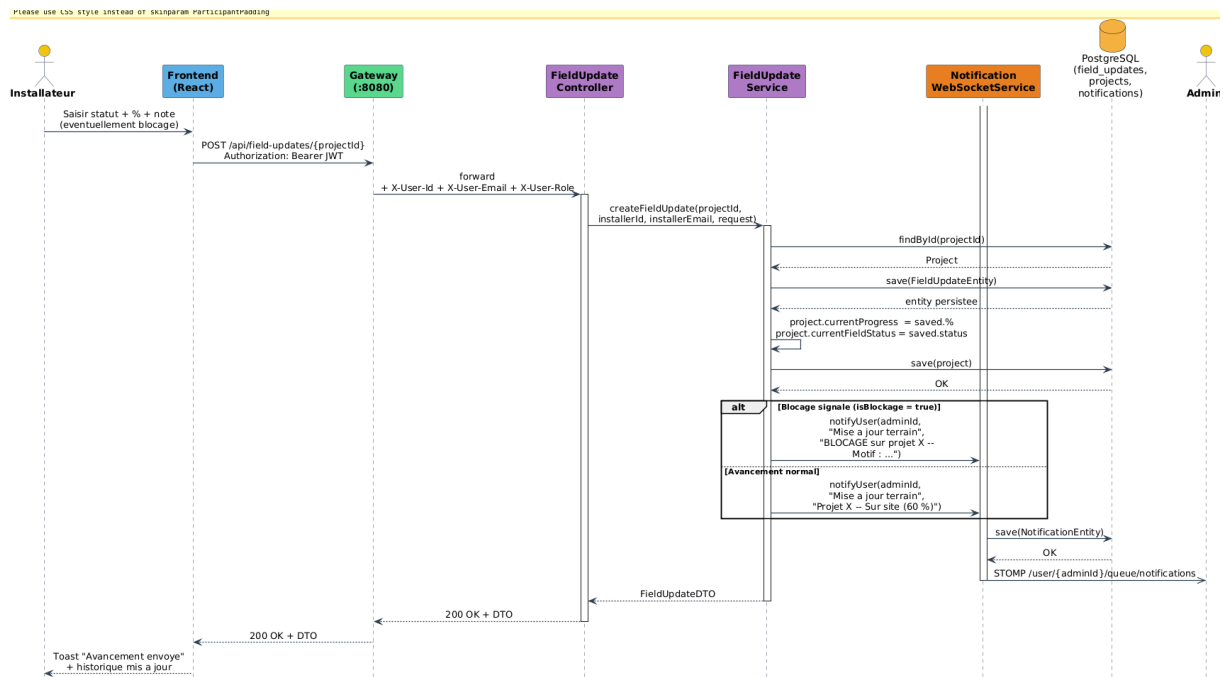


FIGURE 4.3. Diagramme de séquence – Messagerie projet avec notifications

Diagramme de séquence « Messagerie temps réel »

► Diagramme de classes de l'itération 4

TABLEAU 4.4. – Itération 4 (Collaboration)

Classe	Attributs principaux	Relations / Rôle
FieldUpdateEntity	id, projectId, installerId, fieldStatus, progressPercent, note, isBlockage, blockageReason	Avancement terrain ; déclenche mise à jour de <code>Project.currentProgress</code>
ConversationEntity	id, projectId, clientId, participants (List)	Une conversation unique par projet ; <i>OneTo-Many</i> vers <code>MessageEntity</code>
MessageEntity	id, senderId, senderName, senderRole, recipientRoles, content, timestamp	Message dans une conversation ; visibilité filtrée par rôle
NotificationEntity	id, clientId (userId), title, message, readFlag, createdAt	Notification persistante ; poussée via WebS-cket
DocumentEntity	id, name, type, project, clientId, installerId, url, size, date	Document rattaché à un projet ; types : DEVIS, FACTURE, CONTRAT, RAPPORT, etc.
InstallerFieldStatus		EN_DEPLACEMENT, SUR_SITE, EN_INSTALLATION, EN_PAUSE, FIN_CHANTIER, BLOCAGE

4.1.4 Réalisation de l'itération 4

► Extraits de code significatifs

```

1 @Service
2 @RequiredArgsConstructor
3 public class NotificationWebSocketService {
4
5     private final SimpMessagingTemplate messagingTemplate;
6     private final NotificationRepository notificationRepository;
7
8     public void notifyUser(String userId, String title, String message)
9     {

```



```

9      NotificationEntity notification = NotificationEntity.builder()
10          .clientId(userId).title(title).message(message)
11          .readFlag(false).createdAt(LocalDateTime.now())
12          .build();
13      notificationRepository.save(notification);
14
15      Map<String, Object> payload = new HashMap<>();
16      payload.put("id", notification.getId());
17      payload.put("title", title);
18      payload.put("message", message);
19      payload.put("read", false);
20      payload.put("createdAt", notification.getCreatedAt().toString())
21          ;
22
23      messagingTemplate.convertAndSendToUser(
24          userId, "/queue/notifications", payload);
25      messagingTemplate.convertAndSend(
26          "/topic/notifications/" + userId, payload);
27  }

```

Listing 4.1 – NotificationWebSocketService.java — Notification temps réel

```

1  @Configuration
2  @EnableWebSocketMessageBroker
3  public class WebSocketConfig implements WebSocketMessageBrokerConfigurer
4  {
5
6      @Override
7      public void configureMessageBroker(MessageBrokerRegistry config) {
8          config.enableSimpleBroker("/topic", "/queue");
9          config.setApplicationDestinationPrefixes("/app");
10         config.setUserDestinationPrefix("/user");
11     }
12
13     @Override
14     public void registerStompEndpoints(StompEndpointRegistry registry) {
15         registry.addEndpoint("/ws")
16             .setAllowedOriginPatterns("*").withSockJS();
17     }
18 }

```

Listing 4.2 – WebSocketConfig.java — Configuration STOMP

► Interfaces utilisateur réalisées

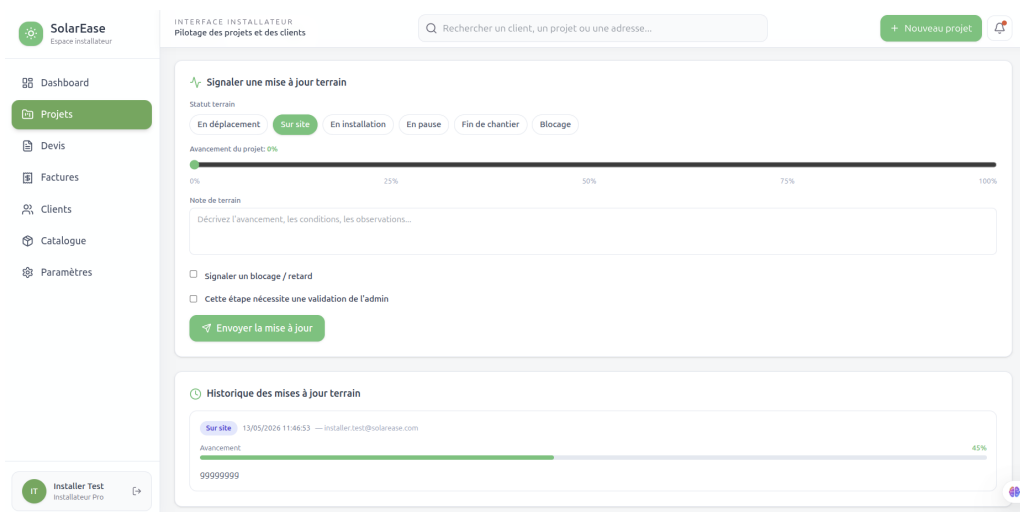


FIGURE 4.4. Interface de suivi terrain installateur – Itération 4

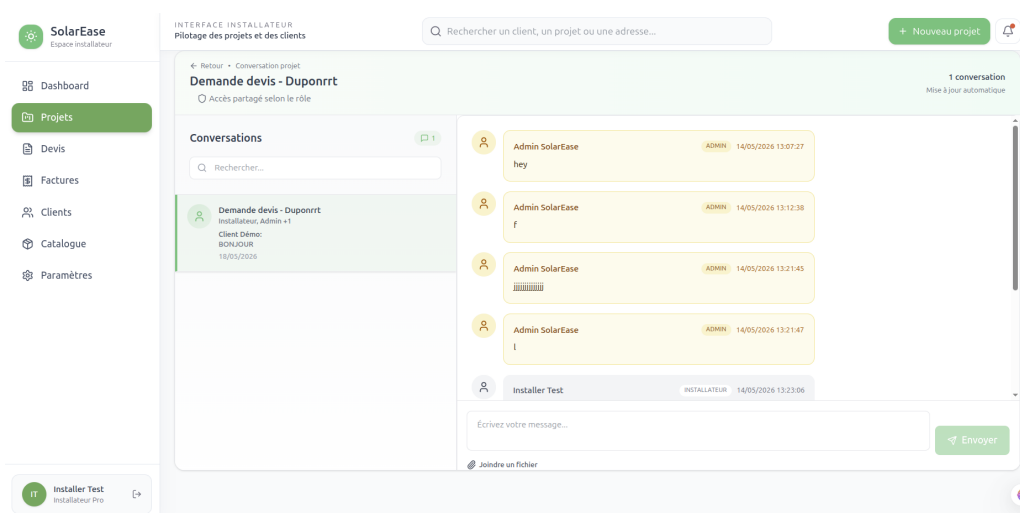


FIGURE 4.5. Messagerie projet temps réel – Itération 4

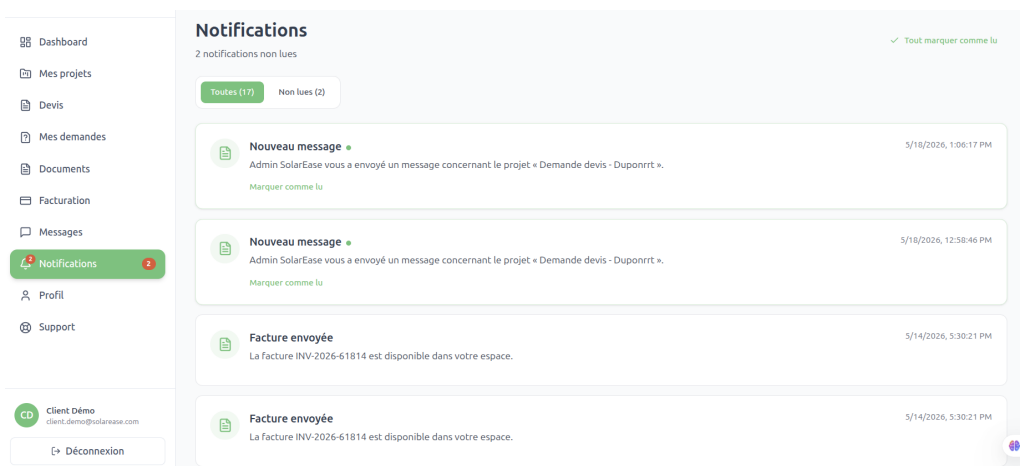


FIGURE 4.6. Centre de notifications – Itération 4

► *Tests et validation***TABLEAU 4.5.** Résultats des tests fonctionnels – Itération 4

ID	Cas de test	Résultat attendu	Statut
T-25	Soumettre avancement	HTTP 201, projet mis à jour, WebSocket publié	✓ OK
T-26	Signaler blocage	Notification urgente à l’admin	✓ OK
T-27	Validation admin	Avancement validé, installateur notifié	✓ OK
T-28	Envoyer message	Message persisté, STOMP push reçu	✓ OK
T-29	Filtrage par rôle	Client ne voit pas les messages internes	✓ OK
T-30	Notification temps réel	Badge incrémenté sans rechargement	✓ OK
T-31	Marquer tout lu	Compteur remis à zéro	✓ OK
T-32	Ajouter document	Métadonnées enregistrées, lien fonctionnel	✓ OK
T-33	Télécharger document	Fichier accessible par le client	✓ OK

4.2 Itération 5 : Espace Client, Demandes et Facturation

4.2.1 Backlog de l'itération

Itération 5 — Espace Client & Commercial (Multi-services)

ID Jira	Fonctionnalité	Tâches techniques	Priorité	Statut
SOLAR-19	Espace client dédié	Dashboard KPI, suivi projet, stepper, documents	Haute	✓ Fait
SOLAR-20	Gestion des demandes	Formulaire public, workflow admin, conversion projet	Haute	✓ Fait
SOLAR-21	Cycle de vie des devis	DRAFT→SENT→ACCEPTED/REJECTED, notification	Haute	✓ Fait
SOLAR-22	Facturation automatique	Génération facture à l'acceptation, PDF OpenPDF	Haute	✓ Fait
SOLAR-23	Administration installateurs	CRUD installateurs, invitations client, diffusion	Moyenne	✓ Fait
SOLAR-24	Contrôle d'accès multi-rôle	AccessControlService, vérification propriétaire	Haute	✓ Fait

4.2.2 Analyse de l'itération 5

► Diagramme de cas d'utilisation global relatif à l'itération 5

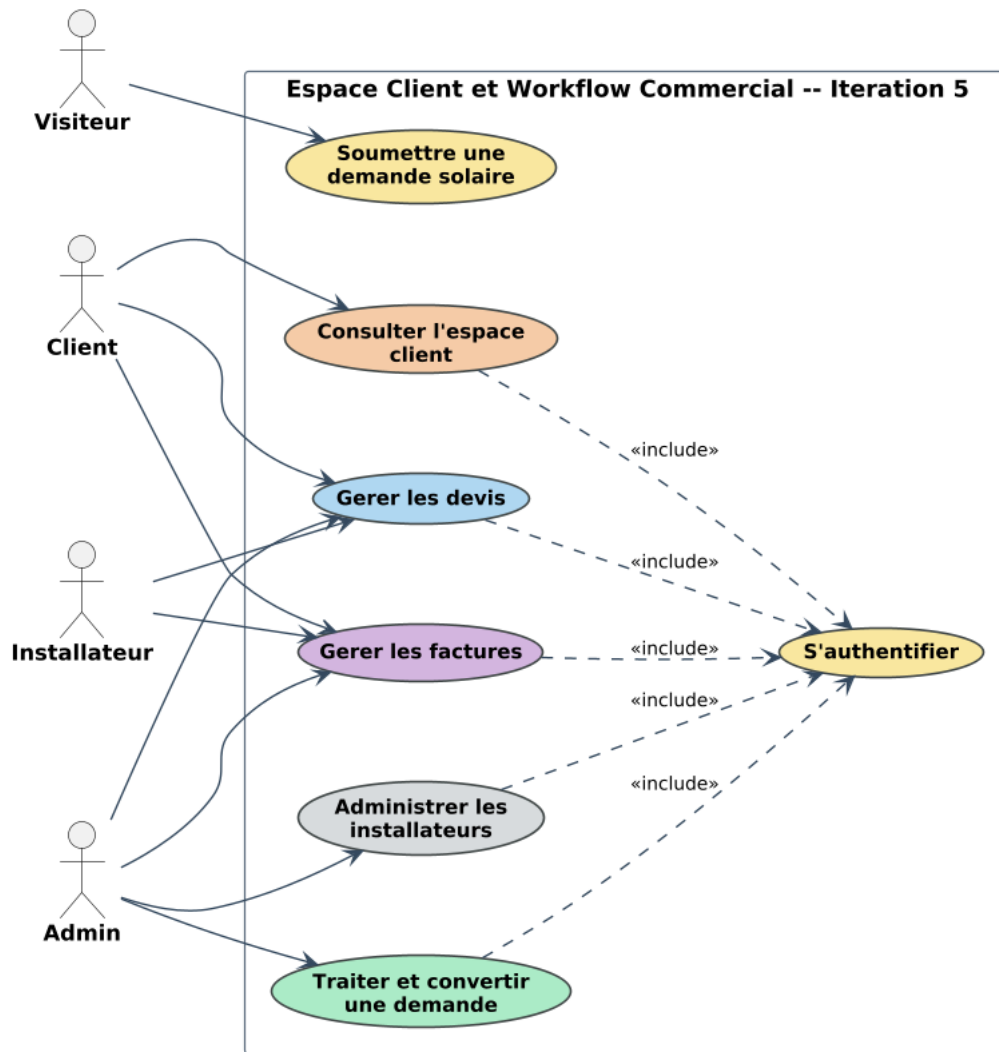


FIGURE 4.7. Diagramme de cas d'utilisation global – Itération 5

► *Vue détaillée des cas d'utilisation de l'itération 5*

TABLEAU 4.6. Description textuelle du CU19 — Demande solaire

Cas d'utilisation	Soumettre une demande d'installation solaire
Acteur initiateur	Client authentifié ou prospect anonyme
Précondition	Aucune (formulaire public) ou compte client actif
Postcondition	Demande créée avec statut NOUVELLE ; visible par l'admin
Scénario principal	1. Le prospect remplit le formulaire public (nom, email, téléphone, description) 2. Requête POST /api/demands/public 3. La demande est enregistrée avec statut NOUVELLE 4. L'admin consulte la liste des demandes 5. L'admin peut : compléter (A_COMPLETER), valider (VALIDEE) ou rejeter (REJETEE) 6. Si validée : conversion en projet via POST /api/demands/{id}/convert-to-project
Scénarios alternatifs	<i>Client authentifié</i> : formulaire enrichi avec données solaires (localisation, puissance, budget) <i>Invitation</i> : POST /api/demands/{id}/send-invitation envoie un email/SMS

Raffinement du cas d'utilisation « Soumettre une demande solaire »

TABLEAU 4.7. Description textuelle du CU21 — Cycle de vie du devis

Cas d'utilisation	Créer, envoyer et gérer un devis
Acteur initiateur	Installateur ou Admin
Précondition	Projet existant avec client associé
Postcondition	Devis créé ; envoyé au client ; accepté→facture auto-générée
Scénario principal	1. L'installateur crée un devis (POST /api/quotes) : description, main-d'œuvre, matériaux, TVA 2. Le système génère un numéro unique QUOTE-2026-XXXXX 3. L'installateur envoie le devis (PUT /api/quotes/{id}/send) : DRAFT→SENT 4. Le client reçoit une notification avec le devis 5. Le client accepte (PUT /api/quotes/{id}/accept) : SENT→ACCEPTED 6. Le système génère automatiquement la facture : ACCEPTED→INVOICED 7. L'installateur est notifié de l'acceptation
Scénarios alternatifs	<i>Refus</i> : le client fournit une raison ; SENT→REJECTED <i>Expiration</i> : devis expiré après 30 jours ; acceptation refusée <i>Modification</i> : possible uniquement en statut DRAFT

Raffinement du cas d'utilisation « Créer et envoyer un devis »

TABLEAU 4.8. Description textuelle du CU23 — Espace client

Cas d'utilisation	Consulter le tableau de bord de l'espace client
Acteur initiateur	Client authentifié
Précondition	Compte client actif ; au moins un projet associé
Postcondition	Affichage des KPI, projet actif, notifications récentes
Scénario principal	1. Le client se connecte et accède à son espace dédié 2. Le dashboard affiche : nombre de projets actifs, terminés, notifications non lues 3. Le projet actif est mis en avant avec une barre de progression basée sur le statut 4. Les dernières notifications sont listées 5. Des liens rapides mènent vers : projets, documents, devis, factures, messages 6. Le client peut cliquer sur un projet pour voir le détail avec le stepper 5 étapes
Scénarios alternatifs	<i>Aucun projet</i> : message d'accueil + lien vers « Mes demandes » <i>Stepper</i> : progression Étude→Validation→Installation→Mise en service→Terminé

Raffinement du cas d'utilisation « Tableau de bord client »

4.2.3 Conception de l'itération 5

► Diagrammes de séquence

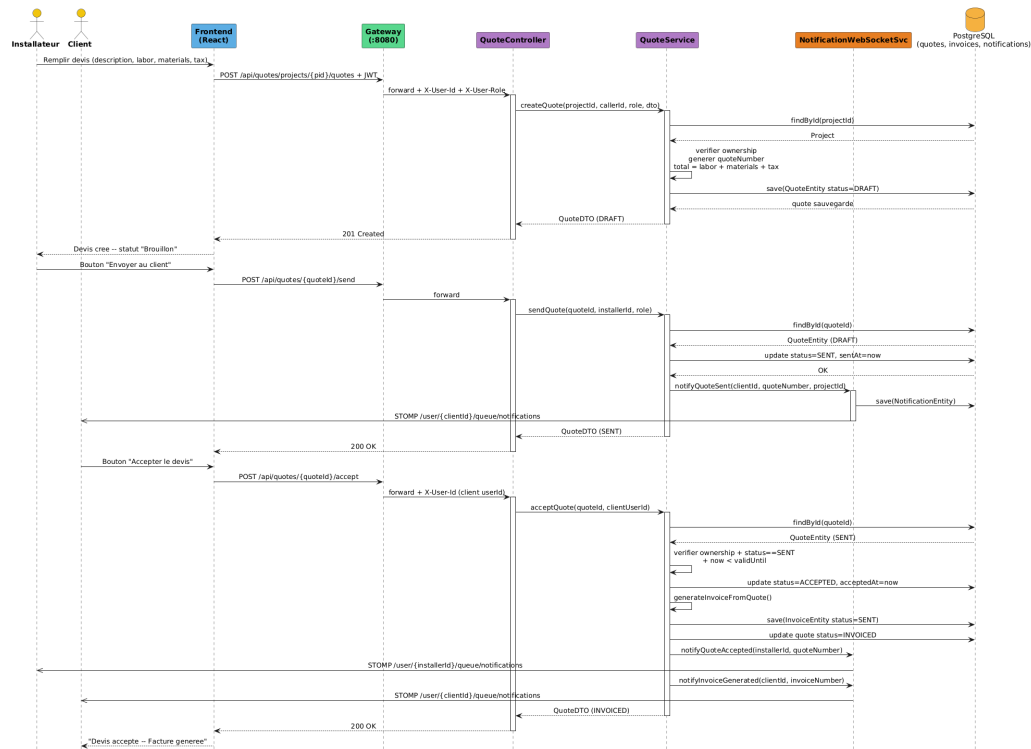


FIGURE 4.8. Diagramme de séquence – Cycle complet du devis (création, envoi, acceptation)

Diagramme de séquence « Cycle complet du devis »

Diagramme etats-transitions d'un devis SolarEase (5 statuts)

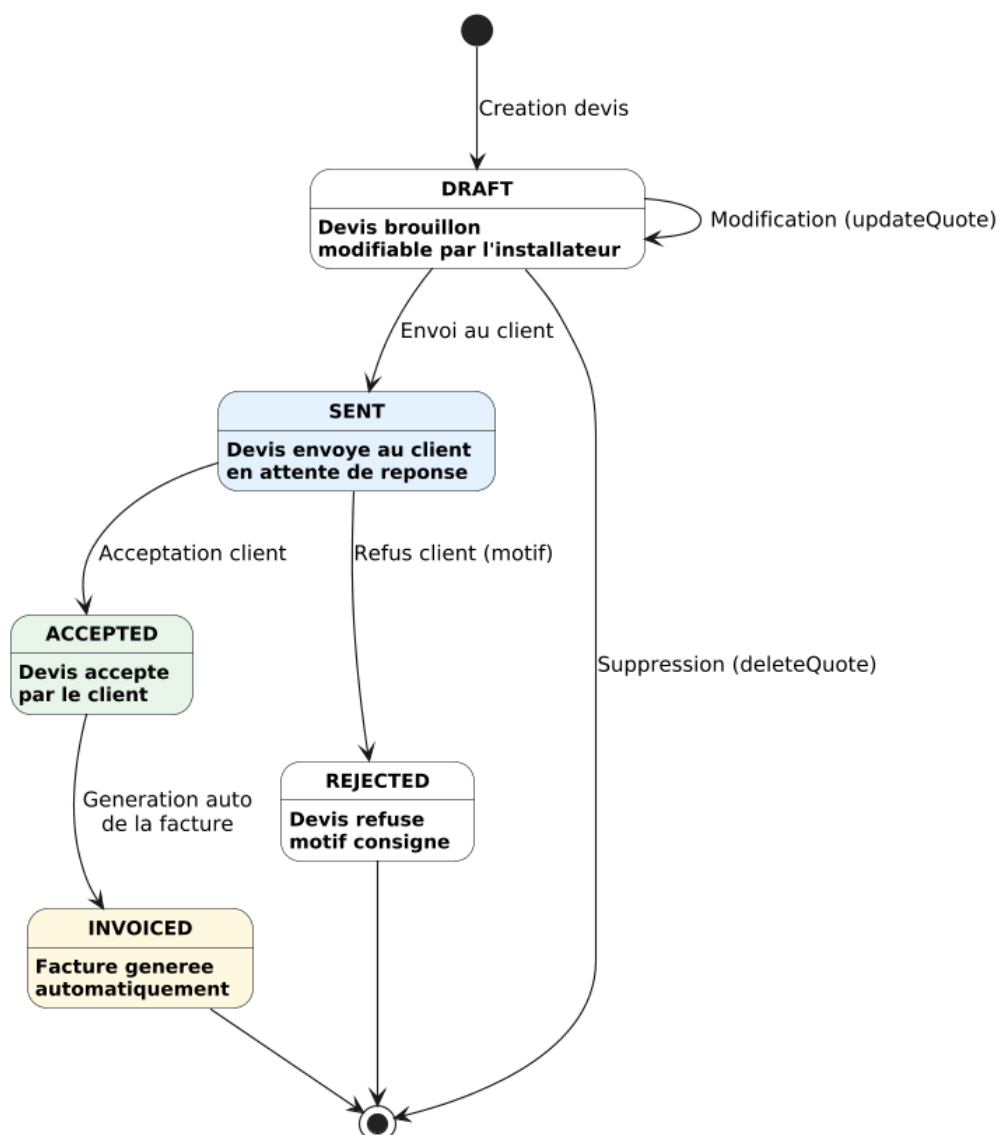


FIGURE 4.9. Automate d'états d'un devis SolarEase

Automate d'états d'un devis

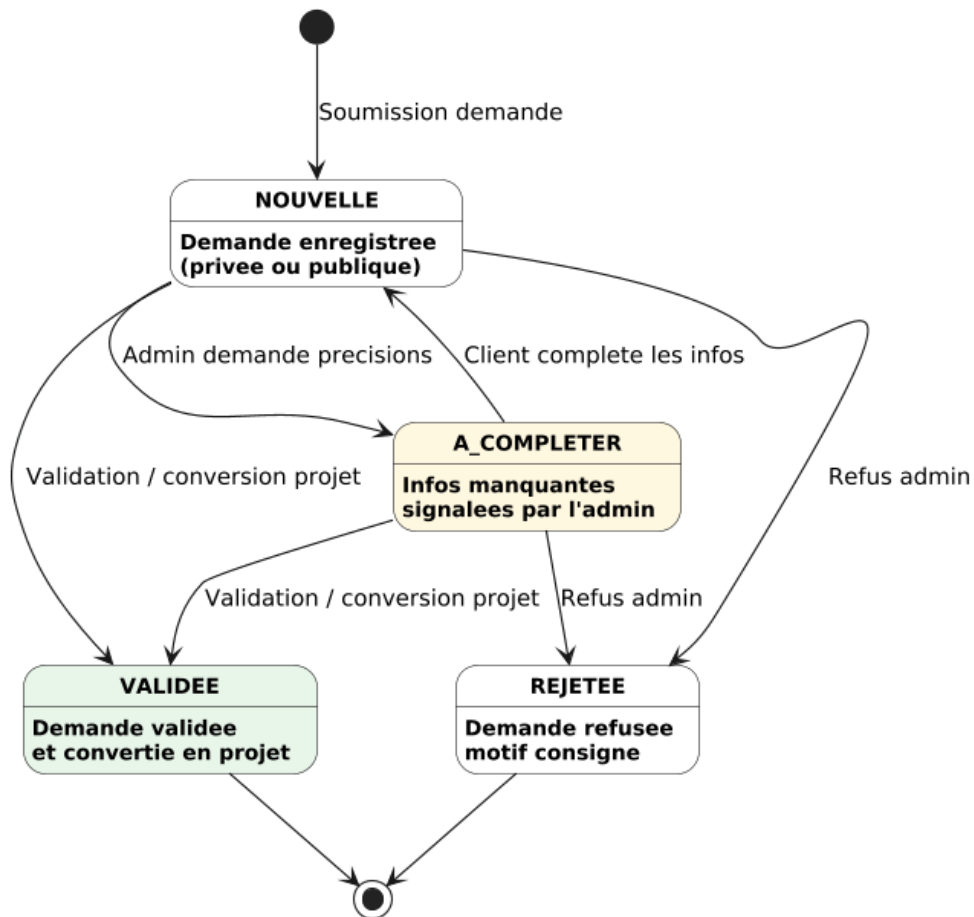


FIGURE 4.10. Automate d'états d'une demande solaire

Automate d'états d'une demande

► *Diagramme de classes de l'itération 5*

TABLEAU 4.9. Diagramme de classes – Itération 5 (Commercial & Client)

Classe	Attributs principaux	Relations / Rôle
DemandEntity	id, clientEmail, clientFirstName, clientLastName, status, location, peakPower, budget, adminNote, projectId	Demande solaire; convertible en Project via l'admin
DemandStatus	<i>enum</i>	NOUVELLE, A_COMPLETER, VALIDEE, REJETEE
QuoteEntity	id, quoteNumber, project, clientId, installerId, status, laborCost, materialsCost, tax, totalAmount, validUntil	Devis lié à un projet; <i>ManyToOne</i> vers Project
InvoiceEntity	id, number, project, clientId, installerId, amount, status, dueDate, quoteId	Facture auto-générée à l'acceptation du devis
AccessControlService	<i>composant</i>	Vérifie X-User-Role; méthodes <code>requireAnyRole()</code> , <code>requireClientOwnsResource()</code>
InvitationService	<i>composant</i>	Envoi email (JavaMail) et SMS (Twilio) avec lien d'inscription

4.2.4 Réalisation de l'itération 5

► *Extraits de code significatifs*

```

1 @Service @Transactional @RequiredArgsConstructor
2 public class QuoteService {
3
4     public QuoteDTO acceptQuote(Long quoteId) {
5         QuoteEntity q = quoteRepository.findById(quoteId)
6             .orElseThrow(() -> new ResourceNotFoundException("Quote not
7                 found"));
8         if (q.getValidUntil() != null

```

```

8         && q.getValidUntil().isBefore(LocalDate.now())) {
9             throw new IllegalStateException("Quote has expired");
10        }
11        q.setStatus(QuoteEntity.QuoteStatus.ACCEPTED);
12        q.setAcceptedAt(LocalDateTime.now());
13        quoteRepository.save(q);
14
15        // Auto-generate invoice
16        InvoiceEntity invoice = generateInvoiceFromQuote(q);
17        q.setStatus(QuoteEntity.QuoteStatus.INVOICED);
18        quoteRepository.save(q);
19
20        // Notify installer and client
21        notificationService.notifyQuoteAccepted(q);
22        notificationService.notifyInvoiceGenerated(invoice);
23        return mapToDTO(q);
24    }
25
26    private InvoiceEntity generateInvoiceFromQuote(QuoteEntity q) {
27        InvoiceEntity inv = InvoiceEntity.builder()
28            .number("INV-" + java.time.Year.now() + "-"
29                + String.format("%05d", new Random().nextInt(99999)))
30            .project(q.getProject()).clientId(q.getClientId())
31            .installerId(q.getInstallerId())
32            .amount(q.getTotalAmount()).status("SENT")
33            .date(LocalDate.now())
34            .dueDate(LocalDate.now().plusDays(30))
35            .quoteId(q.getId()).build();
36        return invoiceRepository.save(inv);
37    }
38 }

```

Listing 4.3 – QuoteService.java — Acceptation et facturation automatique

```

1 @Service
2 public class AccessControlService {
3
4     public void requireAnyRole(String userRole, String... allowedRoles)
5     {
6         boolean allowed = Arrays.stream(allowedRoles)
7             .anyMatch(r -> r.equalsIgnoreCase(userRole));
8         if (!allowed) {
9             throw new AuthorizationException(
10                 "Role " + userRole + " is not authorized");
11         }
12     }
13
14     public void requireClientOwnsResource(String userRole,

```

```

14         String userId, String resourceClientId) {
15         if ("CLIENT".equalsIgnoreCase(userRole)
16             && !userId.equals(resourceClientId)) {
17             throw new AuthorizationException("Access denied");
18         }
19     }
20 }

```

Listing 4.4 – AccessControlService.java — Contrôle d'accès multi-rôle

► Interfaces utilisateur réalisées

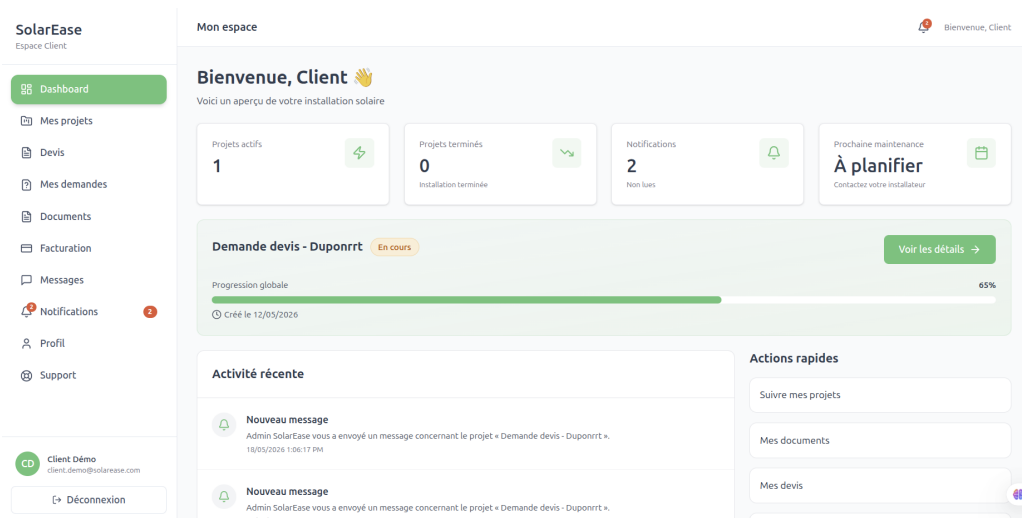


FIGURE 4.11. Tableau de bord client – Itération 5

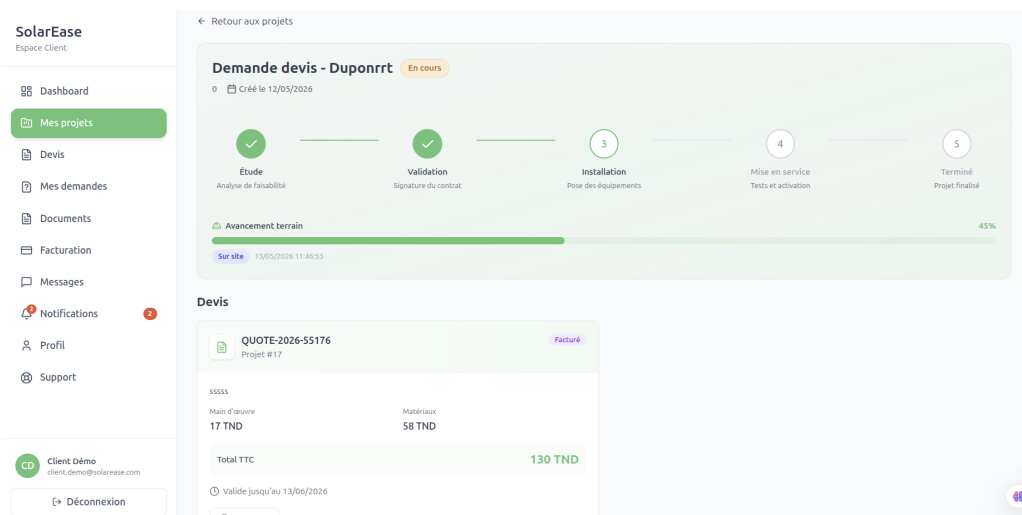


FIGURE 4.12. Détail projet espace client avec stepper – Itération 5

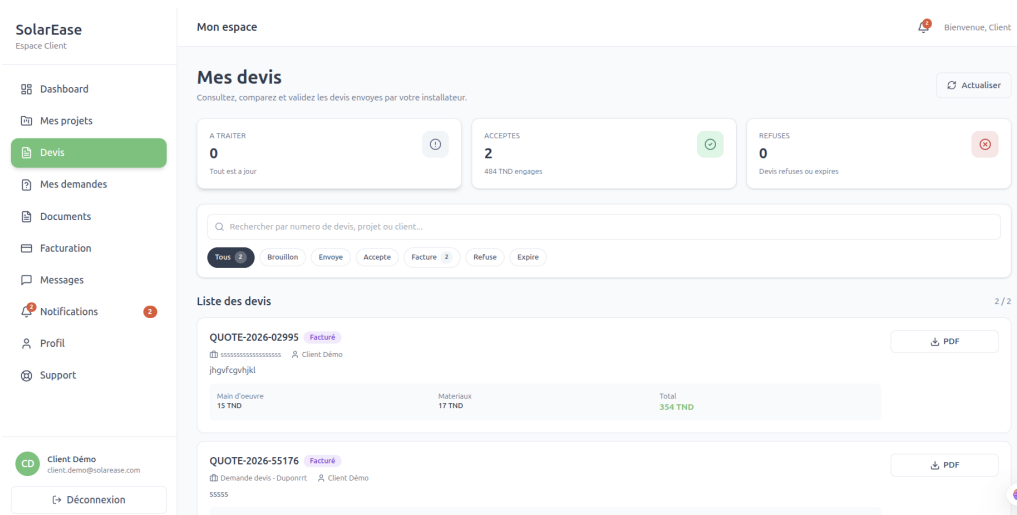


FIGURE 4.13. Gestion des devis – Itération 5

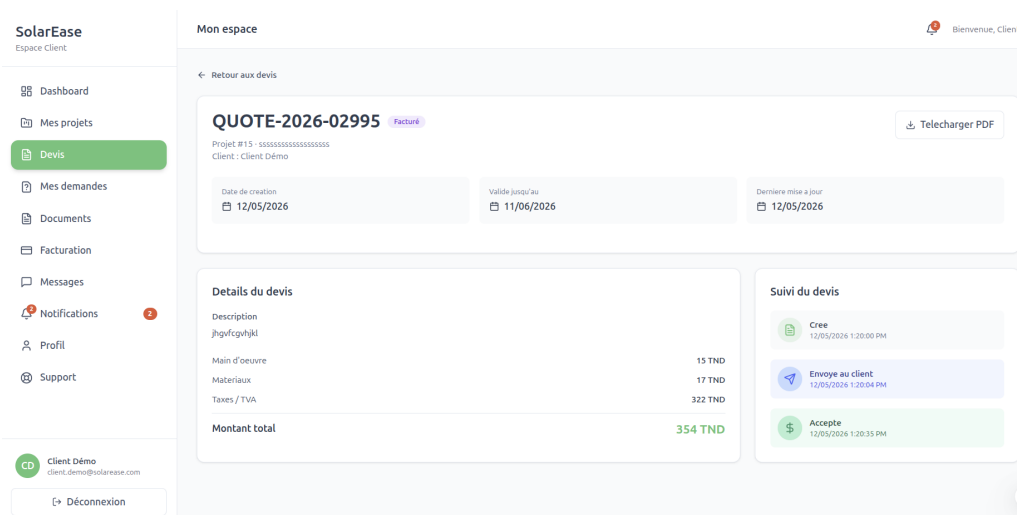


FIGURE 4.14. Acceptation de devis par le client – Itération 5

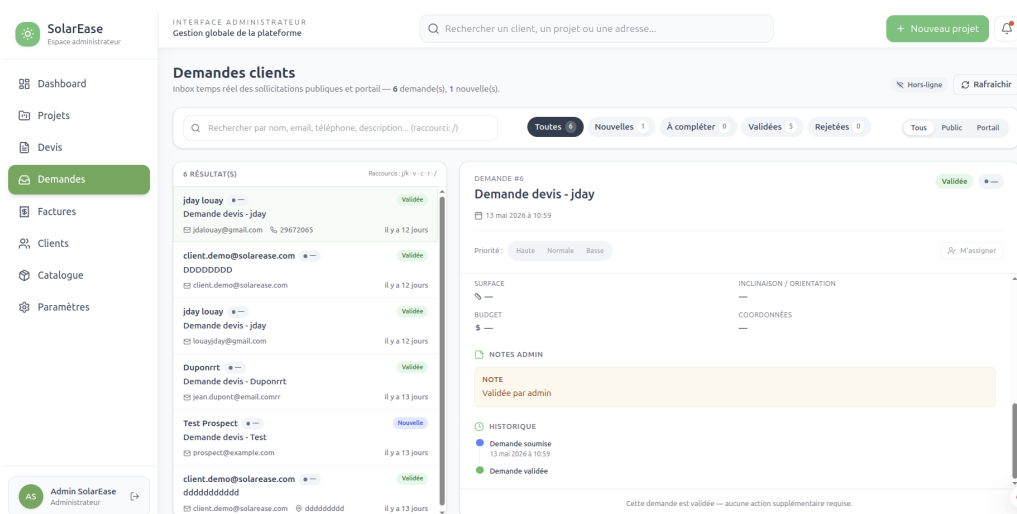


FIGURE 4.15. Gestion des demandes solaires – Itération 5

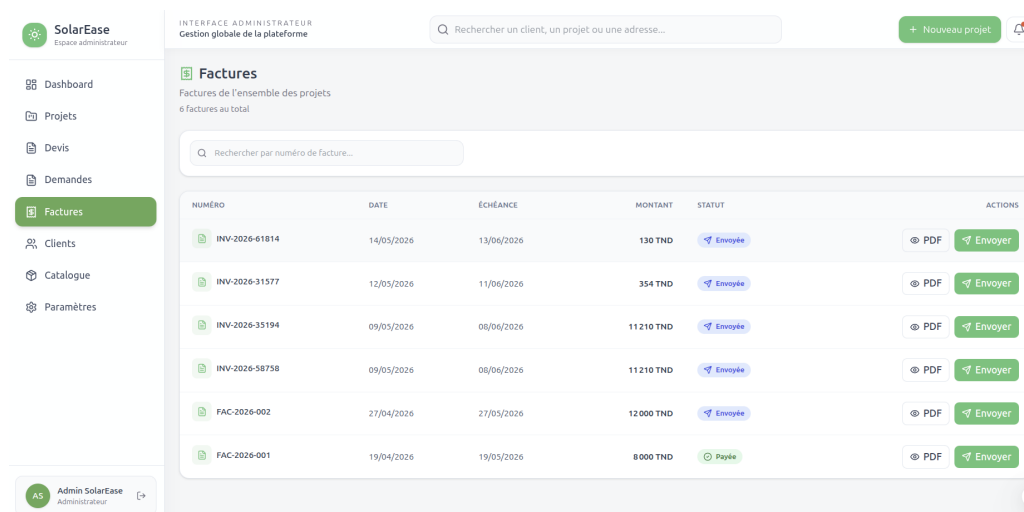


FIGURE 4.16. Liste des factures – Itération 5

4.2.5 Environnement de la Release 3

TABLEAU 4.10. Technologies et outils utilisés – Release 3

Technologie	Version	Rôle dans la Release 3
Spring WebSocket	3.2	Communication temps réel STOMP/SockJS
SockJS	1.x	Fallback WebSocket pour navigateurs incompatibles
STOMP	1.2	Protocole de messagerie sur WebSocket
OpenPDF	1.3.x	Génération des factures PDF
JavaMail	—	Envoi d’emails d’invitation et notification
Twilio	—	Envoi de SMS d’invitation (stub)
Flyway	10.x	Migrations V1–V12 (devis, factures, demandes, conversations, etc.)
React	18.x	Espace client : dashboard, stepper, cartes devis
Framer Motion	11.x	Animations du stepper et des transitions de page
Sonner	1.x	Toasts de notification côté frontend

► Tests et validation

TABLEAU 4.11. Résultats des tests fonctionnels – Itération 5

ID	Cas de test	Résultat attendu	Statut
T-34	Demande publique	HTTP 201, statut NOUVELLE	✓ OK
T-35	Conversion demande	Projet créé, demande VALIDEE	✓ OK
T-36	Créer devis DRAFT	HTTP 201, numéro auto-généré	✓ OK
T-37	Envoyer devis	DRAFT→SENT, client notifié	✓ OK
T-38	Accepter devis	SENT→ACCEPTED→INVOICED, facture créée	✓ OK
T-39	Refuser devis	SENT→REJECTED, raison enregistrée	✓ OK
T-40	Devis expiré	Acceptation refusée, HTTP 400	✓ OK
T-41	Facture PDF	PDF valide généré, téléchargement OK	✓ OK
T-42	Dashboard client	KPI affichés, projet actif en avant	✓ OK
T-43	Stepper client	Progression synchronisée avec statut	✓ OK
T-44	Accès non autorisé	HTTP 403, client ne voit que ses données	✓ OK
T-45	CRUD installateurs	Création, modification, suppression admin	✓ OK
T-46	Invitation client	Email envoyé avec lien d'inscription	✓ OK

La Release 3 finalise la plateforme SolarEase en la dotant de toutes les fonctionnalités nécessaires à une exploitation opérationnelle. Le suivi terrain temps réel permet à l'installateur de communiquer son avancement depuis le chantier. La messagerie et les notifications assurent une collaboration fluide entre les trois rôles (admin, installateur, client). L'espace client offre une expérience dédiée avec un suivi visuel du projet à travers un stepper intuitif. Enfin, le workflow commercial complet — de la demande solaire à la facturation automatique en passant par le devis — transforme SolarEase en une solution professionnelle de gestion de projets solaires, prête pour le marché tunisien.

Chapitre 5

DevOps et mise en production

5

DevOps et mise en production

5.1 Démarche et architecture DevOps cible

5.1.1 Principes retenus

La démarche DevOps appliquée à SolarEase s'appuie sur les cinq piliers du modèle **CALMS** [18] : *Culture, Automation, Lean, Measurement, Sharing*. Concrètement, cela se traduit dans le projet par :

- **Automatisation** – chaque *push* sur le backend déclenche une vérification automatique (build + tests + scans) sans intervention humaine ;
- **Reproductibilité** – les environnements (local, staging, production) sont décrits sous forme de code (Docker, Kustomize) ;
- **Observabilité** – chaque service expose des métriques Prometheus et ses logs sont agrégés dans Loki ;
- **Sécurité « shift-left »** – les secrets sont sortis du dépôt, les scans Trivy et Gitleaks s'exécutent à chaque PR ;
- **Livraison continue** – les branches `develop` et les tags `v*.*.*` déclenchent automatiquement des déploiements backend sur Railway, tandis que le frontend est livré en continu via Vercel.

5.1.2 Vue d'ensemble de la chaîne

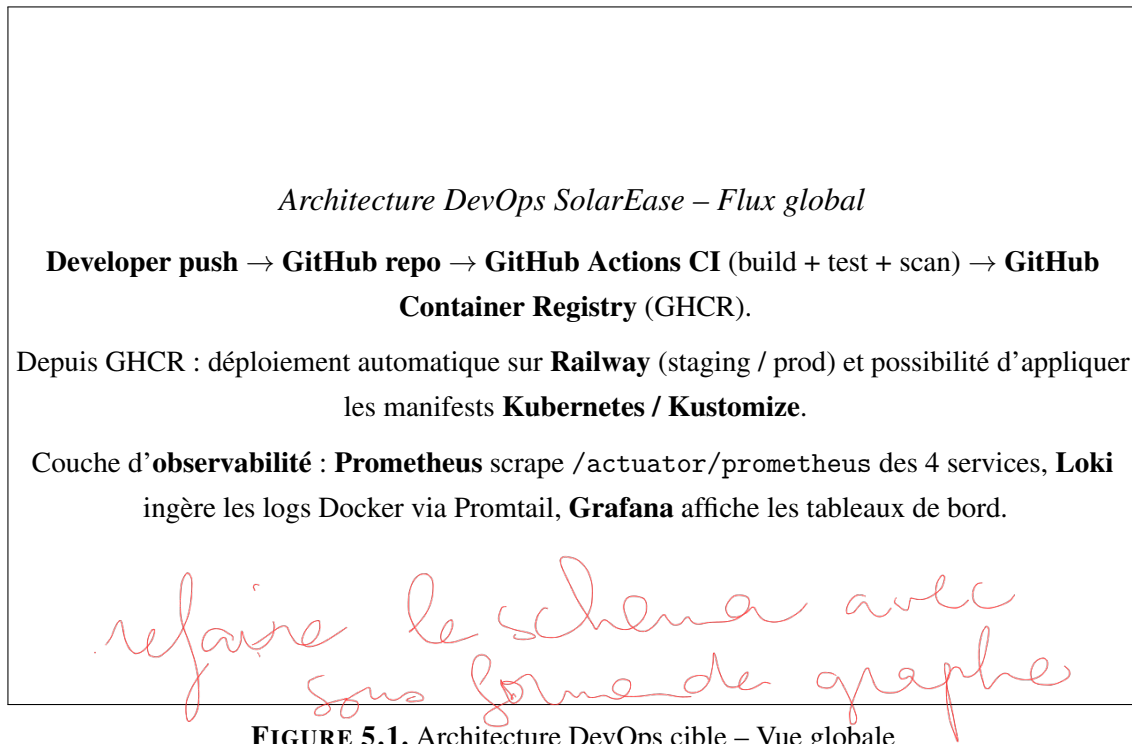


FIGURE 5.1. Architecture DevOps cible – Vue globale

5.2 Containerisation

5.2.1 Stratégie multi-stage

Chaque microservice backend est packagé via un Dockerfile **multi-stage** : une étape de build Maven (maven:3.9.6-eclipse-temurin-17) produit le jar exécutable, puis une étape *runtime* basée sur eclipse-temurin:17-jre-alpine embarque uniquement l'artefact final. Cette approche :

- réduit la taille des images (d'environ **600 Mo** en single-stage à **180 Mo** en multi-stage) ;
- supprime les dépendances de build de l'image finale, ce qui réduit la surface d'attaque ;
- bénéficie du *layer caching* pour des rebuilds rapides.

```

1 # Build stage
2 FROM maven:3.9.6-eclipse-temurin-17 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 COPY src ./src
6 RUN mvn clean package -Dmaven.test.skip=true
7
8 # Run stage

```

```
9 FROM eclipse-temurin:17-jre-alpine
10 WORKDIR /app
11 COPY --from=build /app/target/*.jar app.jar
12 EXPOSE 8081
13 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 5.1 – backend/identity-service/Dockerfile — Multi-stage Maven → JRE

5.2.2 Topologie du docker-compose unifié

Le fichier backend/docker-compose.yml orchestre l'ensemble du stack backend (bases de données + microservices + Ollama + n8n) sur un réseau solarease-net dédié. Tous les secrets sont injectés via un fichier .env (template fourni dans backend/.env.example). Le frontend Vite est démarré séparément (npm run dev) en développement ou servi par Vercel en production.

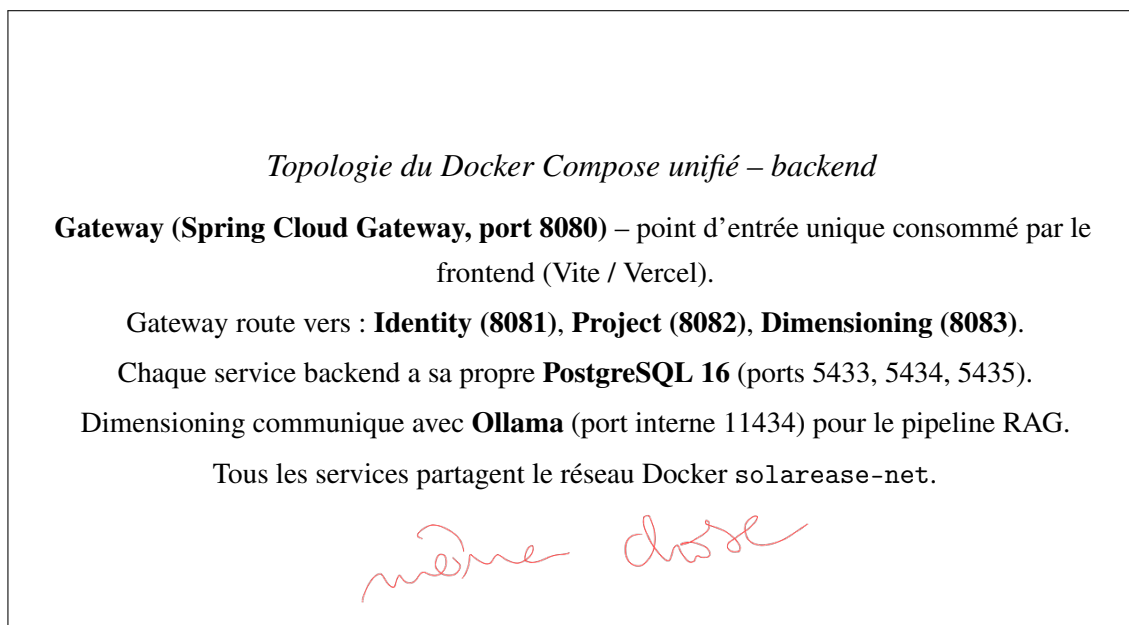


FIGURE 5.2. Topologie Docker Compose – 4 microservices backend + Ollama + n8n

5.3 Pipeline CI/CD GitHub Actions

5.3.1 Workflows mis en place

Le dossier .github/workflows/ contient quatre pipelines :

TABLEAU 5.1. Workflows GitHub Actions du projet SolarEase

Workflow	Déclencheur	Rôle
backend-ci.yml	push / PR sur backend/**	mvn verify en matrice sur les 4 services ; upload des rapports JaCoCo
docker-publish.yml	push main, tags v*	Build multi-arch + push sur GHCR (ghcr.io/<owner>/solarease/<service>)
security-scan.yml	push, PR, cron hebdo	Trivy (filesystem + images) + Gitleaks (détection de secrets)
deploy.yml	push develop (staging), tag v* (prod)	Déploiement Railway via CLI + création de GitHub Release

5.3.2 Workflow CI backend – exécution en matrice

```

1 jobs:
2   build-and-test:
3     runs-on: ubuntu-latest
4     strategy:
5       matrix:
6         service:
7           - gateway-service
8           - identity-service
9           - project-service
10          - dimensioning-service
11     steps:
12       - uses: actions/checkout@v4
13       - uses: actions/setup-java@v4
14         with:
15           java-version: "17"
16           distribution: temurin
17           cache: maven
18       - run: mvn -B -ntp verify
19         working-directory: backend/${{ matrix.service }}
20       - uses: actions/upload-artifact@v4
21         with:
22           name: jacoco-${{ matrix.service }}
23           path: backend/${{ matrix.service }}/target/site/jacoco/

```

Listing 5.2 – .github/workflows/backend-ci.yml (extrait)

5.3.3 Étapes du pipeline CI/CD

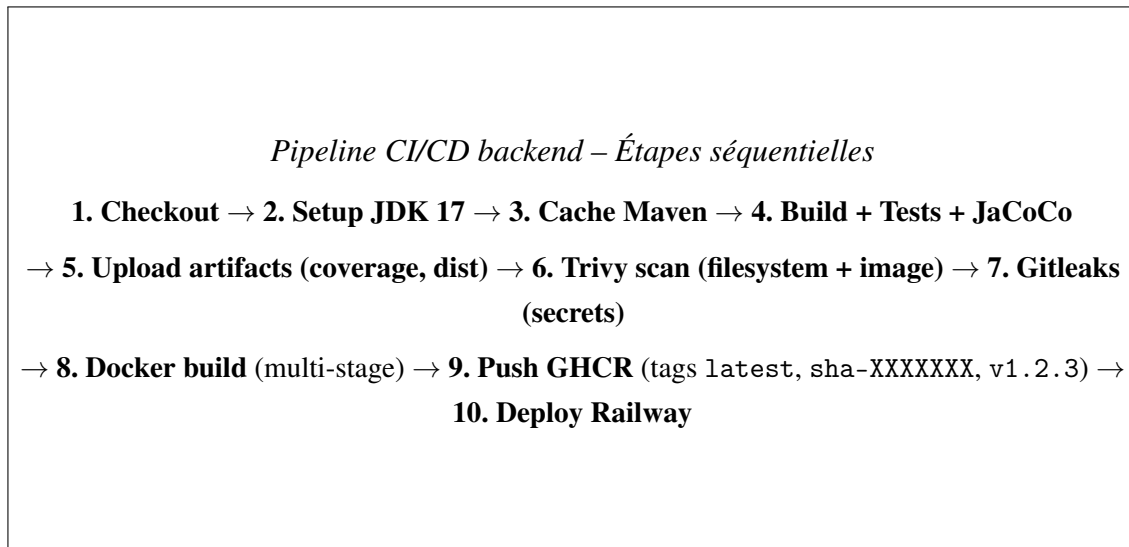


FIGURE 5.3. Pipeline CI/CD – Étapes séquentielles du commit au déploiement

5.3.4 Gestion des dépendances avec Dependabot

Le fichier `.github/dependabot.yml` programme des PR hebdomadaires pour : les 4 `pom.xml` Maven, les 4 `Dockerfile`, et les versions des actions GitHub elles-mêmes. Cela maintient le backend à jour sans intervention manuelle.

5.4 Observabilité (Prometheus + Grafana + Loki)

5.4.1 Exposition des métriques applicatives

Chaque microservice intègre la dépendance `spring-boot-starter-actuator` ainsi que `micrometer-registry-prometheus`. La configuration suivante est ajoutée à chaque `application.properties` :

```

1 management.endpoints.web.exposure.include=health,info,metrics,prometheus
2 management.endpoint.health.show-details=always
3 management.endpoint.health.probes.enabled=true
4 management.health.livenessstate.enabled=true
5 management.health.readinessstate.enabled=true
6 management.metrics.tags.application=${spring.application.name}
7 management.prometheus.metrics.export.enabled=true
  
```

Listing 5.3 – Configuration Actuator commune aux 4 services

Chaque service expose ainsi :

— `/actuator/health` – état global (UP / DOWN);

- /actuator/health/liveness – sonde *liveness* K8s;
- /actuator/health/readiness – sonde *readiness* K8s;
- /actuator/prometheus – métriques au format Prometheus (latence HTTP, mémoire JVM, GC, threads, requêtes par endpoint).

5.4.2 Architecture de la stack observabilité

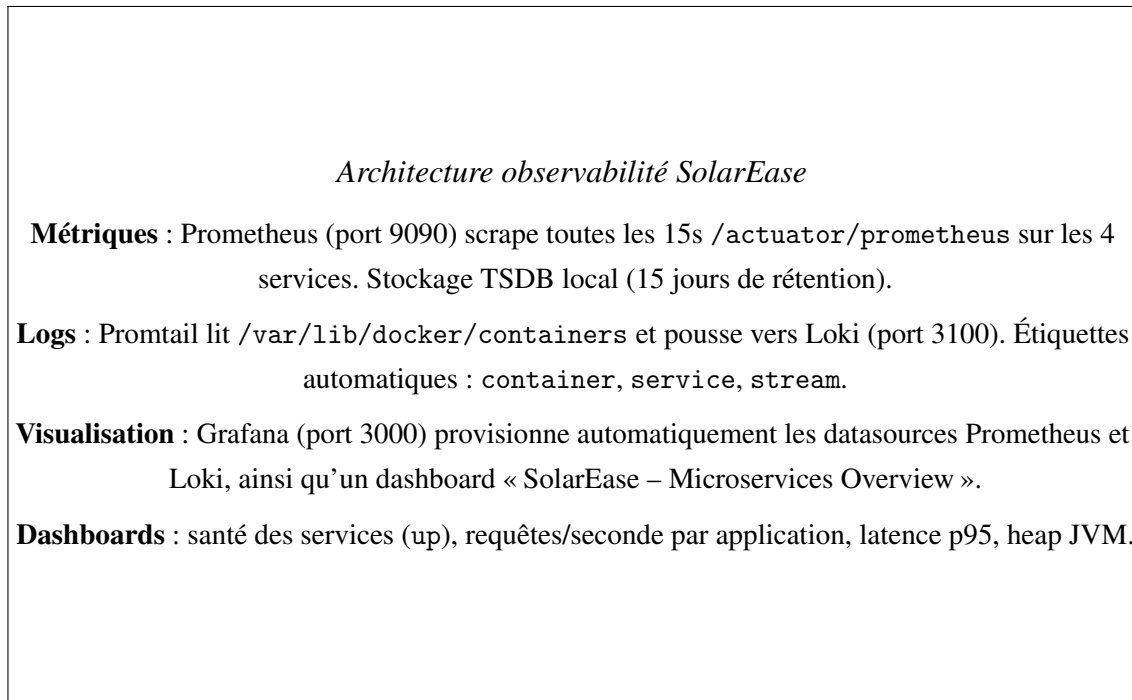


FIGURE 5.4. Architecture observabilité – Prometheus + Loki + Grafana

5.4.3 Stack déployée via Docker Compose

Le fichier backend/observability/docker-compose.observability.yml est *composable* avec le compose principal : il ajoute Prometheus, Loki, Promtail et Grafana au même réseau solarease-net, sans modifier la stack applicative. Démarrage en une commande :

```
1 docker compose \  
2   -f backend/docker-compose.yml \  
3   -f backend/observability/docker-compose.observability.yml \  
4   up -d
```

Listing 5.4 – Démarrage du stack complet (applicatif + observabilité)

5.5 Qualité et sécurité du code

5.5.1 Couverture de tests backend – JaCoCo

Le plugin `jacoco-maven-plugin` (version 0.8.12) est ajouté aux 4 `pom.xml` avec deux exécutions : `prepare-agent` (au démarrage des tests) et `report` (à la phase `verify`). Le service `identity-service` ajoute également une règle de qualité (*quality gate*) avec un seuil minimal de 30 % de lignes couvertes.

5.5.2 Scans de sécurité

Trois scans automatisés s'exécutent dans le workflow `security-scan.yml` :

- **Trivy filesystem** – détecte les vulnérabilités dans les dépendances Maven et npm (sévérités CRITICAL et HIGH) ;
- **Trivy image** – scanne les images Docker construites avant publication ;
- **Gitleaks** – recherche tout secret accidentellement commité dans l'historique Git.

Les résultats sont remontés au format SARIF dans l'onglet *Security* de GitHub.

5.5.3 Gestion des secrets

Tous les secrets ont été extraits du dépôt :

- En local : fichier `backend/.env` (gitignoré), template public dans `backend/.env.example` ;
- En CI : secrets GitHub (`RAILWAY_TOKEN`, `RAILWAY_PROD_TOKEN`, etc.) ;
- En Kubernetes : objet `Secret` `solarease-secrets` injecté par variables d'environnement, dont le *template* figure dans `k8s/base/secret.template.yaml`.

5.6 Infrastructure as Code (Kubernetes + Kustomize)

5.6.1 Organisation des manifests

Le dossier `k8s/` suit le pattern **base + overlays** de Kustomize :

```
1 k8s/
2   base/
3     namespace.yaml
4     configmap.yaml           # variables non sensibles
5     secret.template.yaml     # template (à remplir hors Git)
6     postgres-identity.yaml   # StatefulSet + Service + PVC
```

```

7   postgres-project.yaml
8   postgres-dimensioning.yaml
9   ollama.yaml
10  identity-service.yaml      # Deployment + Service
11  project-service.yaml
12  dimensioning-service.yaml
13  gateway-service.yaml
14  ingress.yaml              # /api et /ws -> gateway
15  kustomization.yaml
16  overlays/
17    dev/ kustomization.yaml  # 1 replica, demo-seed=true
18    prod/ kustomization.yaml # 2-3 replicas, ddl-auto=validate

```

Listing 5.5 – Arborescence du dossier k8s/

5.6.2 Sondes Actuator et limites de ressources

Chaque Deployment déclare trois sondes pointant sur les endpoints Actuator, ce qui rend les rollouts **sûrs** (pas de trafic vers un pod non prêt) et l’auto-réparation **automatique** :

```

1 startupProbe:
2   httpGet: { path: /actuator/health, port: 8081 }
3   failureThreshold: 30
4   periodSeconds: 5
5 readinessProbe:
6   httpGet: { path: /actuator/health/readiness, port: 8081 }
7   initialDelaySeconds: 10
8   periodSeconds: 10
9 livenessProbe:
10  httpGet: { path: /actuator/health/liveness, port: 8081 }
11  initialDelaySeconds: 30
12  periodSeconds: 30
13 resources:
14   requests: { cpu: 200m, memory: 384Mi }
15   limits:   { cpu: 1000m, memory: 768Mi }

```

Listing 5.6 – Sondes Actuator dans un Deployment K8s (extrait)

5.6.3 Routage Ingress

Un Ingress `nginx` unique expose le backend sur un domaine :

- `/api` → `gateway-service:8080`
- `/ws` → `gateway-service:8080` (WebSocket activé)

Le frontend, hébergé sur Vercel, consomme l’Ingress via la variable d’environnement `VITE_API_BASE_URL` qui pointe vers le domaine du backend.

5.6.4 Overlays dev / prod

Les *overlays* appliquent des *patches JSON* sur la base commune :

- **dev** : 1 replica par service, `SOLAREASE_DEMO_SEED=true`, `ddl-auto=update`, namespace `solarease-dev`;
- **prod** : 2 à 3 replicas, `SOLAREASE_DEMO_SEED=false`, `ddl-auto=validate` (migrations Flyway uniquement), namespace `solarease-prod`.

Application en une commande : `kubectl apply -k k8s/overlays/prod`.

5.7 Stratégie de déploiement multi-environnements

5.7.1 Modèle de branches (GitFlow simplifié)

Stratégie GitFlow – 3 branches, 2 environnements automatiques

main (protégée) ← **develop** (intégration) ← **feature/SOLAR-X-<slug>**

Règles :

- toute modification passe par une **Pull Request**;
- la CI doit être **verte** (backend + frontend + security);
- une revue minimale d’un autre développeur est requise sur **main**;
- merge sur **develop** → déploiement automatique **staging**;
- tag `v*.*.*` sur **main** → déploiement **production** + GitHub Release.

FIGURE 5.5. Stratégie GitFlow et environnements automatiques

5.7.2 Environnements et URLs

TABLEAU 5.2. Environnements de déploiement de SolarEase

Environnement	Source	Plateforme	Usage
Local	Docker Compose	Poste développeur	Développement et tests manuels
Staging	Push develop	Railway	Validation avant production
Production	Tag v*.*.*	Railway (& K8s en option)	Live, monitoring 24/7

5.7.3 Procédure de release

```

1 git checkout main
2 git pull --ff-only
3 git merge --no-ff develop
4 git tag -a v1.2.0 -m "Release 1.2.0 – Field tracking + RAG"
5 git push origin main --tags
6 # -> deploy.yml se déclenche automatiquement
7 # -> images Docker poussees sur GHCR
8 # -> services deployes sur Railway production
9 # -> GitHub Release creee avec les notes

```

Listing 5.7 – Procédure standard de mise en production

5.7.4 Procédure de rollback

En cas de régression, le rollback s'effectue en une commande Railway (redéploiement d'une version précédente) ou via Kubernetes (`kubectl rollout undo deployment/<svc> -n solarease-prod`). L'ensemble des procédures opérationnelles (rollback, hotfix, rotation de secrets, backup PostgreSQL, redémarrage Ollama) est documenté dans le fichier `RUNBOOK.md` à la racine du dépôt.

5.8 Tests de validation DevOps

TABLEAU 5.3. Résultats des tests de validation DevOps

ID	Cas de test	Résultat attendu	Statut
D-01	<code>docker compose up</code> avec <code>.env</code> valide	Stack complète UP en <60s	✓ OK
D-02	PR ouverte sur develop	backend-ci vert sur les 4 services	✓ OK
D-03	Push de tag v0.1.0	Images poussées sur GHCR avec 3 tags	✓ OK
D-04	<code>curl localhost:8080/actuator/prometheus</code>	Métriques au format Prometheus	✓ OK
D-05	Cibles Prometheus	4 services UP sur :9090/targets	✓ OK
D-06	Dashboard Grafana « SolarEase Overview »	4 panneaux peuplés en <30s	✓ OK
D-07	Logs containers → Loki	Lignes filtrables par {service="identity"}	✓ OK
D-08	Scan Trivy sur PR	Aucune vulnérabilité CRITICAL non corrigée	✓ OK
D-09	Gitleaks sur PR	Aucun secret détecté	✓ OK
D-10	<code>kubectl apply -k k8s/overlays/dev</code>	Tous les pods passent Ready	✓ OK
D-11	Sondes K8s readiness / liveness	Rollout sans perte de trafic	✓ OK
D-12	Rollback Railway via redeploy	Version précédente restaurée en <2 min	✓ OK

La mise en place de la chaîne DevOps a transformé SolarEase d'un assemblage de microservices en une plateforme déployable, observable et sécurisée. Les automatisations CI/CD garantissent un retour rapide sur chaque modification, l'observabilité offre une vue temps-réel sur la santé du système, l'IaC rend les environnements reproductibles, et la stratégie GitFlow + Railway permet des livraisons fréquentes en toute confiance. Les perspectives ouvertes par cette base technique incluent la mise en place de tests end-to-end automatisés (Playwright), l'ajout d'un service mesh (Istio ou Linkerd) pour la résilience inter-services, et l'intégration d'un système d'alerting (Alertmanager + Slack) sur les métriques critiques.

Conclusion Générale et Perspectives

Synthèse des réalisations

Le projet **SolarEase** a permis de concevoir et de réaliser une plateforme web complète répondant aux besoins des sociétés d'installation de panneaux solaires. Décliné en trois sprints successifs, le travail accompli couvre :

- **Sprint 1 – Authentification** : service d'identité complet avec inscription OTP, connexion JWT, refresh token et gestion du profil ;
- **Sprint 2 – Gestion métier** : service de gestion clients/projets avec cycle de vie contrôlé, pagination et tableau de bord KPI ;
- **Sprint 3 – Intelligence technique** : moteur de dimensionnement (données PVGIS réelles), mode Night Panel avec stockage batterie, comparaison de scénarios, analyse financière sur 25 ans, génération PDF et recommandations IA (Ollama / TinyLLaMA).

Bilan des objectifs atteints

TABLEAU 5.4. Bilan des objectifs du projet SolarEase

Objectif	Atteint	Commentaire
Authentification sécurisée JWT+OTP	✓ Oui	Entièrement fonctionnel
Gestion clients et projets	✓ Oui	CRUD + cycle de vie
Moteur de dimensionnement	✓ Oui	PVGIS + fallback
Innovation Night Panel	✓ Oui	Formules validées
Analyse financière 25 ans	✓ Oui	ROI, payback, cash-flow
Génération rapport PDF	✓ Oui	Téléchargement direct
Recommandation IA	✓ Oui	Ollama en fallback contrôlé
Architecture microservices	✓ Oui	4 services + Docker Compose

Bilan critique

Points forts :

- Architecture moderne, découplée et évolutive ;
- Données de production basées sur PVGIS (source officielle européenne) ;
- Innovation significative avec le mode Night Panel et la couverture nocturne ;
- Expérience utilisateur professionnelle avec deux espaces distincts ;
- Résilience assurée par les mécanismes de fallback (PVGIS et IA).

Limites identifiées :

- Dépendance partielle à des services externes (PVGIS, Ollama) ;
- Paramètres de calibration (prix kWh, taux dégradation) à valider avec chaque société partenaire ;
- Absence de tests de charge exhaustifs en conditions de production.

Perspectives d'évolution

À court terme :

- Intégration d'un **tableau de bord analytique avancé** avec visualisations des économies réalisées ;
- Enrichissement de la **base de connaissances IA** avec des données spécifiques au marché tunisien ;
- Ajout d'un **module de signature électronique** pour les devis.

À moyen terme :

- **Architecture multi-tenant** pour servir plusieurs sociétés d'installation depuis une même instance ;
- **Application mobile** React Native pour les commerciaux terrain ;
- **Observabilité complète** avec Prometheus, Grafana et alertes ;
- **Déploiement cloud** supervisé (Railway, AWS ou Azure) ;
- Connexion aux systèmes de **monitoring en temps réel** des installations existantes.

SolarEase démontre qu'il est possible de développer, en solo, une plateforme professionnelle complète en appliquant rigoureusement les bonnes pratiques d'ingénierie logicielle : architecture microservices, sécurité centralisée, tests automatisés et déploiement conteneurisé. Ce projet constitue une base solide et extensible pour une mise en production réelle dans le secteur des énergies renouvelables.

Bibliographie et Netographie

Ouvrages et articles

- [1] FOWLER, M. (2018). *Microservices Patterns*. Manning Publications.
- [2] NEWMAN, S. (2019). *Building Microservices, 2nd edition*. O'Reilly Media.
- [3] WALLS, C. (2022). *Spring Boot in Action*. Manning Publications.
- [4] BANKS, A. & PORCELLO, E. (2020). *Learning React, 2nd edition*. O'Reilly Media.
- [5] KRUCHTEN, P. (2004). *The Rational Unified Process : An Introduction*, Addison-Wesley.
- [6] SCHWABER, K. & SUTHERLAND, J. (2020). *The Scrum Guide*. Scrum.org.

Documentation technique

- [7] Spring Boot Reference Documentation (3.2.x). <https://docs.spring.io/spring-boot/docs/3.2.x/reference/> [Consulté en mars 2026]
- [8] Spring Security Reference Documentation. <https://docs.spring.io/spring-security/reference/> [Consulté en mars 2026]
- [9] React Documentation. <https://react.dev> [Consulté en février 2026]
- [10] PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/> [Consulté en janvier 2026]
- [11] Docker Compose Documentation. <https://docs.docker.com/compose/> [Consulté en janvier 2026]

APIs et services externes

- [12] PVGIS – Commission Européenne. API de calcul photovoltaïque. https://re.jrc.ec.europa.eu/pvg_tools/en/ [Consulté en février 2026]
- [13] Ollama – LLM local. <https://ollama.ai> [Consulté en mars 2026]
- [14] TinyLLaMA – Modèle de langage compact. <https://github.com/jzhang38/TinyLlama> [Consulté en mars 2026]

- [15] Ministère de l'Industrie, des Mines et de l'Énergie – Plan Solaire Tunisien. <https://www.energymines.gov.tn> [Consulté en mai 2026]
- [16] DevOps Institute – CALMS Model. <https://www.devopsinstitute.com> [Consulté en mai 2026]

Ressources ISET Nabeul

- [17] Département Technologies de l'Informatique, ISET Nabeul. *Guide de rédaction du rapport PFE – Version 2023*.
- [18] Référentiel de la Licence en Développement Systèmes d'Informations, ISET Nabeul, 2025/2026.



Récapitulatif Complet des Endpoints API

A.1 Identity Service (Port 8081)

TABLEAU A.1. Endpoints Identity Service

Méthode	Route	Description	Auth
POST	/api/auth/register	Inscription + OTP	Non
POST	/api/auth/verify-otp	Validation OTP	Non
POST	/api/auth/login	Connexion JWT	Non
POST	/api/auth/refresh	Refresh token	Non
POST	/api/auth/resend-otp	Renvoi OTP	Non
GET	/api/auth/me	Profil utilisateur	JWT
PUT	/api/auth/me	MAJ profil	JWT
PUT	/api/auth/me/password	Changement MD passe	JWT
GET	/api/auth/health	Health check	Non

A.2 Project Service (Port 8082)

TABLEAU A.2. Endpoints Project Service

Méthode	Route	Description	Auth
GET	/api/clients	Liste clients paginée	JWT
POST	/api/clients	Créer un client	JWT
GET	/api/clients/{id}	Détail client	JWT
PUT	/api/clients/{id}	Modifier client	JWT
DELETE	/api/clients/{id}	Supprimer client	JWT
GET	/api/projects	Liste projets	JWT
POST	/api/projects	Créer projet	JWT
GET	/api/projects/{id}	Détail projet	JWT
PATCH	/api/projects/{id}/status	Changer statut	JWT
GET	/api/projects/dashboard/stats	KPI tableau de bord	JWT

A.3 Dimensioning Service (Port 8083)

TABLEAU A.3. Endpoints Dimensioning Service

Méthode	Route	Description
POST	/api/dimensioning/calculate	Calcul standard + Night Panel
POST	/api/dimensioning/compare	Comparaison Classique vs Night Panel
GET	/api/dimensioning/project/{projectId}	Historique par projet
GET	/api/dimensioning/{id}	Résultat par ID
GET	/api/dimensioning/{id}/pdf	Télécharger rapport PDF
GET	/api/equipment	Liste catalogue équipements
GET	/api/equipment/{id}	Détail équipement
GET	/api/equipment/type/{type}	Par type (SOLAR_PANEL, INVERTER...)

B

Matrice de Tests Complète

TABLEAU B.1. Matrice de tests SolarEase

ID	Module	Précondition	Résultat attendu	Statut
T01	Auth/Register	Email valide, pwd fort	Compte créé, OTP envoyé	✓ OK
T02	Auth/OTP	OTP correct et non expiré	Compte activé, JWT retourné	✓ OK
T03	Auth/Login	Compte activé, credentials valides	Access + Refresh token	✓ OK
T04	Auth/Login	Mot de passe invalide	Rejet 401	✓ OK
T05	Auth/Refresh	Token valide	Nouveau access token	✓ OK
T06	Auth/Refresh	Token expiré	Session invalidée	✓ OK
T07	Client/Create	Données complètes	Client créé 201	✓ OK
T08	Client/Create	Champ obligatoire manquant	Rejet 400 explicite	✓ OK
T09	Client/List	Utilisateur authentifié	Liste paginée	✓ OK
T10	Client/Update	Client existant	Données mises à jour	✓ OK
T11	Client/Delete	Client existant	Client supprimé	✓ OK
T12	Project/Create	Client existant	Projet CREATED	✓ OK
T13	Project/Status	Projet CREATED	Passage IN_PROGRESS	✓ OK
T14	Project/Stats	Projets existants	KPI calculés	✓ OK

ID	Module	Précondition	Résultat attendu	Statut
T15	Dim/Standard	Paramètres cohérents	Résultat complet	✓ OK
T16	Dim/NightPanel	nightPanelId présent	Stockage calculé	✓ OK
T17	Dim/Compare	Données complètes	Tableau comparatif + courbes	✓ OK
T18	Dim/PVGIS	Coordonnées GPS valides	Production PVGIS utilisée	✓ OK
T19	Dim/Fallback	PVGIS indisponible	Calcul statique activé	✓ OK
T20	Dim/PDF	Simulation existante	PDF téléchargeable	✓ OK
T21	IA/Ollama	LLM actif, contexte complet	Recommandation reçue	✓ OK
T22	IA/Fallback	Ollama arrêté	Message dégradé, pas de crash	✓ OK
T23	Gateway/Valid	Token JWT valide	Routage autorisé	✓ OK
T24	Gateway/Invalid	Token absent ou invalide	Rejet 401	✓ OK

C

Configuration Docker Compose

```
1 services:
2   postgres-identity:
3     image: postgres:16-alpine
4     environment:
5       POSTGRES_DB: solarease_identity
6       POSTGRES_USER: postgres
7       POSTGRES_PASSWORD: postgres
8     ports:
9       - "5433:5432"
10
11   identity-service:
12     build: ./identity-service
13     environment:
14       PORT: 8081
15       SPRING_DATASOURCE_URL: jdbc:postgresql://
16         postgres-identity:5432/solarease_identity
17       APP_JWT_SECRET: ${JWT_SECRET}
18     depends_on:
19       postgres-identity:
20         condition: service_healthy
21     ports:
22       - "8081:8081"
23
24   dimensioning-service:
25     build: ./dimensioning-service
26     environment:
27       PORT: 8083
28       OLLAMA_BASE_URL: http://ollama:11434
29       OLLAMA_MODEL: tinyllama:latest
30     depends_on:
31       - postgres-dimensioning
32       - ollama
33     ports:
34       - "8083:8083"
35
36   ollama:
```

```
37     image: ollama/ollama:latest
38     volumes:
39         - ollama_data:/root/.ollama
40
41 gateway-service:
42     build: ./gateway-service
43     environment:
44         PORT: 8080
45         JWT_SECRET: ${JWT_SECRET}
46         IDENTITY_SERVICE_URL: http://identity-service:8081
47         PROJECT_SERVICE_URL: http://project-service:8082
48         DIMENSIONING_SERVICE_URL: http://dimensioning-service:8083
49     ports:
50         - "8080:8080"
```

Listing C.1 – docker-compose.yml – Extrait principal

Résumé

Résumé

Le secteur des énergies renouvelables, et particulièrement le photovoltaïque, connaît une croissance soutenue en Tunisie dans le cadre du Plan Solaire Tunisien. Pourtant, les sociétés d'installation de panneaux solaires peinent encore à industrialiser leurs processus commerciaux et techniques, faute d'un outil unifié et adapté à leurs besoins métiers.

Le présent Projet de Fin de Parcours présente la conception et la réalisation de **SolarEase**, une plateforme web intelligente dédiée aux installateurs photovoltaïques. La solution centralise l'intégralité du cycle de vie d'un projet solaire : authentification sécurisée par **JWT et OTP**, gestion des clients et des projets, calcul automatisé de dimensionnement intégrant l'API **PVGIS** pour des données d'irradiance réelles, comparaison de scénarios *Classique versus Night Panel* avec stockage batterie, analyse financière sur 25 ans, génération de rapports **PDF** professionnels et aide à la décision par **Intelligence Artificielle** (LLM Ollama / TinyLLaMA en mode local).

L'architecture adoptée est de type **microservices** (Spring Boot 3.2 / Java 17), organisée en quatre services indépendants — *Identity, Project, Dimensioning* et *Gateway* — chacun disposant de sa propre base de données **PostgreSQL 16**. Le frontend est développé en **React 18 / TypeScript** avec Vite et TailwindCSS ; le déploiement est orchestré via **Docker Compose**.

Les résultats valident la faisabilité technique et opérationnelle de la solution : temps de traitement réduit, traçabilité accrue et propositions commerciales de qualité professionnelle.

Mots-clés : photovoltaïque · microservices · Spring Boot · React · dimensionnement solaire · PVGIS · Night Panel · intelligence artificielle · Docker

Abstract

The renewable energy sector, particularly photovoltaics, is experiencing sustained growth in Tunisia as part of the national Solar Plan. However, solar panel installation companies still struggle to industrialize their commercial and technical processes due to the lack of an integrated tool tailored to their specific business needs.

This final-year project presents the design and implementation of **SolarEase**, an intelligent web platform dedicated to photovoltaic installers. The solution centralizes the complete lifecycle of a solar project : **JWT and OTP**-based secure authentication, customer and project management, automated sizing calculations integrating the **PVGIS** API for real irradiance data, Classic vs. Night Panel scenario comparison with battery storage, 25-year financial analysis, professional **PDF** report generation, and **AI**-assisted decision support (Ollama / TinyLLaMA LLM running locally).

The adopted architecture is **microservices**-based (Spring Boot 3.2 / Java 17), structured into four independent services — *Identity*, *Project*, *Dimensioning* and *Gateway* — each with its own dedicated **PostgreSQL 16** database. The frontend is built with **React 18 / TypeScript**, Vite and TailwindCSS ; deployment is orchestrated via **Docker Compose**.

The results validate the technical and operational feasibility of the solution : reduced processing time, enhanced traceability, and professional-quality commercial proposals.

Keywords : photovoltaics · microservices · Spring Boot · React · solar sizing · PVGIS · Night Panel · artificial intelligence · Docker